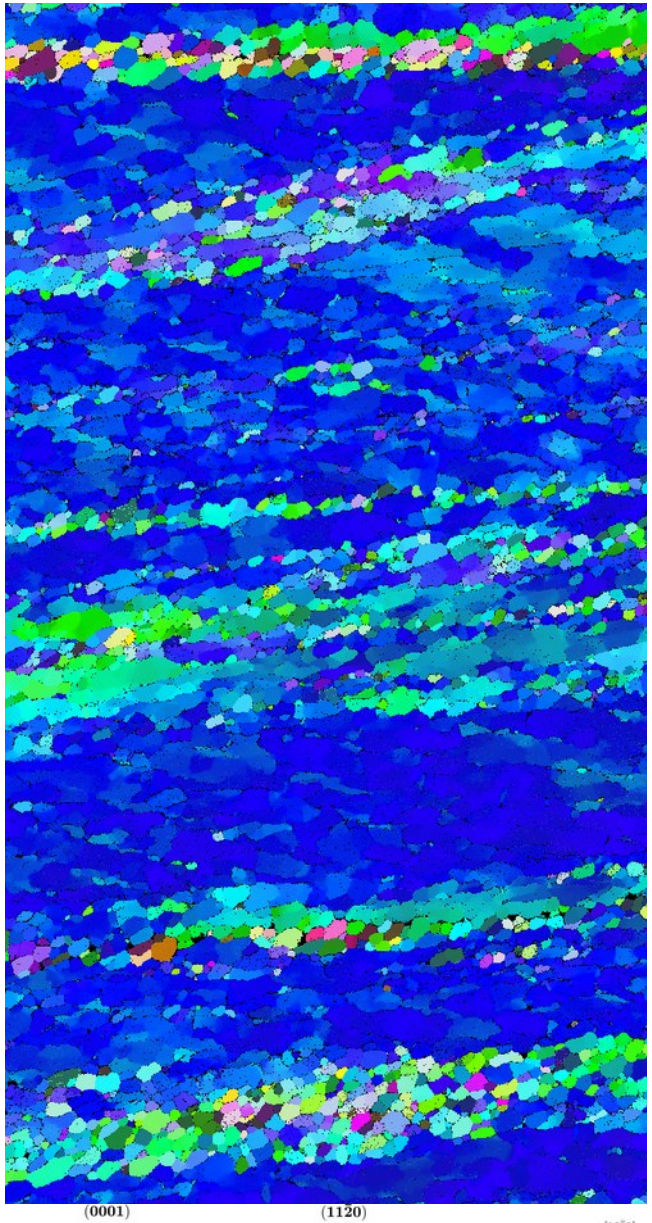
Lecture 1 - EBSD

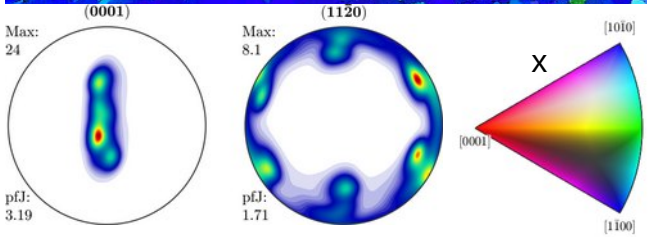
Rüdiger Kilian

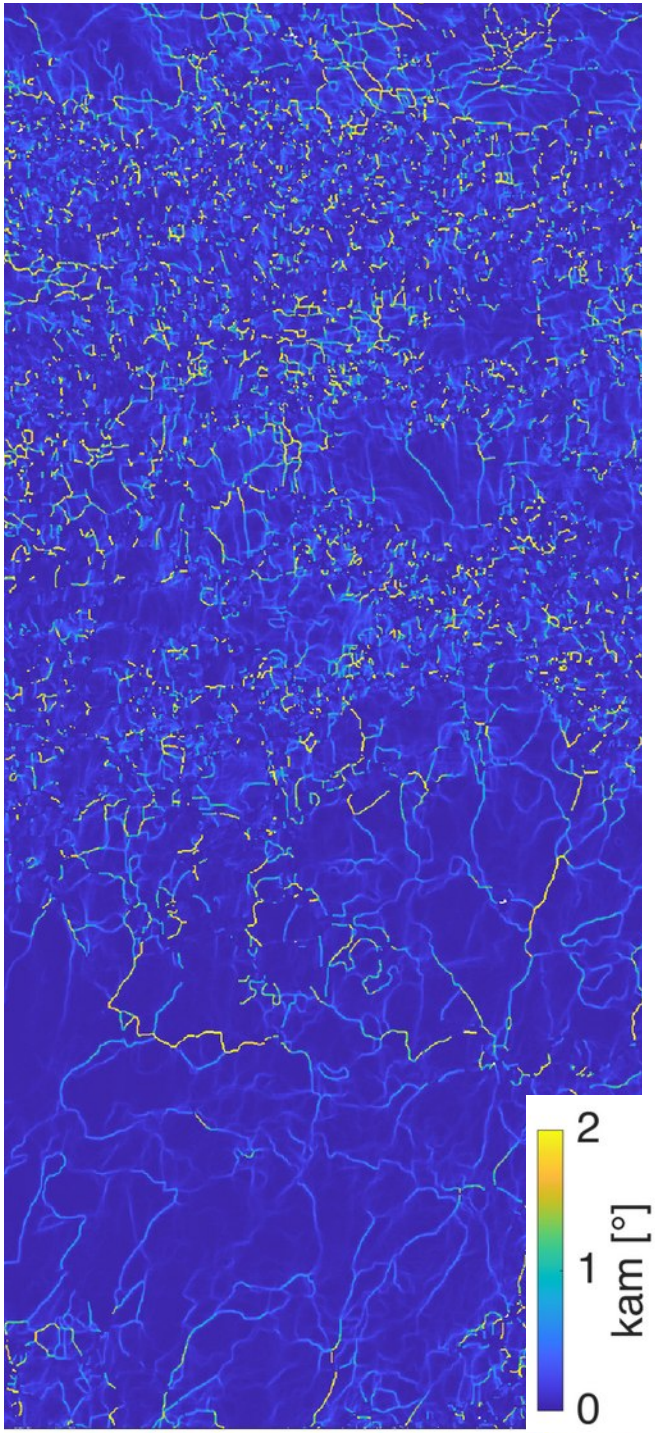
Institute of Geosciences and Geography
Martin-Luther University Halle, Germany

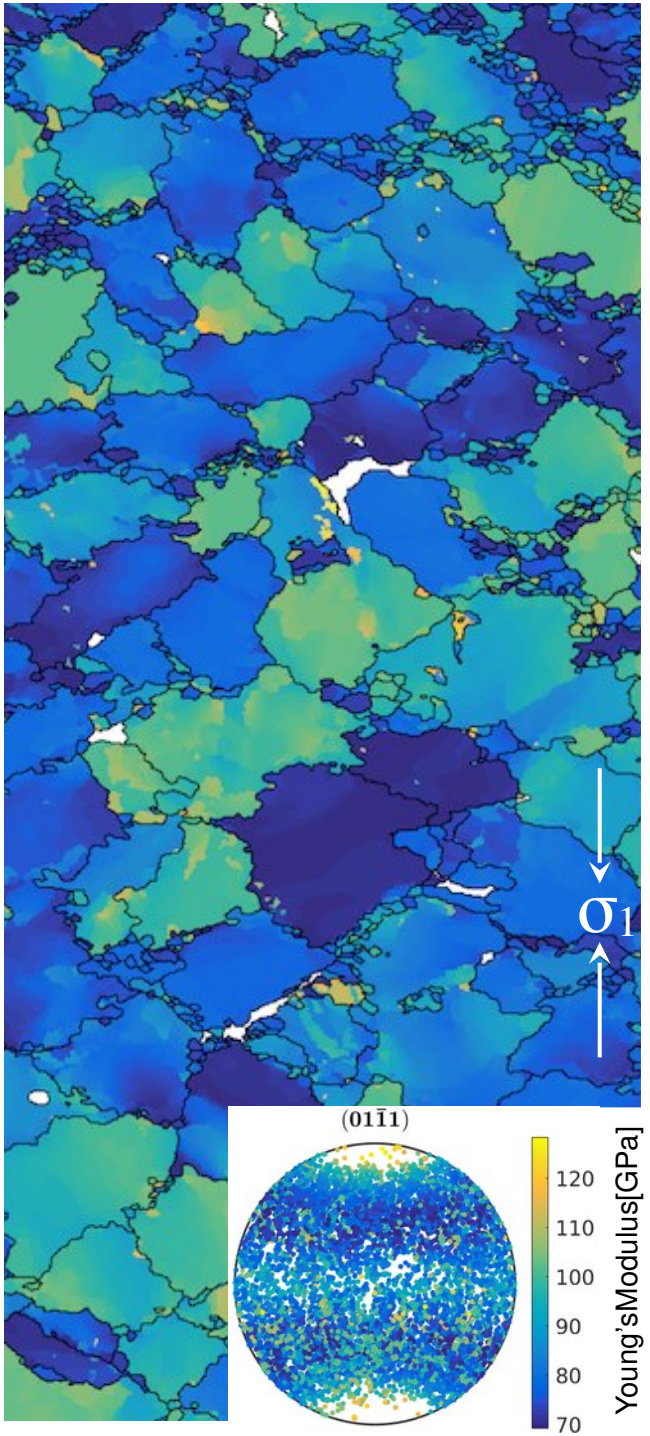
MTEX Workshop 2024

What’s next after data acquisition?









1. EBSD - content

- EBSD
  - general concepts
  - importing data (very short version)
- accessing/ plotting EBSD data
  - property maps
  - color coding
- selecting EBSD
  - by phase
  - by property
  - condition
  - by location
- gridify
  - EBSDsquare, EBSDhex
- importing data (long version)
  - formats
  - reference frames

1. EBSD/data import

minimal example:

```
ebsd = EBSD.load(infile);
% or
ebsd = EBSD.load(infile,'interface','h5oina');
% or
ebsd = loadEBSD_h5oina(infile);
```

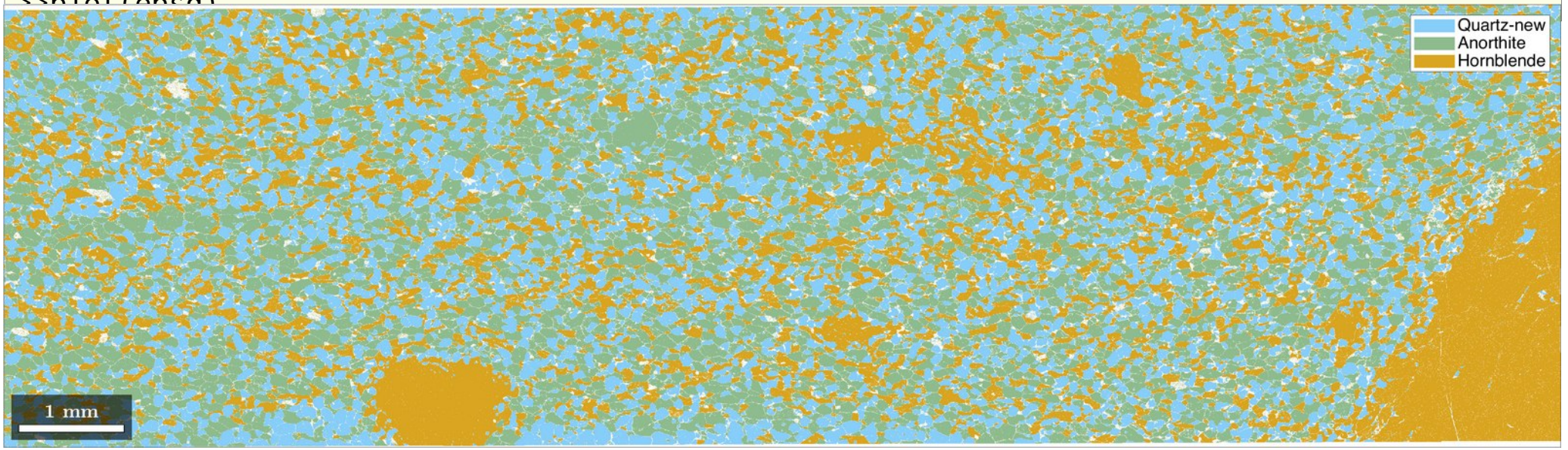
1) Mtex will most likely guess the file format

result:

```
ebsd = EBSD
 Phase   Orientations     Mineral         Color  Symmetry  Crystal reference frame
     0  495637 (9.7%)  notIndexed
     1  1503511 (30%)  Quartz-new  LightSkyBlue       321        X||a*, Y||b, Z||c
     2  1700713 (33%)   Anorthite  DarkSeaGreen        -1             X||a*, Z||c
     3  1390665 (27%)  Hornblende     Goldenrod     12/m1       X||a*, Y||b*, Z||c

 Properties: x, y, bc, bs, bands, MAD, quality, Al_Wt_, Ca_Wt_, Fe_Wt_, K_Wt_, Mg_Wt_,...
 Scan unit : um
 Header: show struct
 Images: show struct
```

1. EBSD/data import

```
ebsd = EBSD
 Phase   Orientations     Mineral         Color  Symmetry  Crystal reference frame
     0  495637 (9.7%)  notIndexed
     1  1503511 (30%)  Quartz-new  LightSkyBlue       321        X||a*, Y||b, Z||c
     2  1700713 (33%)   Anorthite  DarkSeaGreen        -1              X||a*, Z||c
     3  1390665 (27%)  Hornblende     Goldenrod     12/m1       X||a*, Y||b*, Z||c

 Properties: x, y, bc, bs, bands, MAD, quality, Al_Wt_, Ca_Wt_, Fe_Wt_, K_Wt_, Mg_Wt_,...
 Scan unit : um
 Header: show struct
 Images: show struct

>>plot(ebsd)
```



1. EBSD – properties and representation

- in Matlab the EBSD object is a class, representing a list of measurements, properties and available methods

```
>>ebsd
ebsd = EBSD
 Phase   Orientations     Mineral         Color  Symmetry  Crystal reference frame
     0  495637 (9.7%)  notIndexed
     1  1503511 (30%)  Quartz-new  LightSkyBlue       321        X||a*, Y||b, Z||c
     2  1700713 (33%)   Anorthite  DarkSeaGreen        -1              X||a*, Z||c
     3  1390665 (27%)  Hornblende     Goldenrod     12/m1       X||a*, Y||b*, Z||c

 Properties: x, y, bc, bs, bands, MAD, quality, Al_Wt_, Ca_Wt_, Fe_Wt_, K_Wt_, Mg_Wt_,...
 Scan unit : um
 Header: show struct
 Images: show struct

>>ebsd.prop
ans =
  struct with fields:
          x: [5090526×1 double]
          y: [5090526×1 double]
         bc: [5090526×1 double]
         bs: [5090526×1 double]
       ...
```

1. EBSD – properties and representation

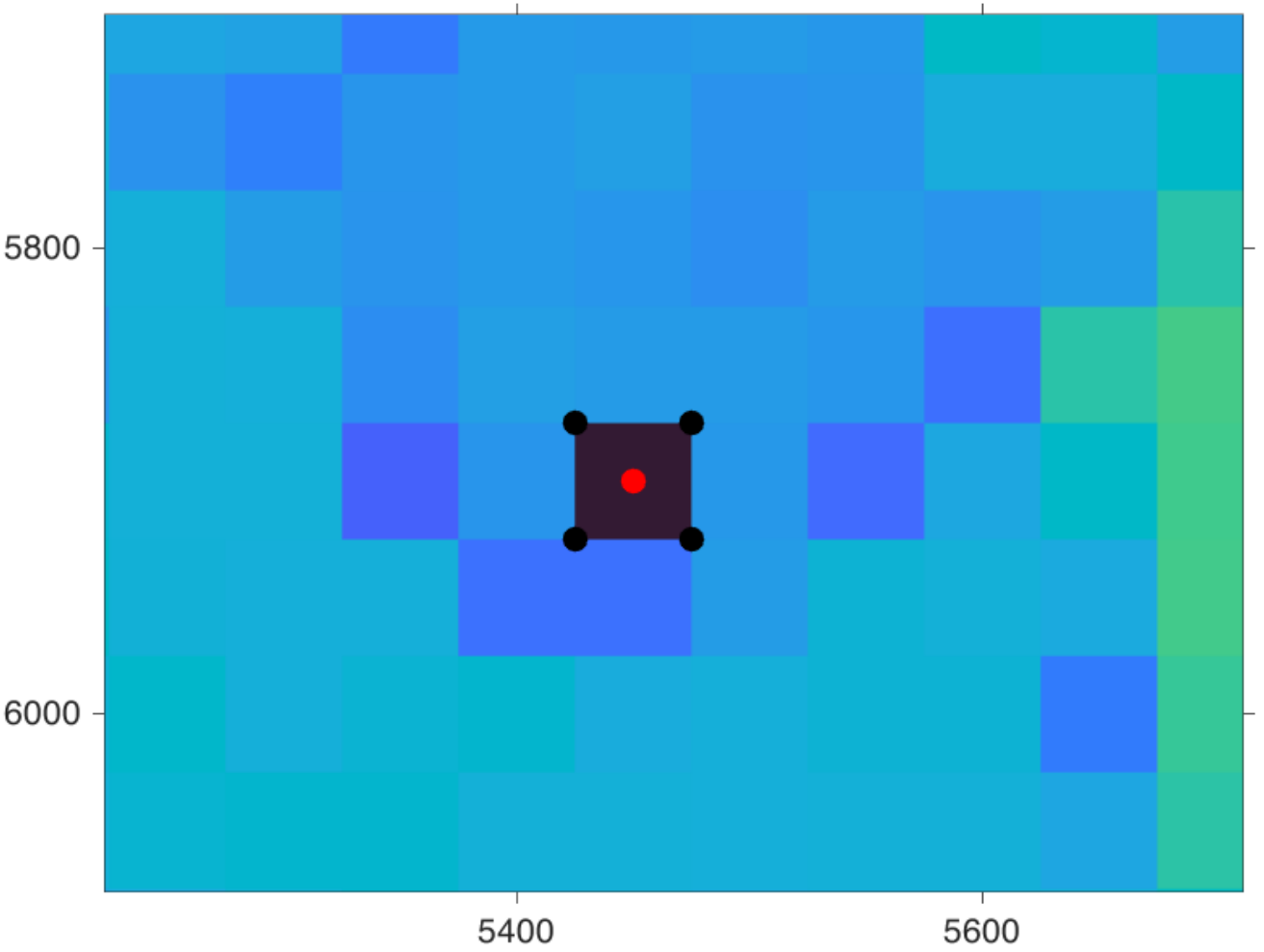
- in Matlab the EBSD object is a class, representing a list of measurements, properties and available methods

```
>>ebsd.prop
ans =
  struct with fields:
           x: [5090526×1 double]
           y: [5090526×1 double]
          bc: [5090526×1 double]
          bs: [5090526×1 double]
       bands: [5090526×1 double]
         MAD: [5090526×1 double]
     quality: [5090526×1 double]
      Al_Wt_: [5090526×1 double]
      Ca_Wt_: [5090526×1 double]
     ...
>> ebsd.prop.Al_Wt_(8000:8004)
ans =
    5.5815
    9.8317
    8.5278
    7.8465
```

1. EBSD – spatial coordinates

- each point/pixel is represented by a unitCell: coordinates of vertices relative to x, y and z

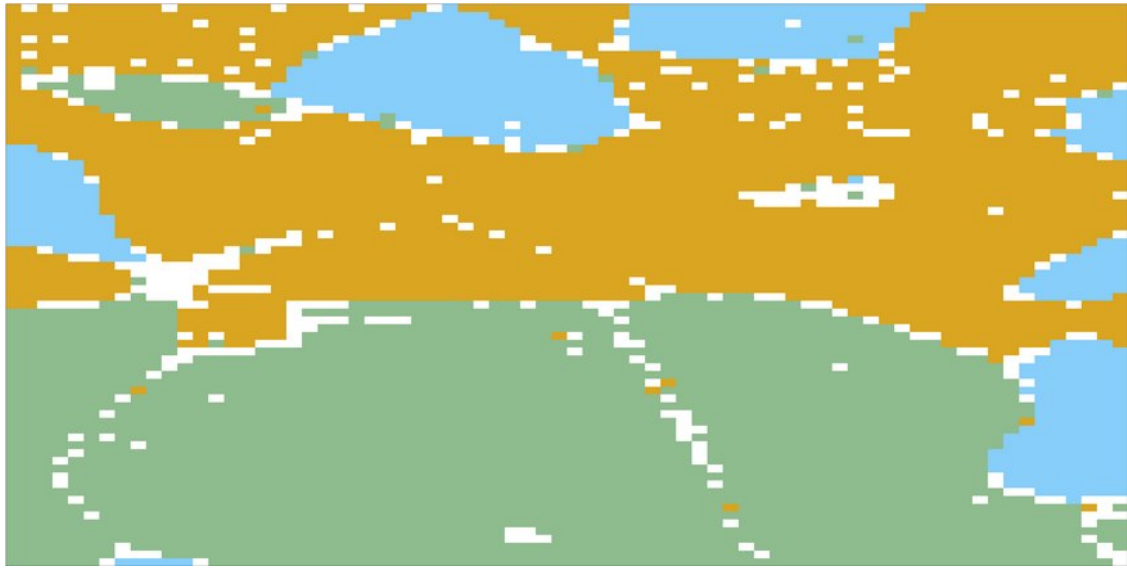
```
>> ebsd.unitCell

ans = vector3d
 size: 4 x 1
      x     y     z
   1.75  1.75     0
  -1.75  1.75     0
  -1.75 -1.75     0
   1.75 -1.75     0
```



1. EBSD – spatial coordinates

- each point/pixel is represented by a unitCell: coordinates of vertices relative to x, y, z
- change coordinates

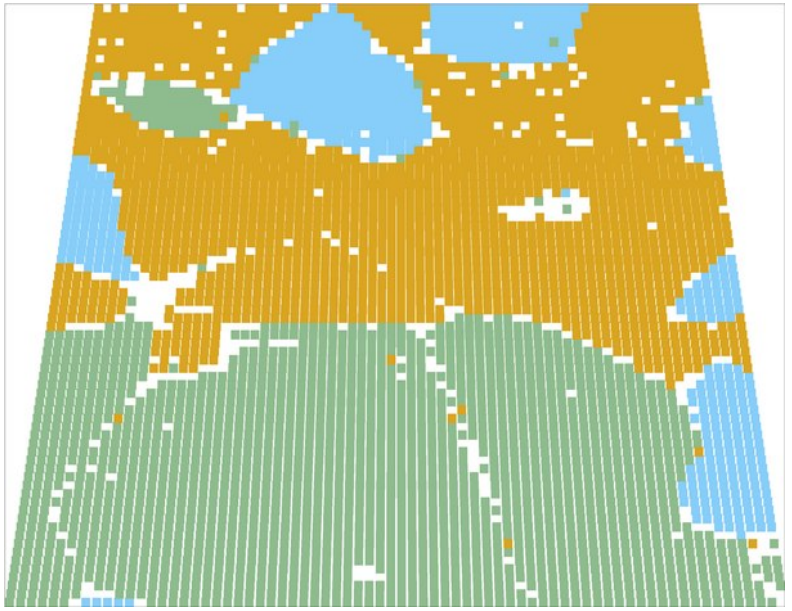
```
>> ebsd.x = ebsd.x * 2;
>> ebsd.unitCell


ans = vector3d
 size: 4 x 1
     x     y     z
   3.5  1.75     0
  -3.5  1.75     0
  -3.5 -1.75     0
   3.5 -1.75     0
```

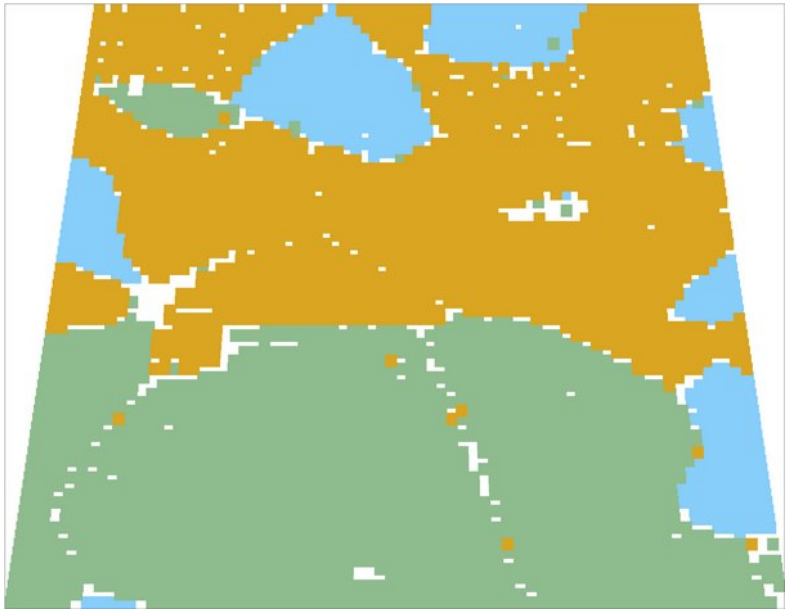


1. EBSD – spatial coordinates

- each point/pixel is represented by a unitCell: coordinates of vertices relative to x, y, z
- change x,y coordinates

```
>> ebsd.x = new_x_coordinates;
>> plot(ebsd)
>> ebsd.unitCell = ebsd.unitCell.*1.3;
>> plot(ebsd)
```





1. EBSD – properties

- in Matlab the EBSD object is a class, representing a list of measurements, properties and available methods

```
>> ebsd.rotations

ans = rotation (show methods, plot)
  size: 245952 x 1



>> ebsd.CSList

ans =

  1×5 cell array

  Columns 1 through 3

    {'notIndexed'}    {1×1 crystalSymmetry}    {1×1 crystalSymmetry}

  Columns 4 through 5

    {1×1 crystalSymmetry}    {1×1 crystalSymmetry}
```

1. EBSD – properties

- in Matlab the EBSD object is a class, representing a list of measurements, properties and available methods

```
>> ebsd.rotations(177)

ans = rotation (show methods, plot)

  Bunge Euler angles in degree
     phi1     Phi    phi2    Inv.
  141.158  85.158  37.895       0



>> ebsd.CSList{3}

ans = crystalSymmetry

  mineral           : Anorthite
  color             : DarkSeaGreen
  symmetry          : -1
  elements          : 2
  a, b, c           : 8.2, 13, 14
  alpha, beta, gamma: 93.172°, 115.952°, 91.222°
  reference frame   : X||a*, Z||c
```

1. EBSD – properties and representation

- in Matlab the EBSD object is a class, representing a list of measurements, properties and available methods

```
>> ebsd.KAM

ans =

    0.0037
    0.0030
    0.0056
...



>> ebsd.inpolygon([5 10 100 100])

...
0
0
0
0
0
...
```

```
>> ls([mtex_path '/EBSDAnalysis/@EBSD'])
calcGrains.m
calcGROD.m
calcMisorientation.m
calcTensor.m
calcTraces.m
cat.m
char.m
curvature.m
display.m
EBSD.m
export_ang.m
export_crc.m
export_ctf.m
export_h5.m
export.m
extent.m
fillByGrainId.m
fill.m
findByLocation.m
findByOrientation.m
getCPRInfo.m
...
```

1. EBSD – selecting EBSD: by phase

```
>> ebsd('Hornblende')

ans = EBSD
 Phase    Orientations     Mineral      Color  Symmetry  Crystal reference frame
     3  1390665 (100%)  Hornblende  Goldenrod     12/m1       X||a*, Y||b*, Z||c

...




>> ebsd({'Hornblende','Anorthite'})

ans = EBSD

 Phase   Orientations     Mineral         Color  Symmetry  Crystal reference frame
     2  1700713 (55%)   Anorthite  DarkSeaGreen        -1              X||a*, Z||c
     3  1390665 (45%)  Hornblende     Goldenrod     12/m1       X||a*, Y||b*, Z||c

...
```

1. EBSD – selecting EBSD: by phase

```
>> ebsd('Hornblende')

ans = EBSD
 Phase    Orientations     Mineral      Color  Symmetry  Crystal reference frame
     3  1390665 (100%)  Hornblende  Goldenrod     12/m1       X||a*, Y||b*, Z||c

...
```

using:
‘Hornblende’, ‘hornblende’,’horn’,’ho’, ‘h’
selects the hornblende phase

what does not work if you have e.g.
‘Titanium a’ and ‘Titanium b’:
‘T’, ‘Tita’ or ‘Titanium’
selects both phases

don’t use a phase name which is a substrings of another phase:
‘Kryptonite’ and ‘Krypto’

```
>> ebsd({'Hornblende','Anorthite'})

ans = EBSD

 Phase   Orientations     Mineral         Color
     2  1700713 (55%)   Anorthite  DarkSeaGreen
     3  1390665 (45%)  Hornblende     Goldenrod

...
```

1. EBSD – extracting orientations

- orientation is only defined for a phase (with a crystal Symmetry)

```
>> ebsd.orientations
Error using phaseList/checkSinglePhase

------------------------------------------------------------------
Your variable contains the phases: Quartz-new, Anorthite

However, you are executing a command that is only
permitted for a single phase!

Please read the chapter "select EBSD data" for how to
restrict EBSD data or grains to a single phase.
------------------------------------------------------------------

Error in phaseList/get.CS (line 154)
      id = checkSinglePhase(pL);
...

>> ebsd('h').orientations

ans = orientation (Hornblende → xyz)
  size: 1390665 x 1
```

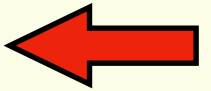
1. EBSD – selecting EBSD: by phase

- the ‘notIndexed’ phase and non-existent points

```
>> ebsd('notIndexed')

ans = EBSD

 Phase   Orientations     Mineral  Color  Symmetry  Crystal reference frame
     0  495637 (100%)  notIndexed
 Properties: x, y, bc, bs, bands, MAD, quality, Al_Wt_, Ca_Wt_, Fe_Wt_, ...
 Scan unit : um
```

- two types of points which do not appear in a map:
  1) ‘notIndexed’ is a point KNOWN to be not indexed
  2) points missing in the list (due to data format or users choice to delete points)
- relevant during grain reconstruction

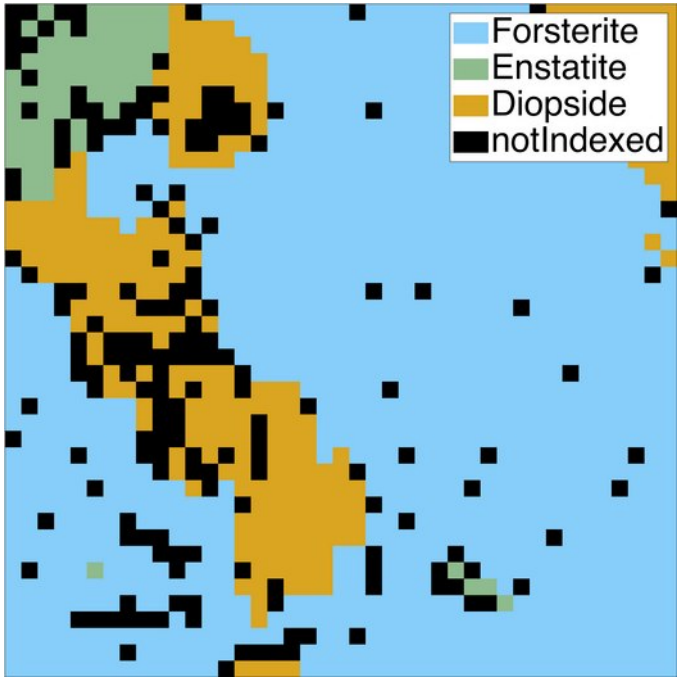
x	y	phase
0	0	1
1	0	1
2	0	1
3	0	0
4	0	1
5	0	1
6	0	1
8	0	1
9	0	1
10	0	1
...		

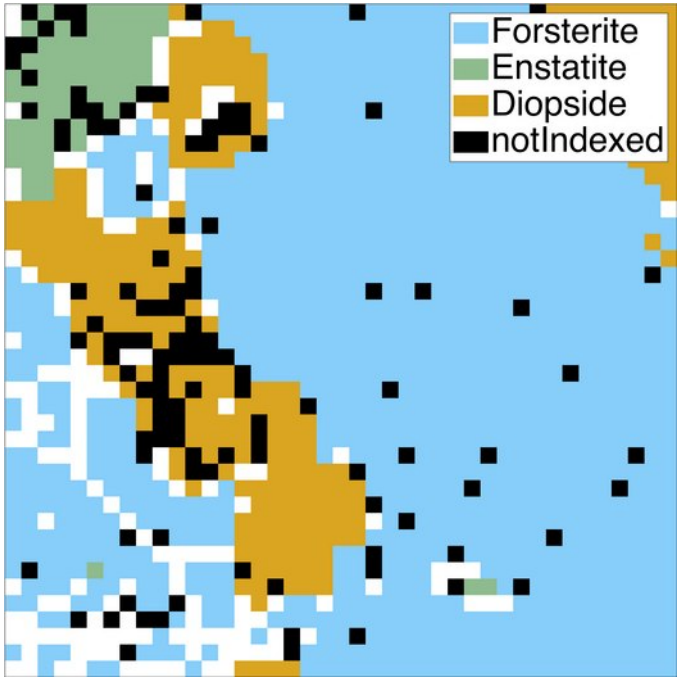


1. EBSD – selecting EBSD: by phase

- the ‘notIndexed’ phase and non-existent points

```
ebsd

     0    338 (6.5%)  notIndexed
     1     659 (13%)  Quartz-new
     2    2124 (41%)   Anorthite
     3    2063 (40%)  Hornblende

plot(ebsd);


% after deleting some points

     0    112 (2.5%)  notIndexed
     1     622 (14%)  Quartz-new
     2    1936 (43%)   Anorthite
     3    1874 (41%)  Hornblende


plot(ebsd);
hold on
plot(ebsd('notIndexed'),'faceColor','k')
hold off
```

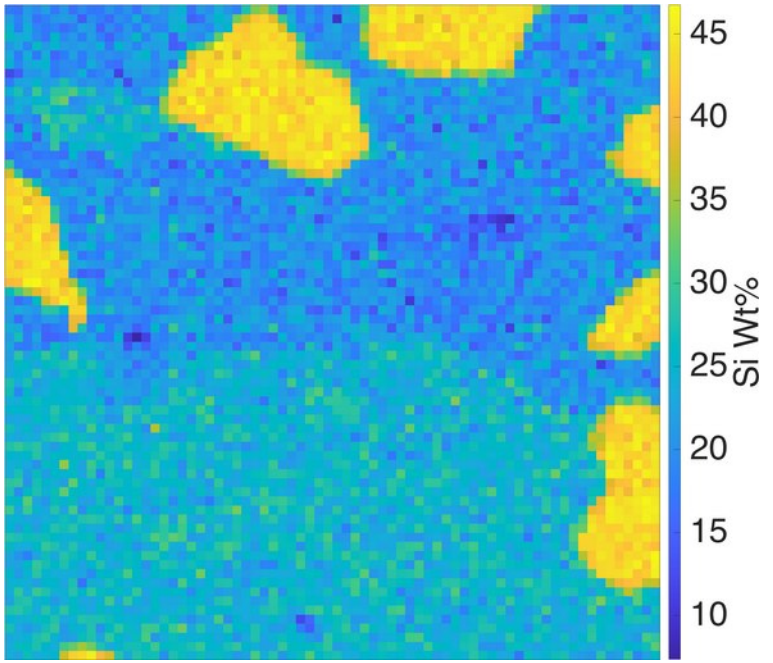


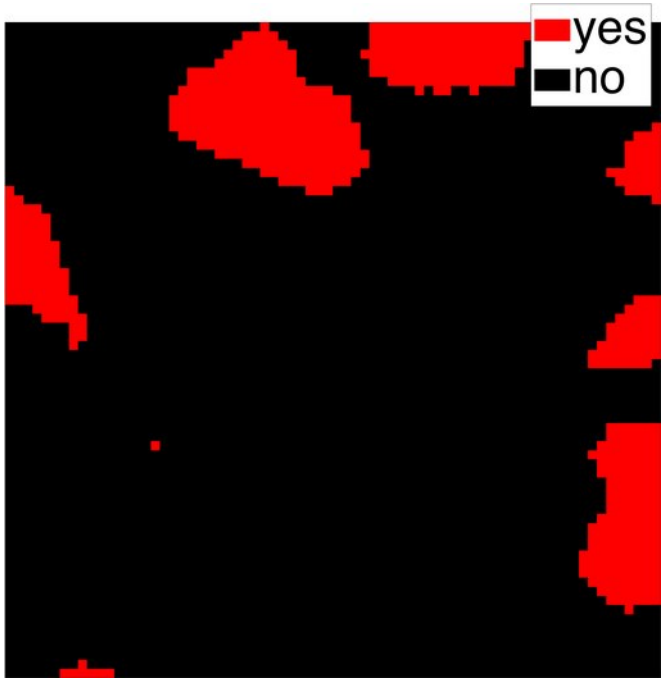


1. EBSD – selecting EBSD: by logical

- 0 or 1, true or false

```
>> ebsd.Si_Wt_  % or ebsd.prop.Si_Wt_

   24.2750
   25.0223
   40.0746
   28.6914
   23.1058

>> ebsd.Si_Wt_ > 35

   0
   0
   1
   0
   0

>> ebsd(ebsd.Si_Wt_ > 35)

 Phase  Orientations      Mineral
     0     36 (5.3%)  notIndexed
     1     632 (93%)  Quartz-new
     2     4 (0.59%)   Anorthite
     3        7 (1%)  Hornblende
...
```





1. EBSD – selecting EBSD: by logical

- by a logical (0 or , true or false)

```
>> logical([1 1 0 0]) & logical([1 0 0 1])

ans =

  1×4 logical array

   1   0   0   0



>> ebsd(ebsd.Si_Wt_ > 35 & ebsd.phase ~= 1)


ans = EBSD

 Phase  Orientations     Mineral
     0      36 (77%)  notIndexed
     2      4 (8.5%)   Anorthite
     3       7 (15%)  Hornblende
```

```
&    AND
|    OR

<,>  smaller, larger
<=   smaller or equal
==   equal
~=   not equal
```

1. EBSD – selecting EBSD: by index, id

- by index : position in the list

```
>> ebsd(111)

ans = EBSD (show methods, plot)

 Phase  Orientations     Mineral         Color  Symmetry  Crystal reference frame
     1      1 (100%)  Forsterite  LightSkyBlue       mmm

  Id   Phase   phi1   Phi   phi2   bands    bc    bs   error   mad      x   y
 111       1     58   162    336       7   129   202       0   0.2   5500   0
 Scan unit : um
```

- by id : uniquely assigned ID of each point

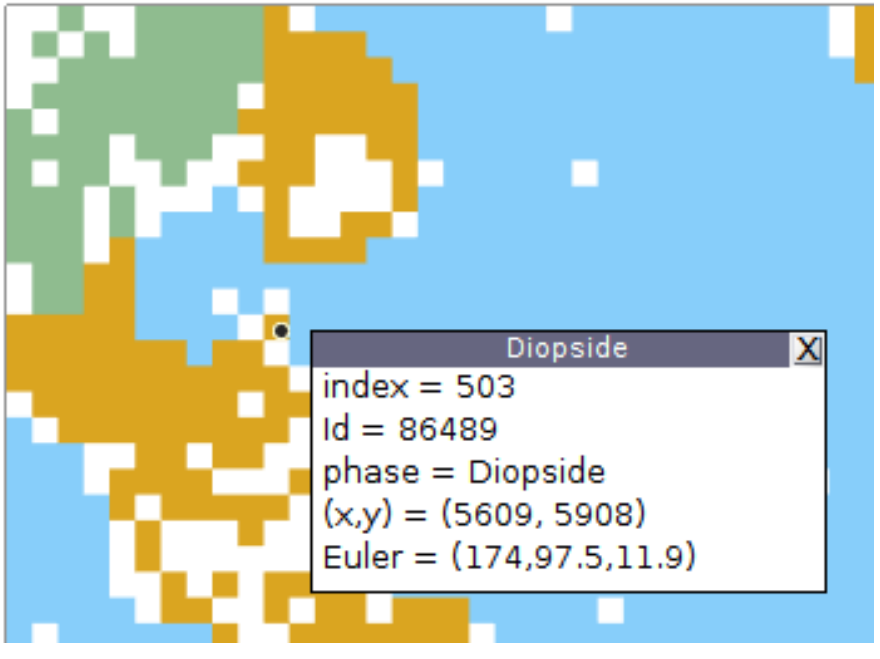
```
>>  ebsd{111}
>>  ebsd('id',111)

ans = EBSD (show methods, plot)

 Phase  Orientations     Mineral         Color  Symmetry  Crystal reference frame
     1      1 (100%)  Forsterite  LightSkyBlue       mmm

  Id   Phase   phi1   Phi   phi2   bands    bc    bs   error   mad      x   y
 111       1     58   162    336       7   129   202       0   0.2   5500   0
 Scan unit : um
```

1. EBSD – selecting EBSD: by index, id

- relation between index and id

```
>> ebsd_cropped(503)

ans = EBSD

 Phase  Orientations    Mineral       Color  Symmetry  Crystal reference frame
     3       1 (100%)  Diopside  Goldenrod     12/m1        X||a*, Y||b*, Z||c

    Id   Phase    phi1    Phi    phi2   bands    bc     bs    error    mad       x       y
 86489       3     174     98      12       7    95    186        0    0.6    5600    5900
 Scan unit : um
```

after cropping

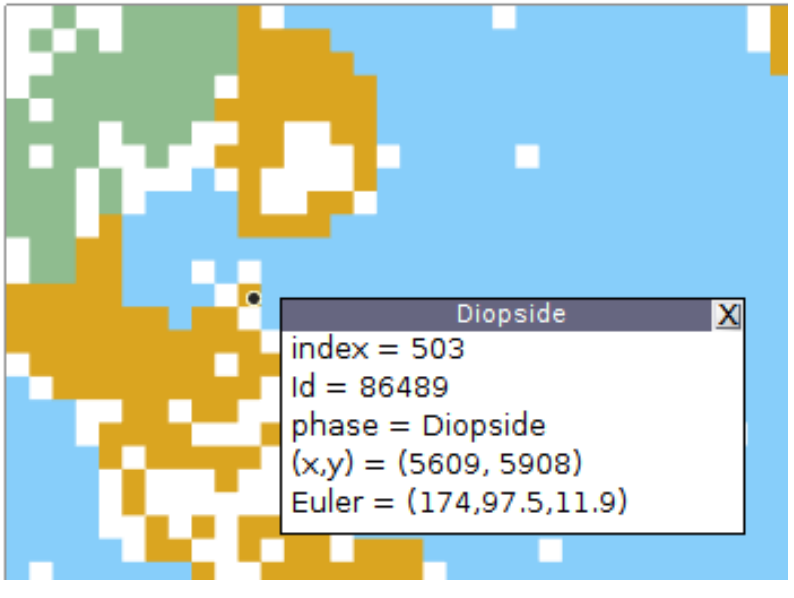


1. EBSD – selecting EBSD: by index, id

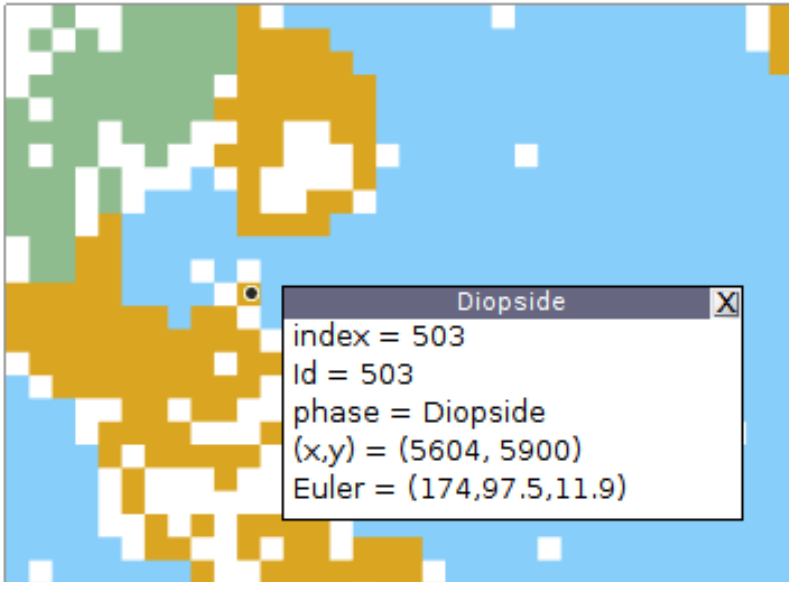
- id2ind: find an index such that ebsd.id(index) = id

```
>> ebsd_cropped.id= id2ind(ebsd_cropped,ebsd_cropped.id);

>> ebsd_cropped(503)

 ans = EBSD

 Phase  Orientations    Mineral       Color  Symmetry  Crystal reference frame
     3      1 (100%)  Diopside  Goldenrod     12/m1       X||a*, Y||b*, Z||c

   Id   Phase   phi1   Phi   phi2   bands   bc    bs   error   mad      x      y
  503       3    174    98     12       7   95   186       0   0.6   5600   5900
 Scan unit : um
```

after cropping



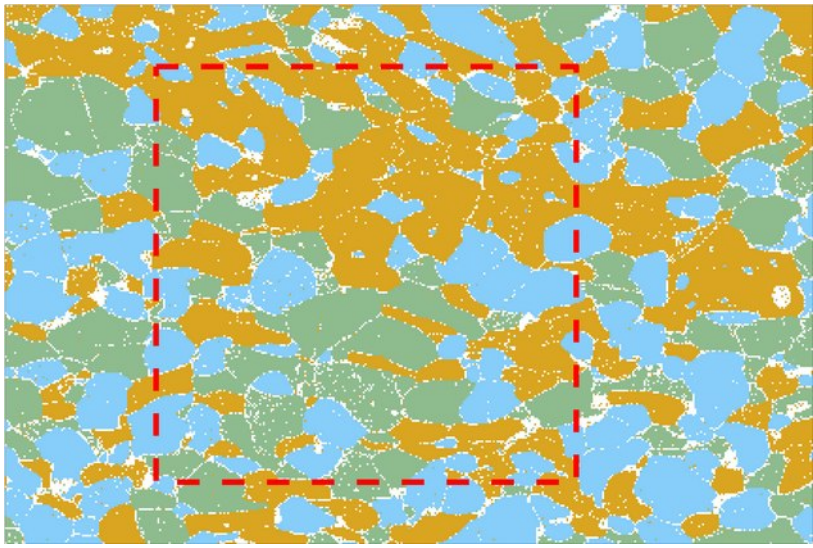
id2ind

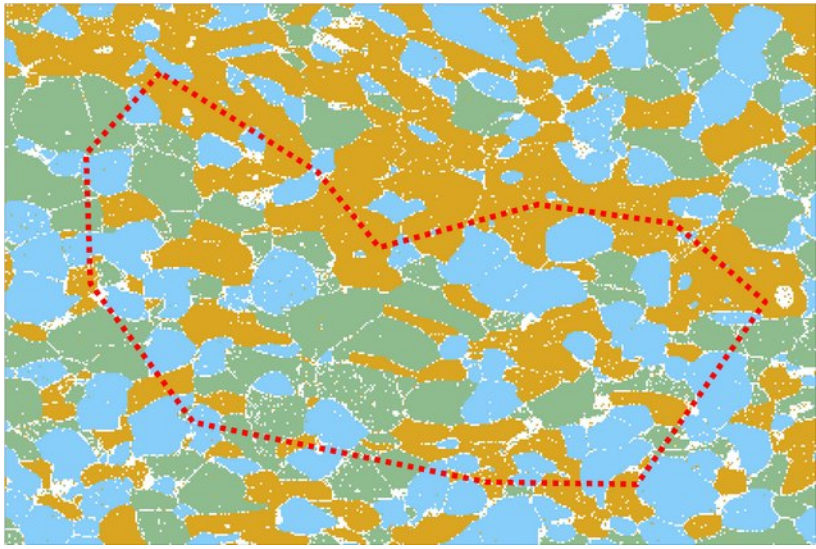


1. EBSD – selecting EBSD: by position

- various options

```
% interactively select a rectangle on the plot
[ebsd_sel,rectangle,id] = selectInteractive(ebsd)


% define a rectangle
inside = inpolygon(ebsd,[5100 5300 2000 2000]);
ebsd_sel = ebsd(inside)


% define an arbitrary polygon
plot(ebsd)
[x y] = ginput;

hold on
plot([x; x(1)],[y; y(1)],':','linewidth',5,'color','r')
hold off
```

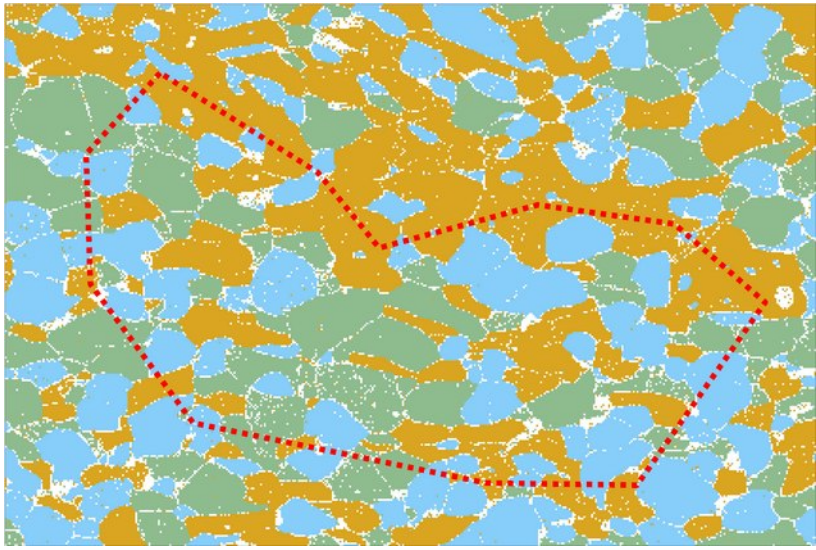


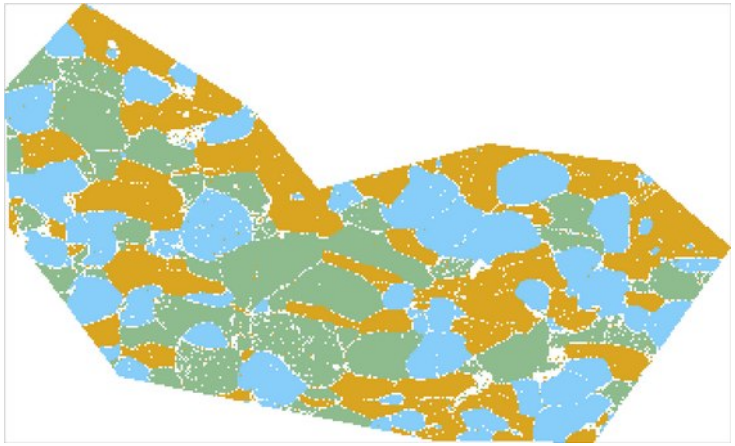


1. EBSD – selecting EBSD: by position

- various options

```
% define an arbitrary polygon
plot(ebsd)
[x y] = ginput;

hold on
plot([x; x(1)],[y; y(1)],':','linewidth',5,'color','r')
hold off

% select EBSD inside the polygon
ebsd_sel = ebsd(inpolygon(ebsd,[x,y]))
plot(ebsd_sel)
```

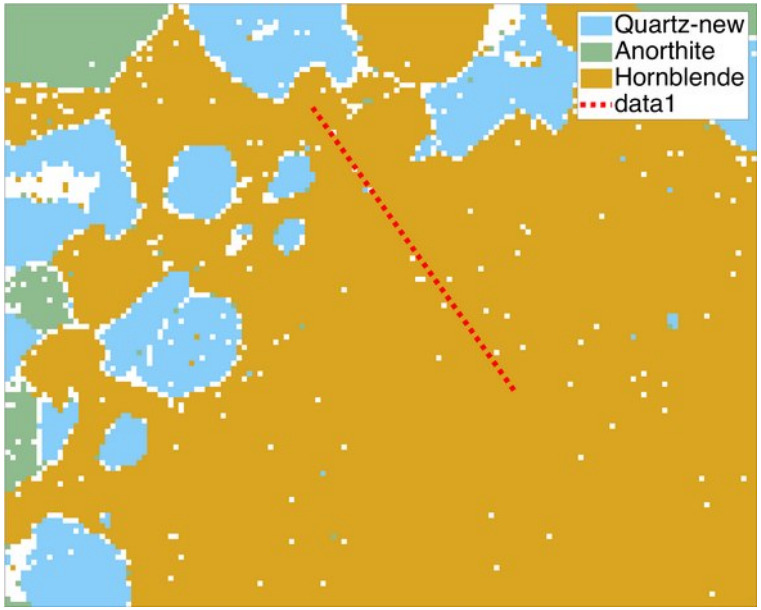


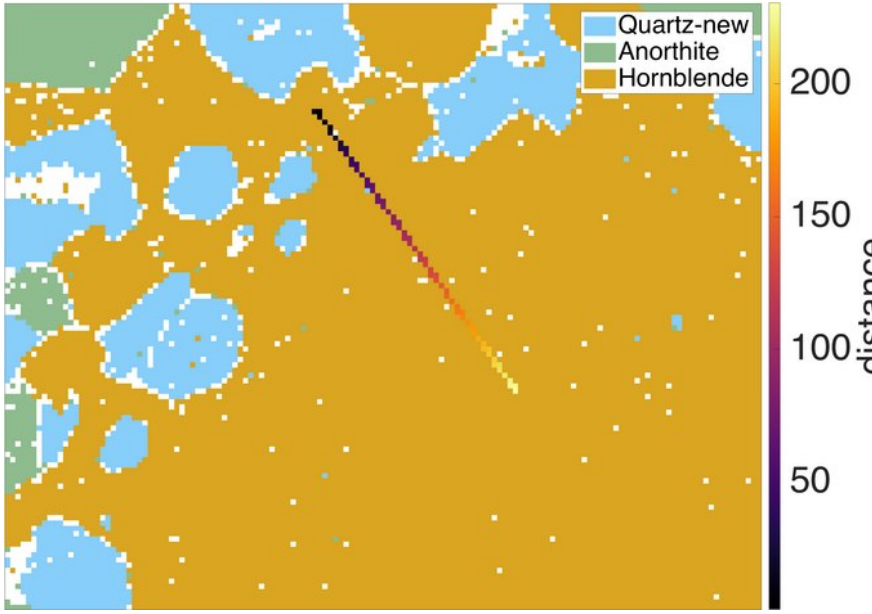


1. EBSD – selecting EBSD: by position

- line selection

```
% select EBSD along a line
plot(ebsd_sel)
[x y] = ginput(2); %click START and END points
hold on
plot(x,y,':','linewidth',5,'color','r')
hold off



[ebsdLine,dist] = spatialProfile(ebsd_sel,x,y);

plot(ebsd_sel)
hold on
plot(ebsdLine,dist)
hold off

>> ebsdLine

ebsdLine = EBSD

 Phase  Orientations     Mineral
    0       2 (3%)  notIndexed
    3     65 (97%)  Hornblende
```

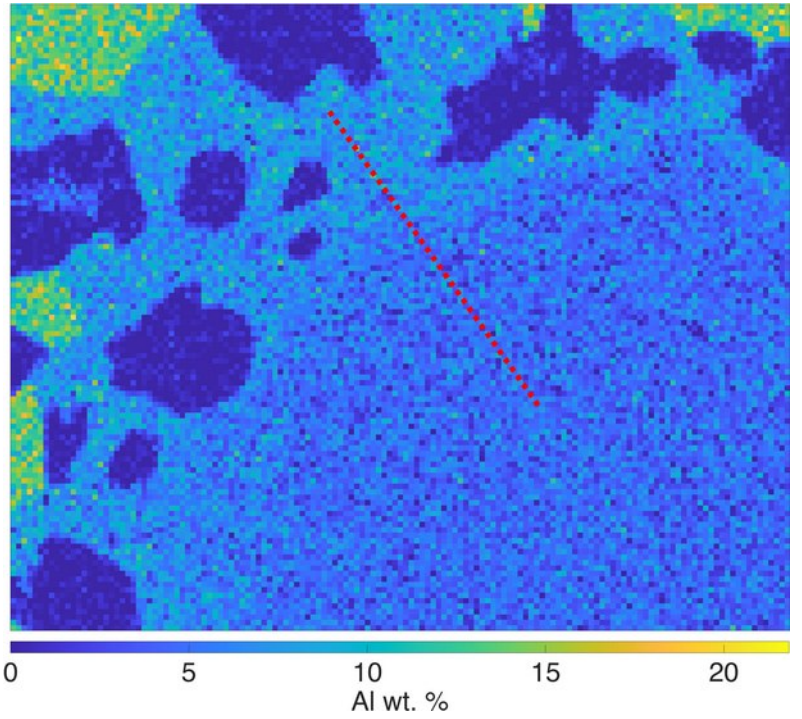


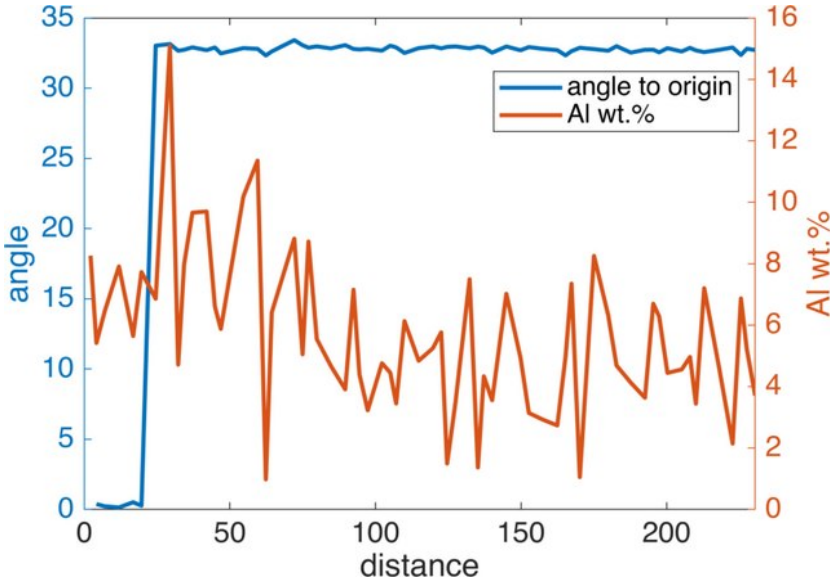


1. EBSD – selecting EBSD: by position

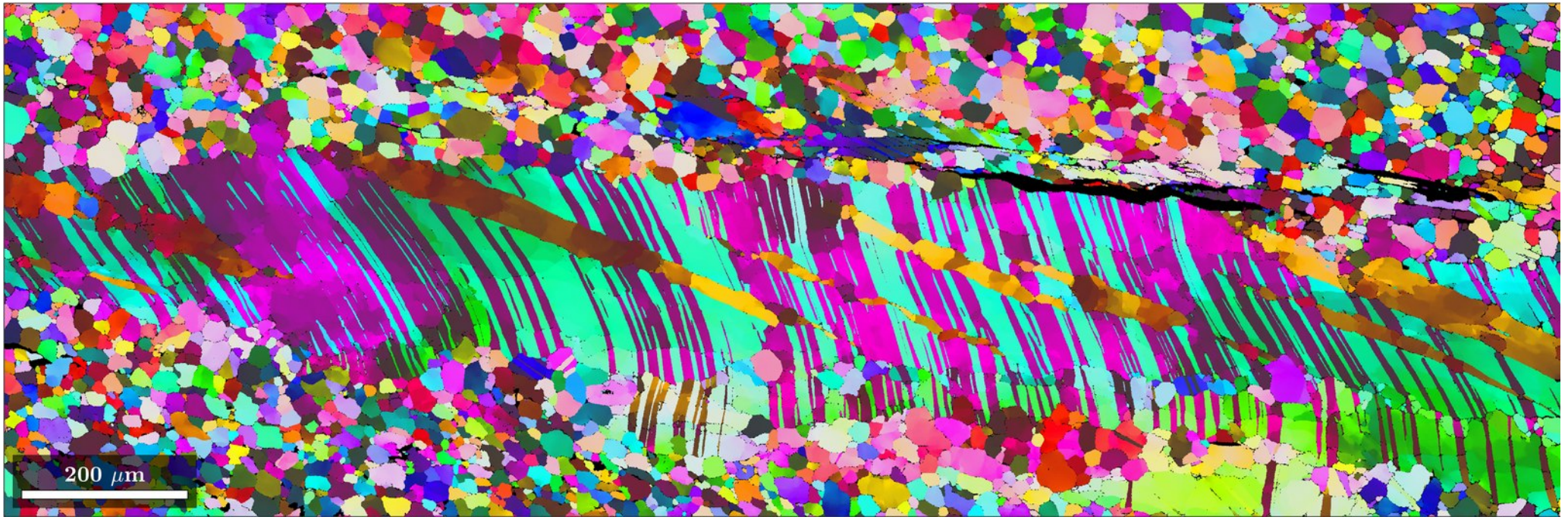
- various options

```
>> ebsdLine

ebsdLine = EBSD

 Phase  Orientations    Mineral
    0      2 (3%)  notIndexed
    3     65 (97%)  Hornblende

% use EBSD along the line to plot a profile
% extract orientations
o = ebsdLine('h').orientations;
% find the hornblende
posi = ebsdLine.phaseId == ebsd('h').indexedPhasesId;
d = dist(posi);
% compute the misorientation angle to the first
% orientation
a2ori = angle(o(1),o(2:end))/degree;
% plot the angle as a function of distance
plot(d(2:end),a2ori)
hold on
% plot any data of ebsdLine
plot(dist,ebsdLine.prop.Al_Wt_)
hold off
```





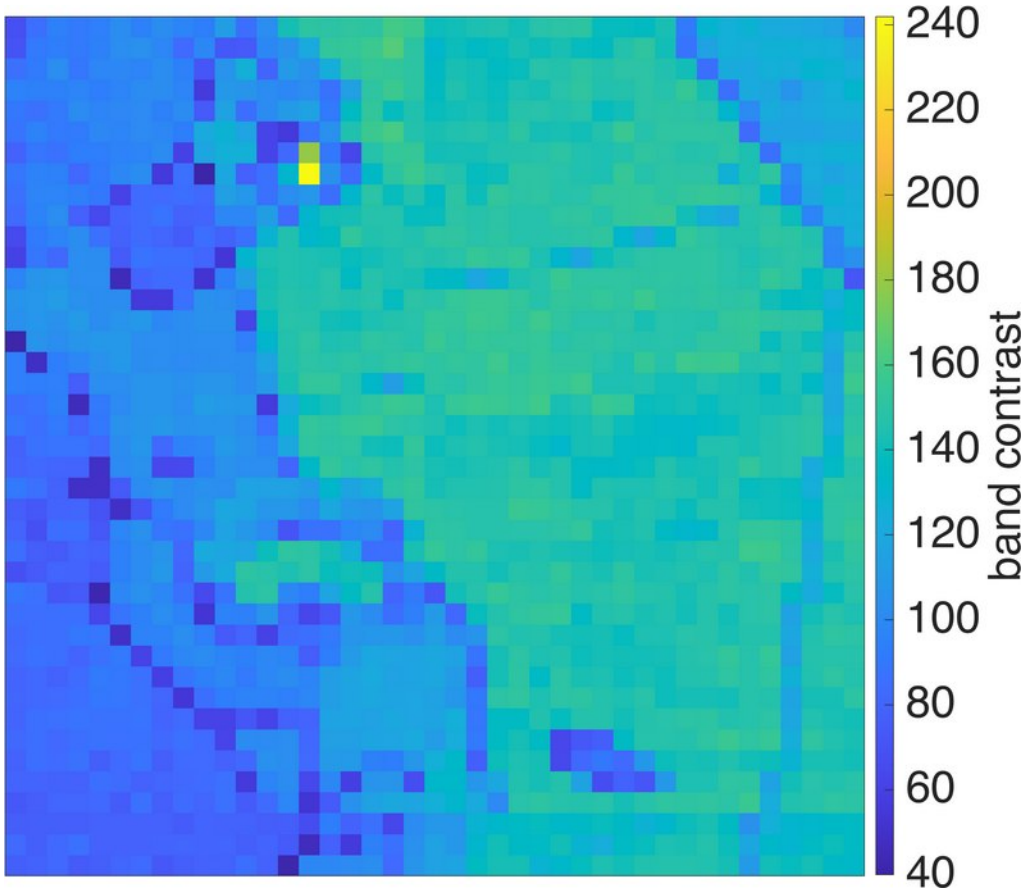
1. EBSD – plotting data

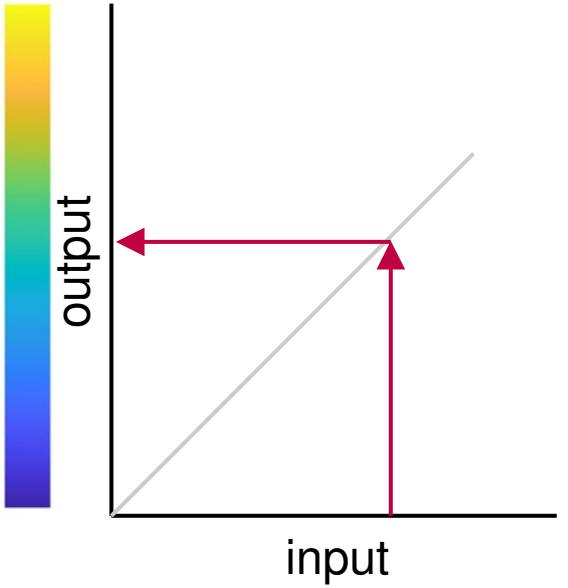


1. EBSD – plotting data

- plotting EBSD maps should behave as every other plotting command in mtex: *plottingcommand(where, what)*
- for scalar values, a 1-D look-up table is used

```
% example for a scalar value
plot(ebsd,ebsd.bc)
mtexColorbar('title','band contrast')
```

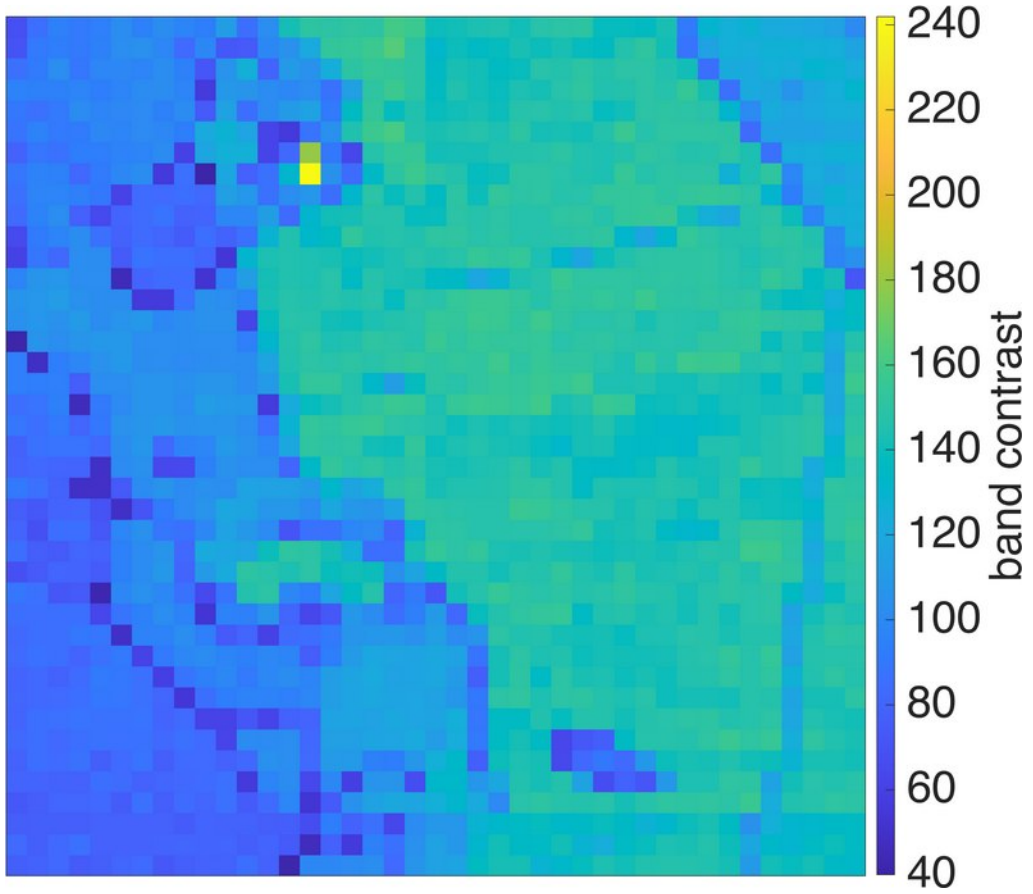




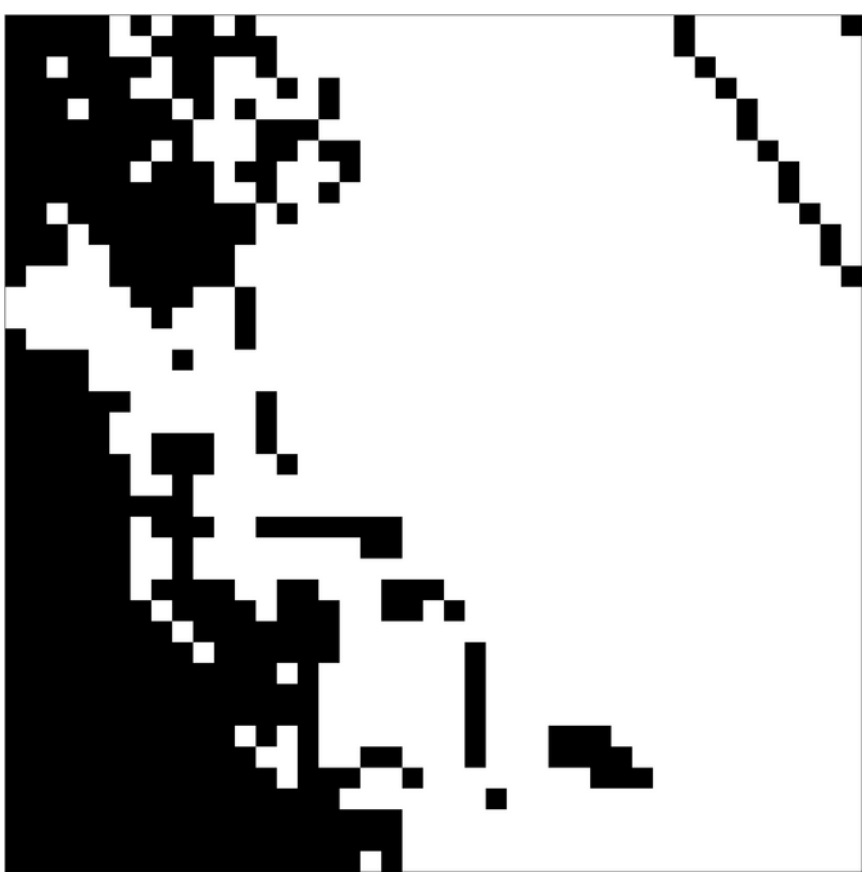
1. EBSD – plotting data

- plotting EBSD maps should behave as every other plotting command in mtex: *plottingcommand(where, what)*
- for scalar values, a 1-D look-up table is used

```
% example for a scalar value
plot(ebsd,ebsd.bc)
mtexColorbar('title','band contrast')
```

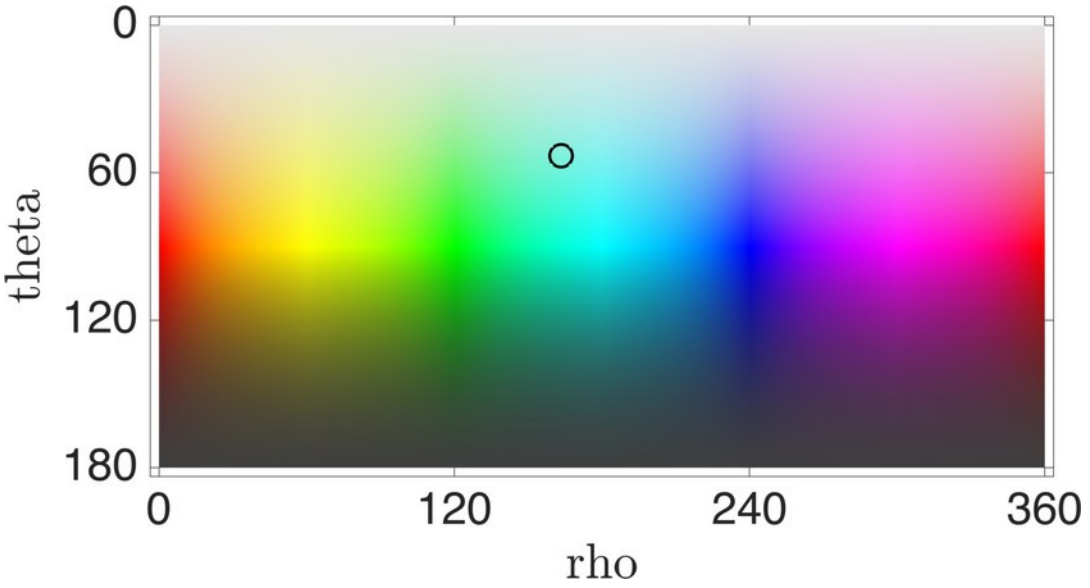


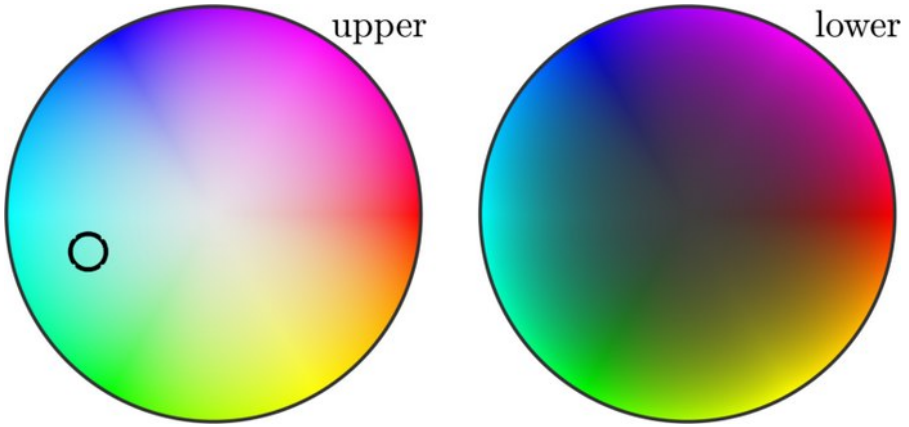

```
% logical
plot(ebsd,ebsd.bc>100)
```



1. EBSD – plotting data

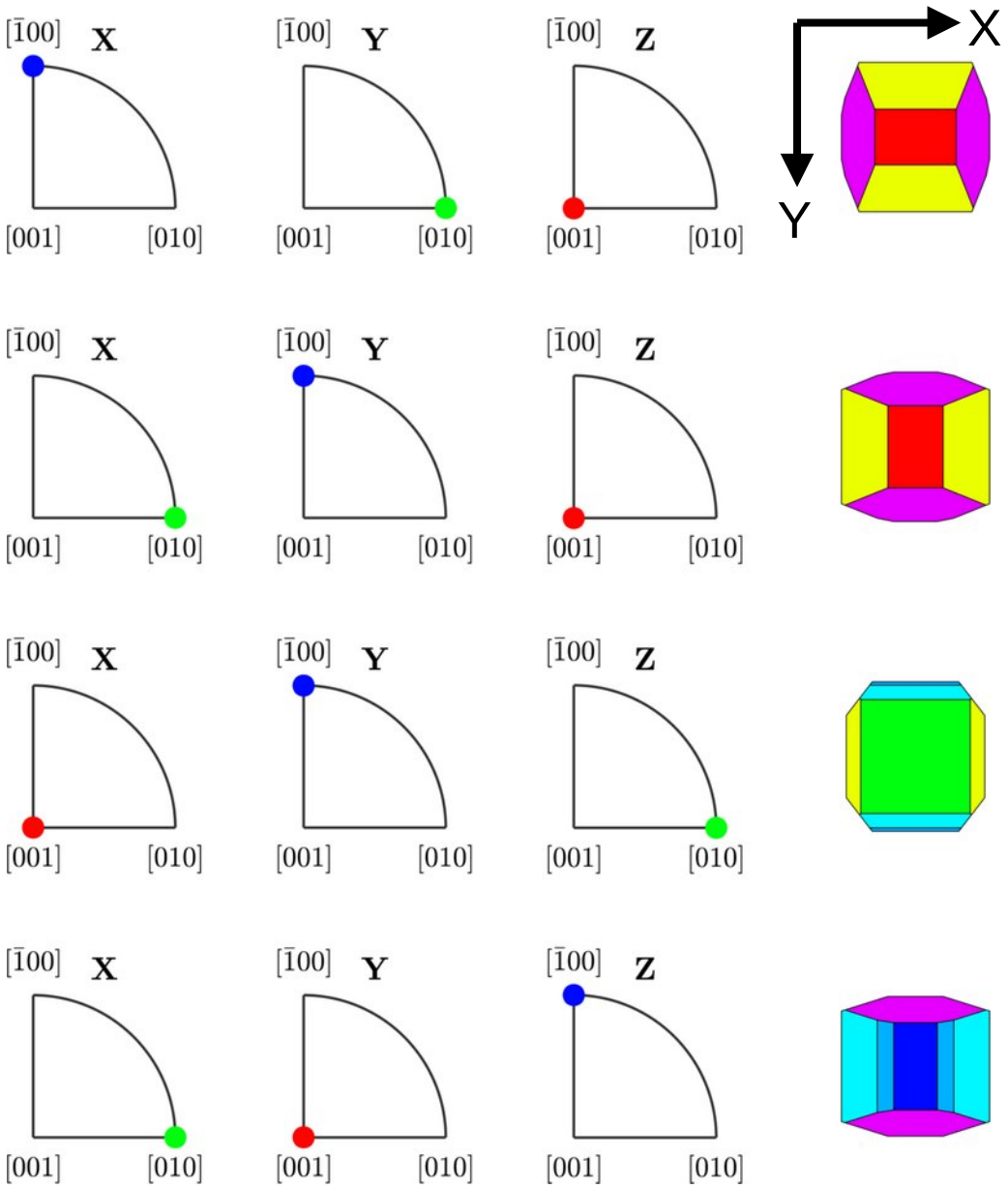
- plotting EBSD maps should behave as every other plotting command in mtex: *plottingcommand(where, what)*
- for non-scalar values (e.g. directions or orientations), Mtex provides different color mappings (color keys) which compute a color [RGB triplet] for each input

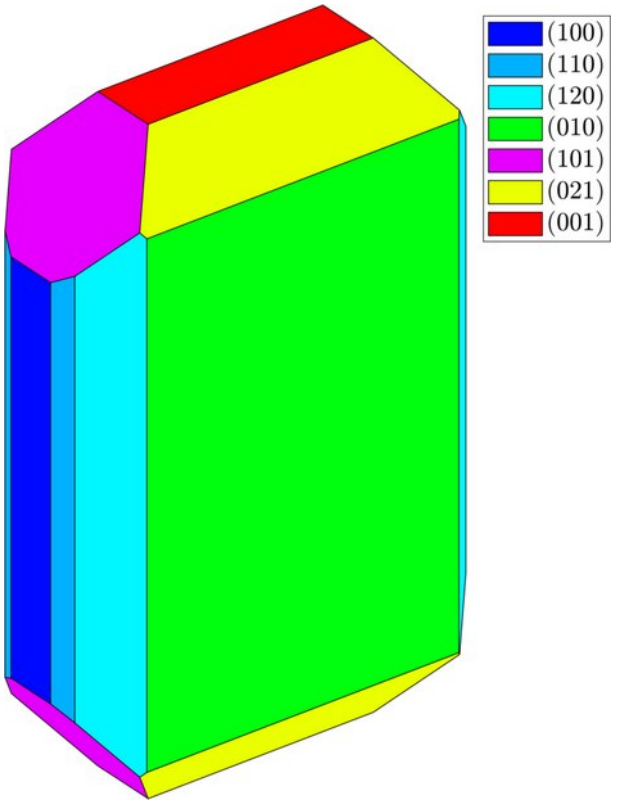




1. EBSD – plotting data:

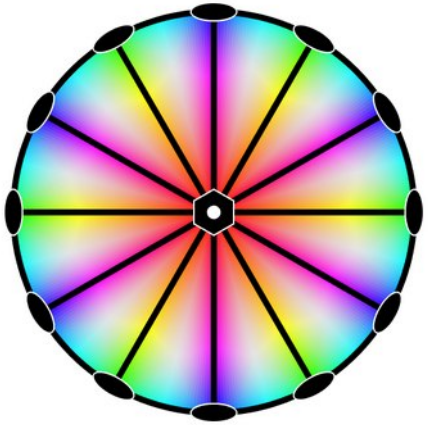
- inverse pole figure / ipf color coding: color-coding the crystal direction parallel to a specimen reference direction

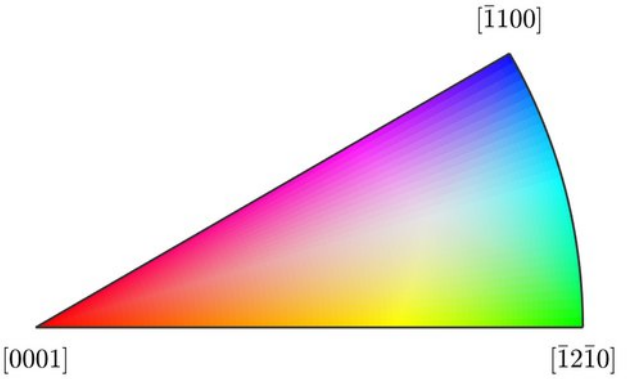


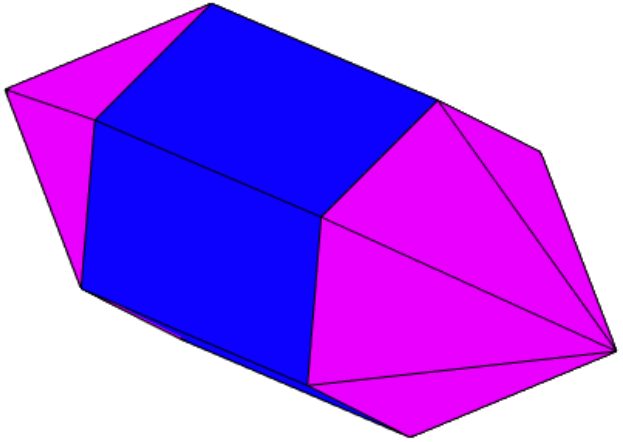


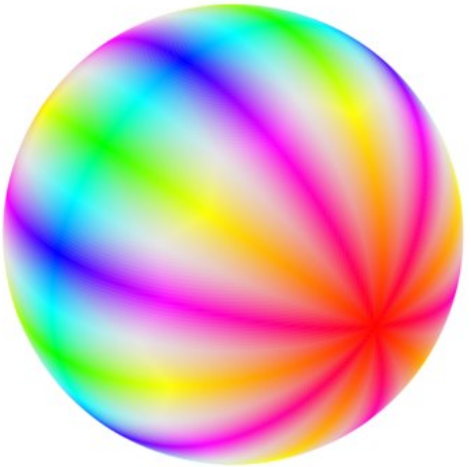
1. EBSD – plotting data:

- inverse pole figure color coding



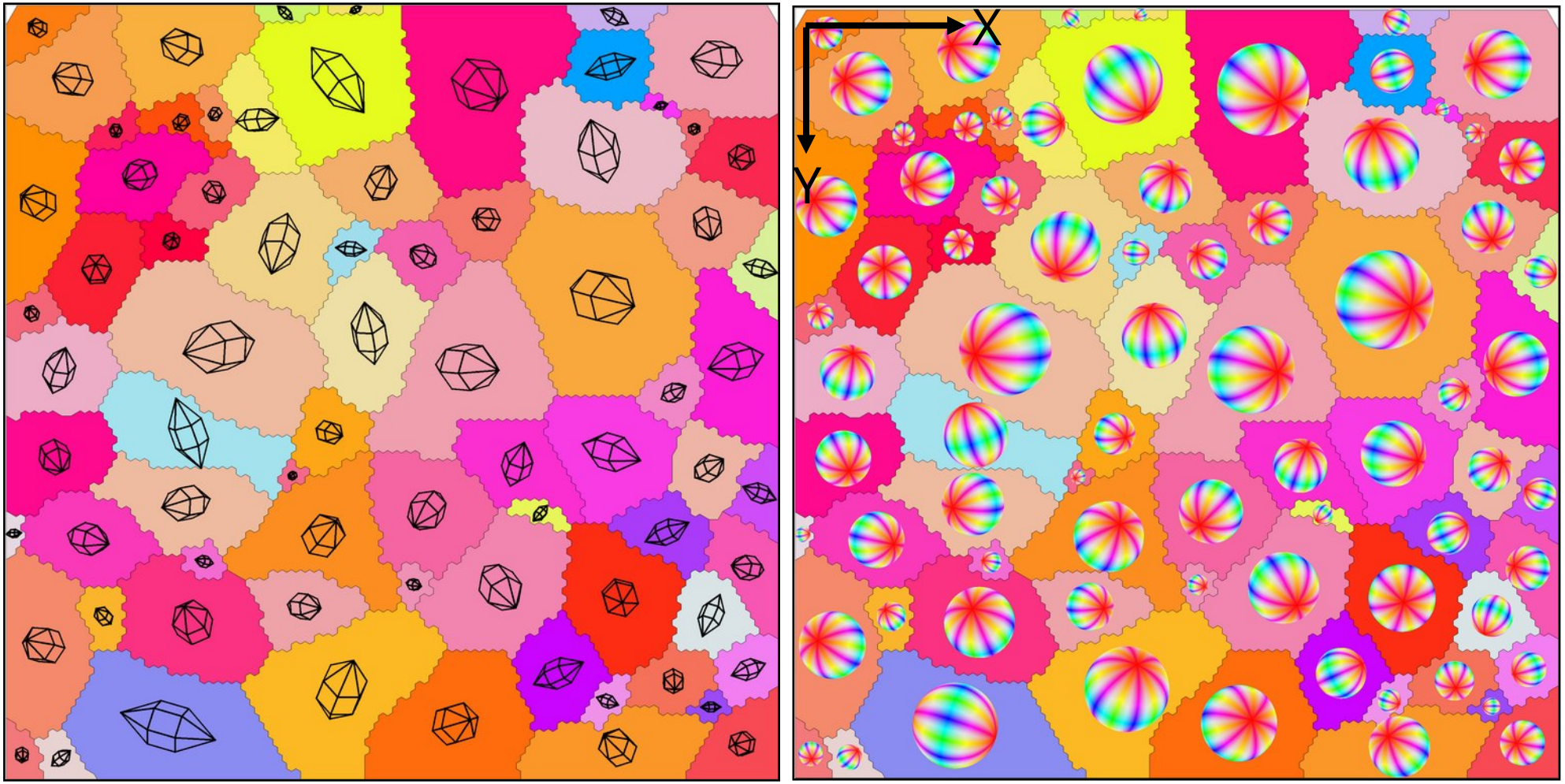






1. EBSD – plotting data:

- producing ipf maps: direction mapping in crystal coordinates
- one ipf map cannot display the entire orientation

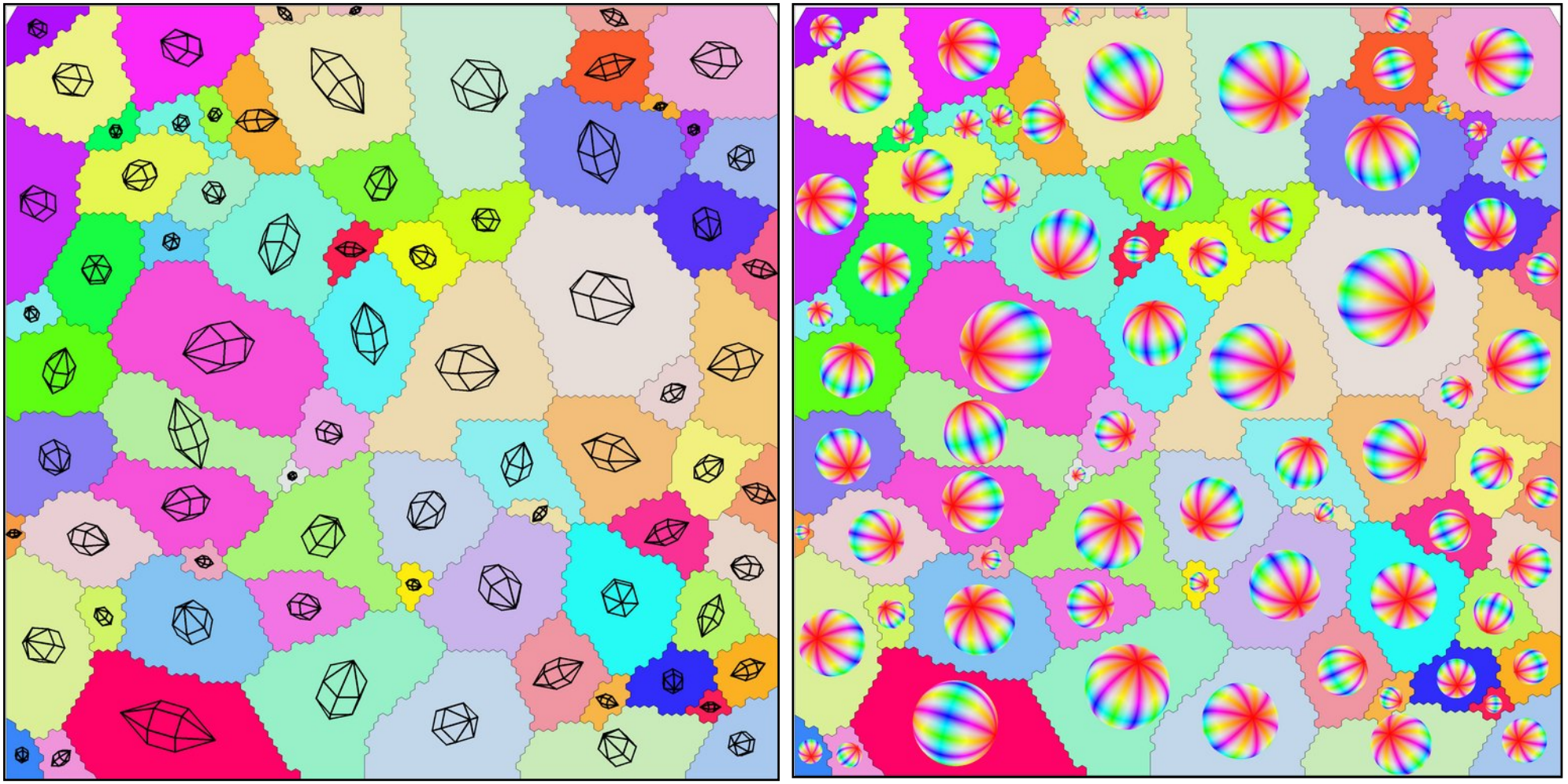


ipf with reference direction (0,0,1)

one ipf map is not enough to read the crystal orientation from the map!

1. EBSD – plotting data:

- producing ipf maps: direction mapping in crystal coordinates
- one ipf map cannot display the entire orientation

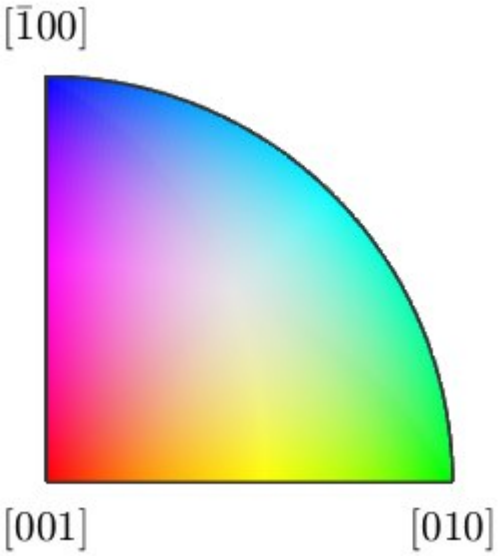


ipf with reference direction (1,0,0)

1. EBSD – plotting data

- directionColorKey
- input Miller for crystalSymmetry / input vector3d for specimenSymmetry

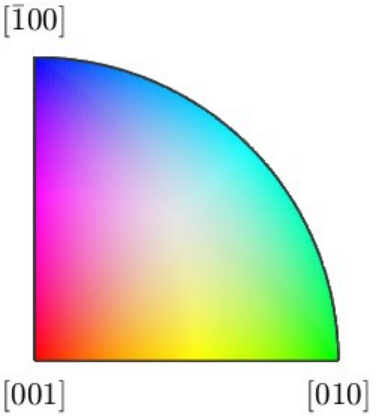
```
% example for a direction color key
cK = HSVDirectionKey(ebsd('f').CS)

cK =

  HSVDirectionKey with properties:

      colorStretching: 1
          whiteCenter: [1×1 vector3d]
            grayValue: [0.2000 0.5000]
         grayGradient: 0.5000
             maxAngle: Inf
...

plot(cK)
```

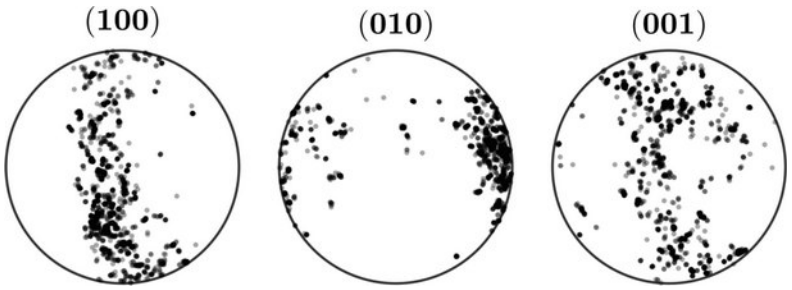


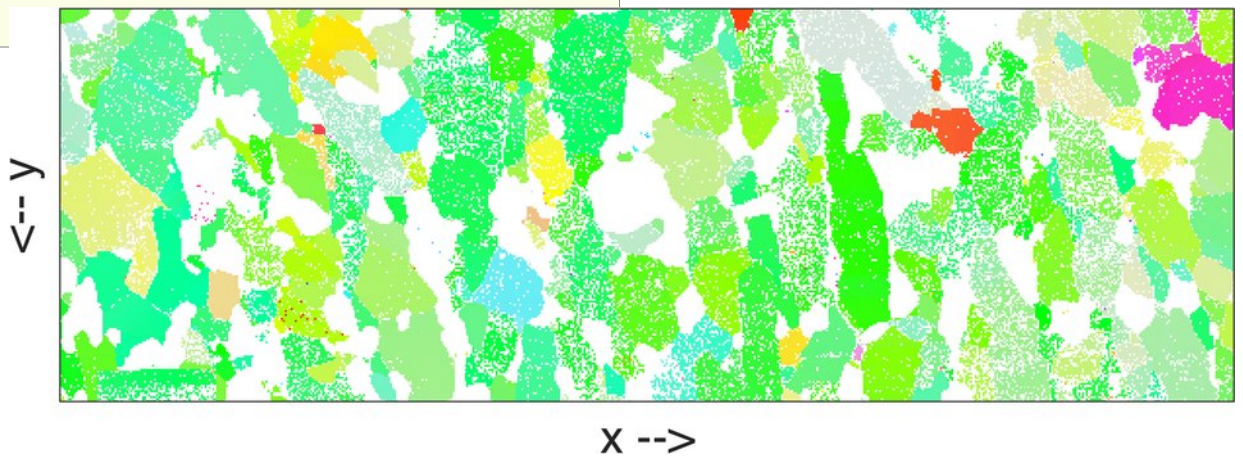
1. EBSD – plotting data

- directionColorKey
- producing [RGB] triplets from directions

```
% example for a direction color key
cK = HSVDirectionKey(ebsd('f').CS);

color = cK.direction2color(inv(ebsd('f').orientations) .* xvector)

    0.9264    0.7016    0.9165
    0.9263    0.7024    0.9165
    0.9264    0.7016    0.9156
    0.9262    0.7027    0.9177
    0.9263    0.7019    0.9171
....

plot(ebsd('f'), color)
```



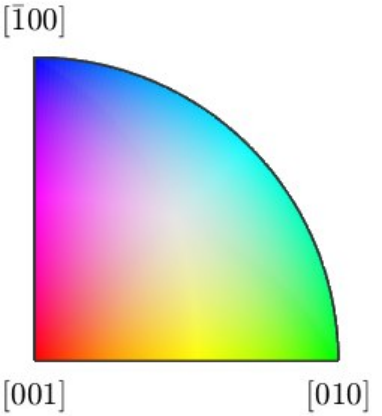


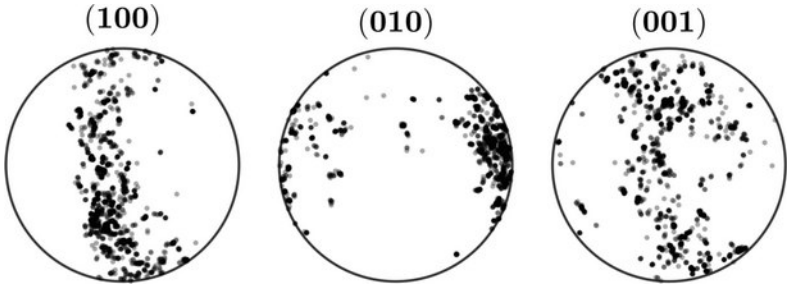


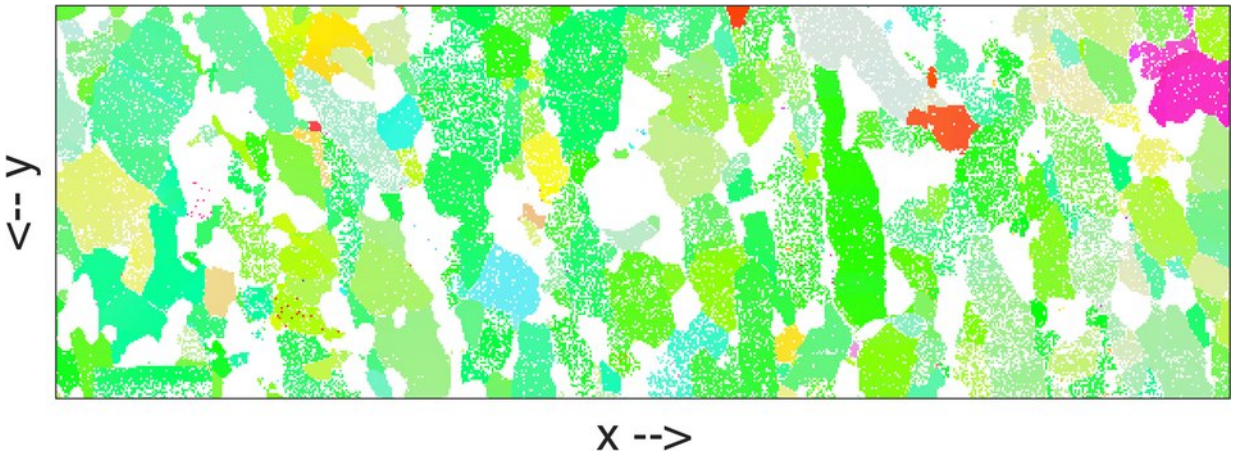
1. EBSD – plotting data

- ipfKey for inverse polefigure color mappings of orientations
- takes orientations as input, requires reference direction

```
% instead of
ck= ipfHSVKey(cs);

ck.inversePoleFigureDirection=vector3d(1,0,0);

color = cK.orientation2color(ebsd('f').orientations);

plot(ebsd('f'), color)
```



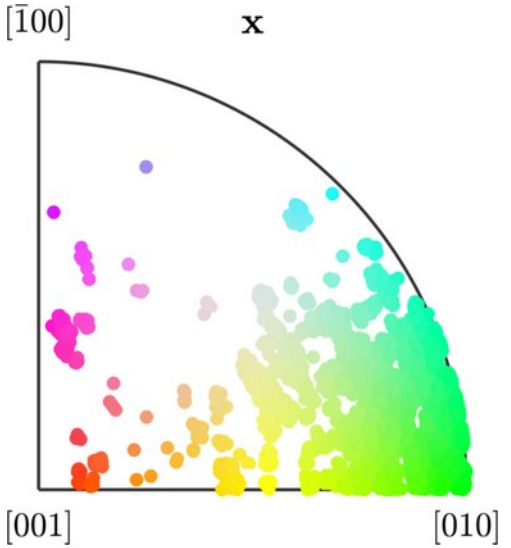


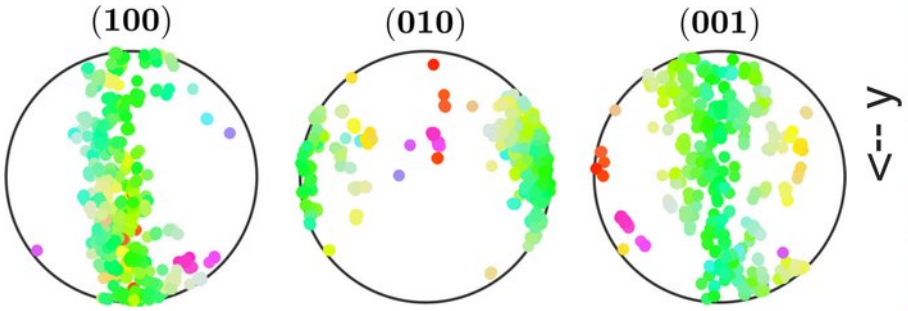


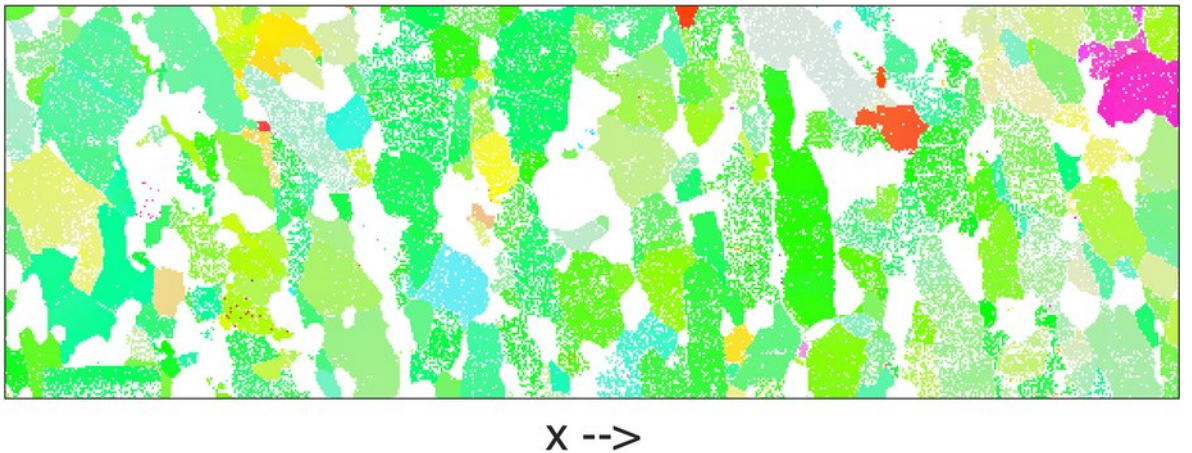
1. EBSD – plotting data

- ipfKey for inverse polefigure color mappings of orientations
- takes orientations as input, requires reference direction

```
% directly specify IPF color key
ck= ipfHSVKey(cs);

ck.inversePoleFigureDirection=vector3d(1,0,0);

color = cK.orientation2color(ebsd('f').orientations);

% plot a map
plot(ebsd('f'), color)
% plot an ipf point plot
plotIPDF(o,color,xvector)
% plot a pf point plot
plotPDF(o,color,h)
```

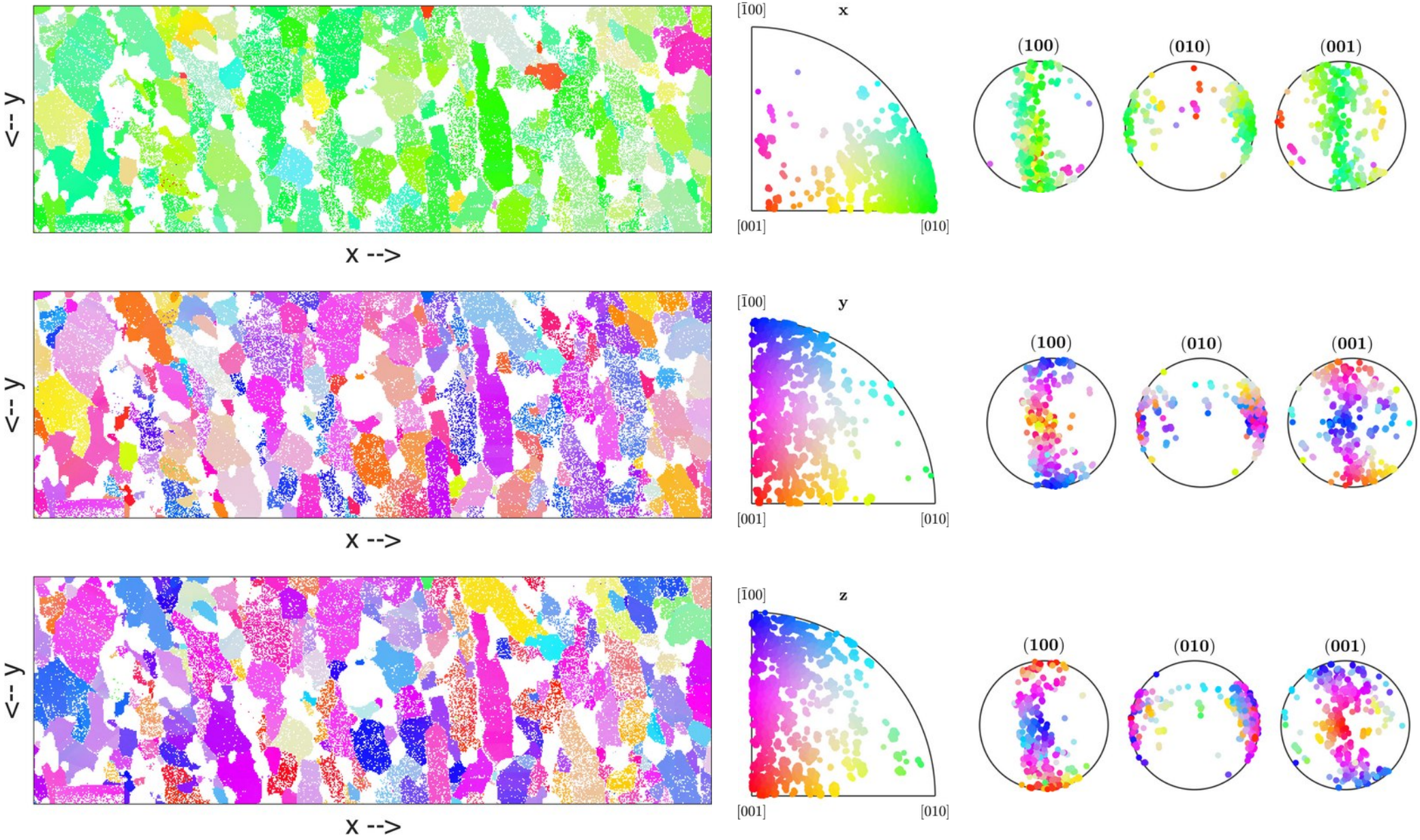






1. EBSD – plotting data

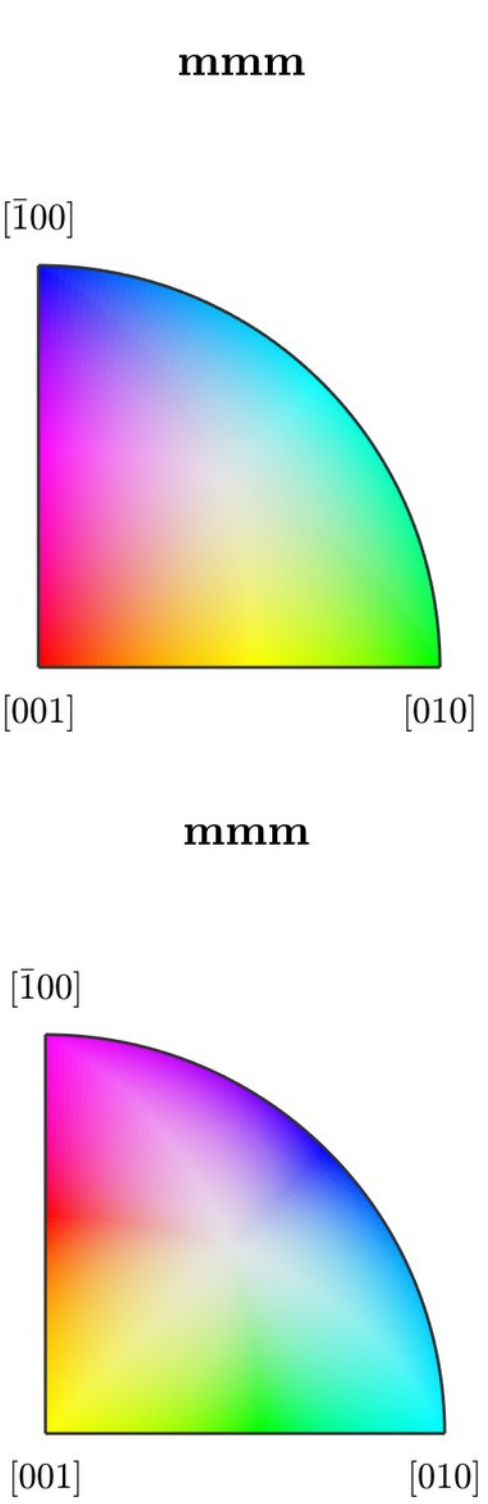
- ipfKey for inverse polefigure color mappings of orientations



1. EBSD – plotting data

- inverse pole figures – ipdfHSV
- various methods to change color key

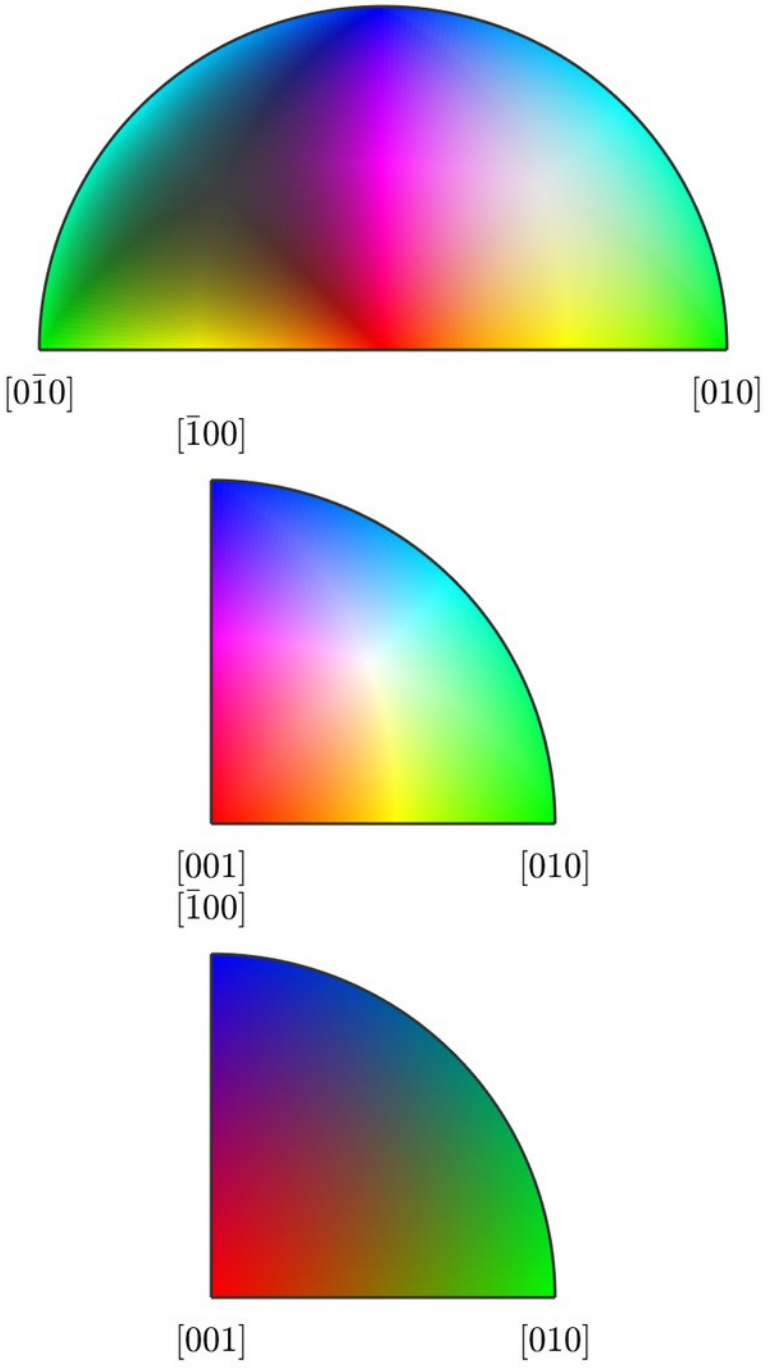
```
% example for a direction color key
cK = ipfHSVKey(ebsd('f').CS)

ans =

  ipfHSVKey with properties:

             colorPostRotation: [1×1 rotation]
               colorStretching: 1
                   whiteCenter: [1×1 vector3d]
                     grayValue: [0.2000 0.5000]
                  grayGradient: 0.5000
                      maxAngle: Inf
    inversePoleFigureDirection: [1×1 vector3d]
                        dirMap: [1×1 HSVDirectionKey]
                           CS1: [1×1 crystalSymmetry]
                           CS2: [1×1 specimenSymmetry]
                     antipodal: 0

rot = rotation.byAxisAngle(zvector,60*degree)
ck.colorPostRotation = rot
plot(ck)
```



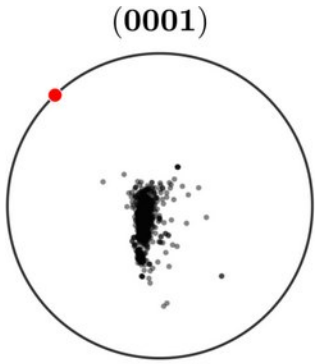
1. EBSD – plotting data

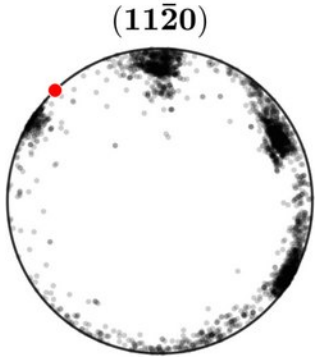
- inverse pole figures – ipdfHSV
- other mappings

```
cs = crystalSymmetry('222')

% mtex default key
ck = HSVDirectionKey(cs);
plot(ck)

% former OIM key, EDAX/OIM recently switched to the
% mtex color keys
ck = TSLDirectionKey(cs);
plot(ck)

% very ancient HKL key
ck = HKLDirectionKey(cs);
plot(ck)
```



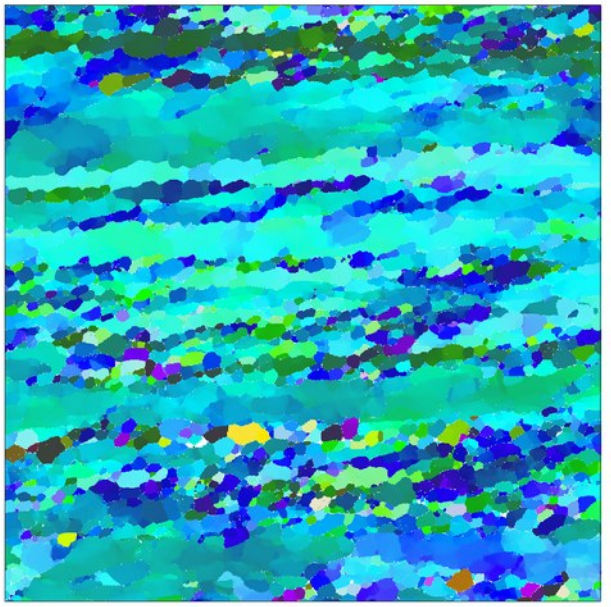
1. EBSD – plotting data



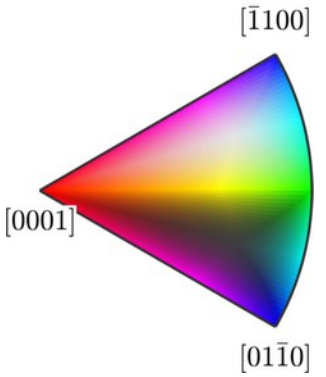


ref. : zvector

ref. : $(0001)_{mean}$ × zvector

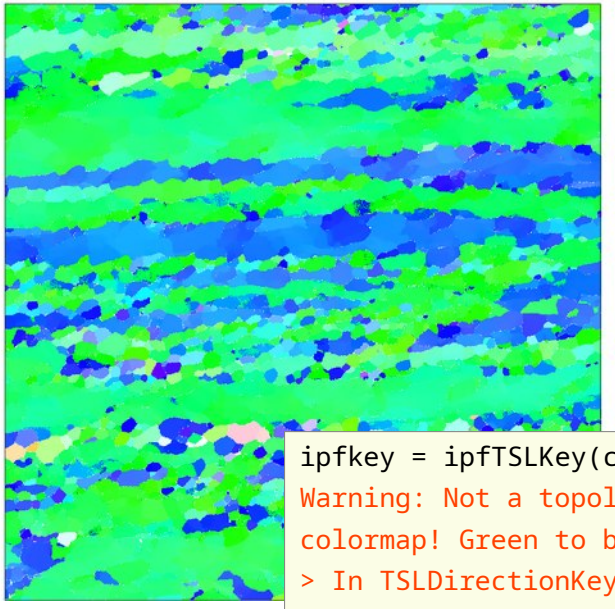


-3m1

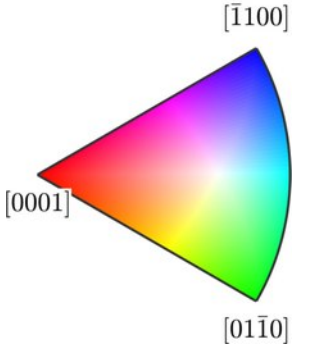


ref. : zvectorTSLkey

ref. : $(0001)_{mean}$ × zvector, TSLkey

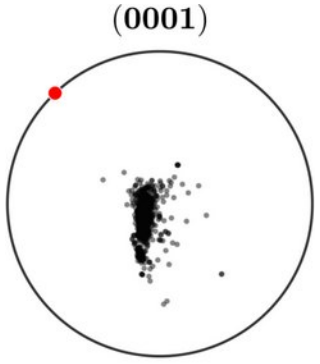


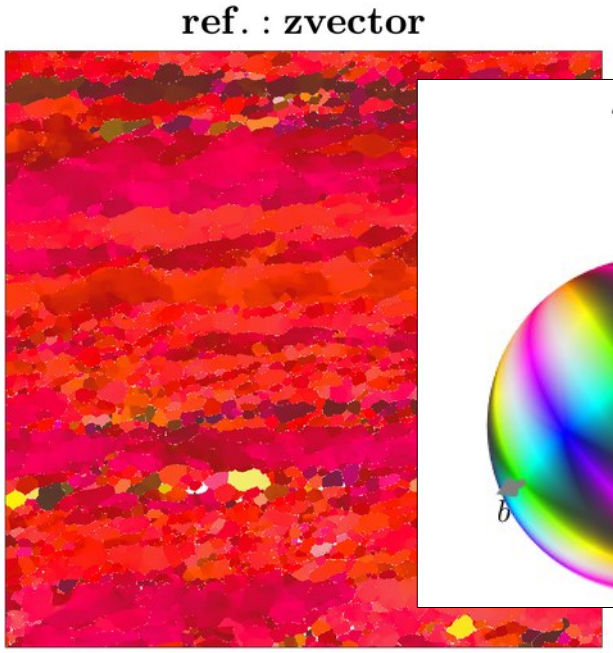
-3m1

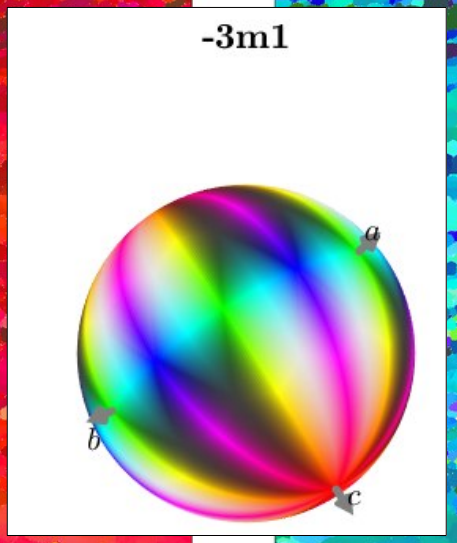


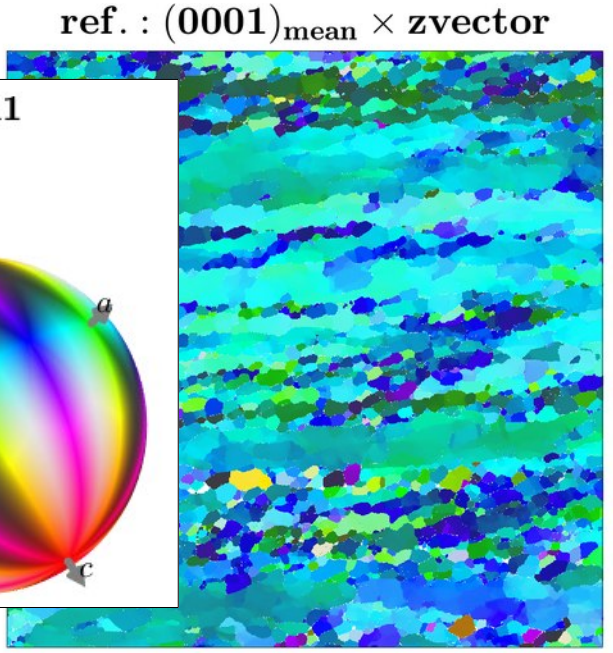

```
ipfkey = ipfTSLKey(cs);
Warning: Not a topological correct
colormap! Green to blue colorjumps possible
> In TSLDirectionKey (line 12)
In ipfTSLKey (line 9)
```

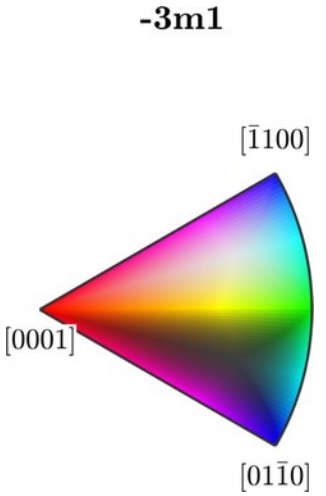
1. EBSD – plotting data

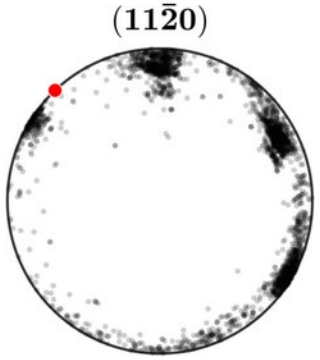


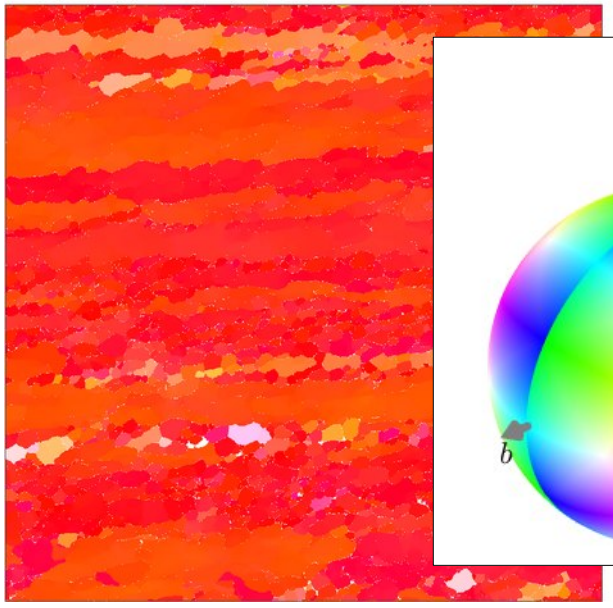


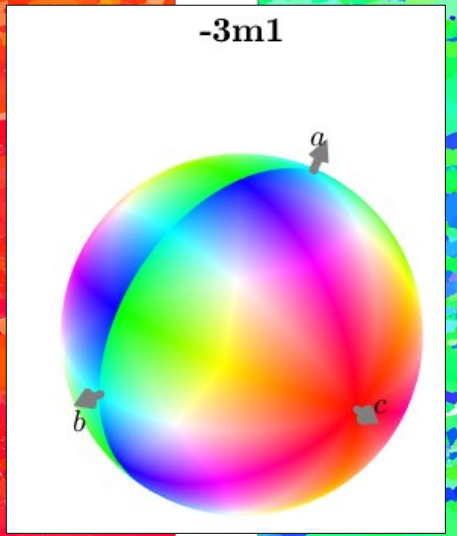


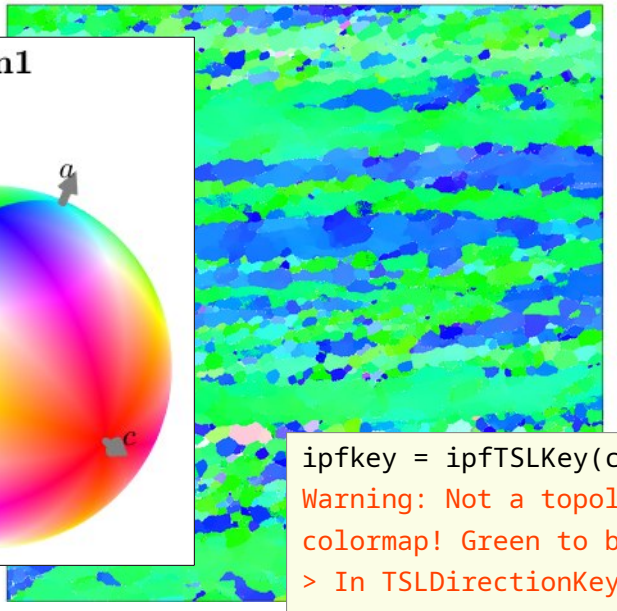


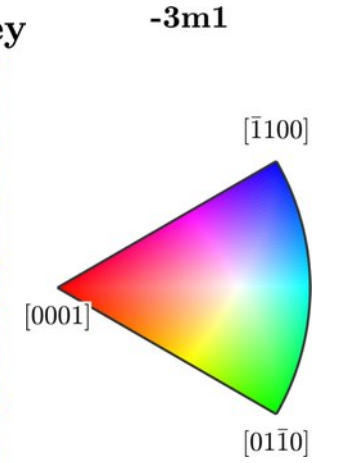








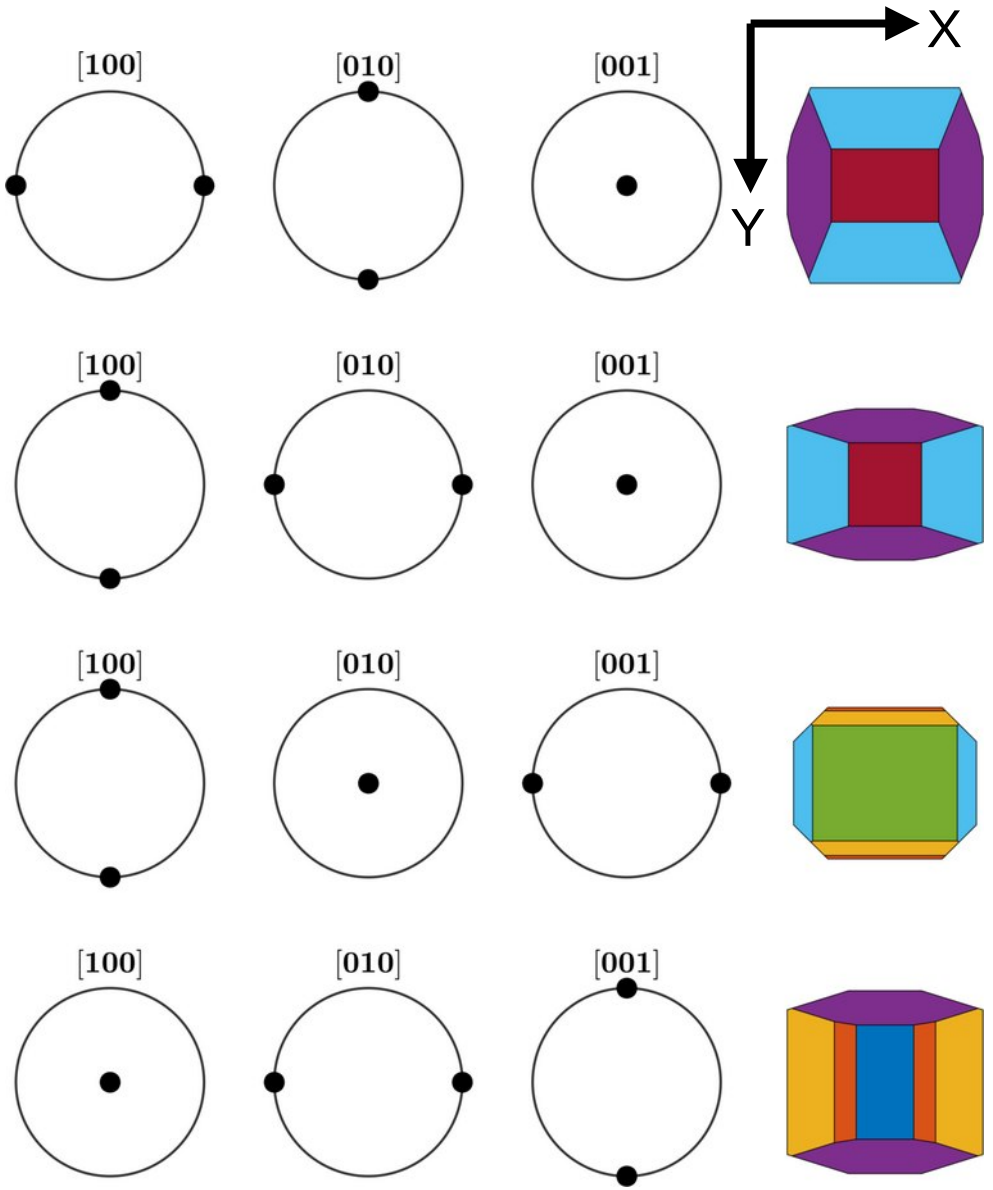


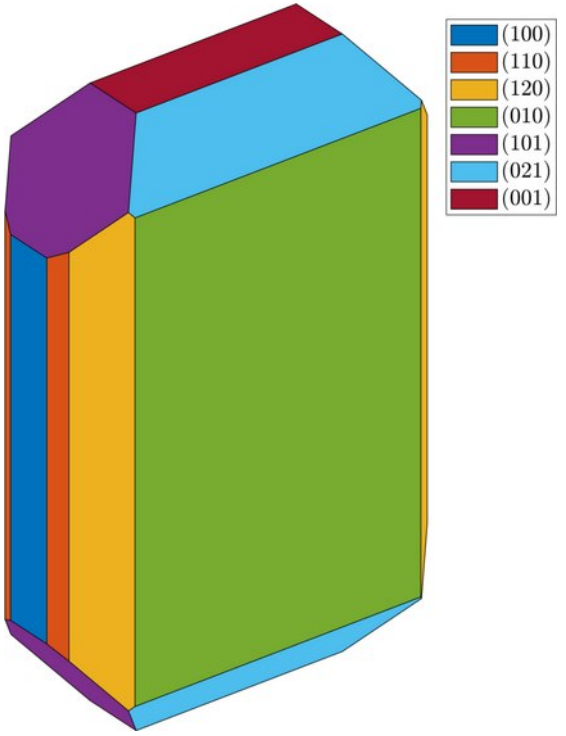



```
ipfkey = ipfTSLKey(cs);
Warning: Not a topological correct
colormap! Green to blue colorjumps possible
> In TSLDirectionKey (line 12)
In ipfTSLKey (line 9)
```

1. EBSD – plotting data

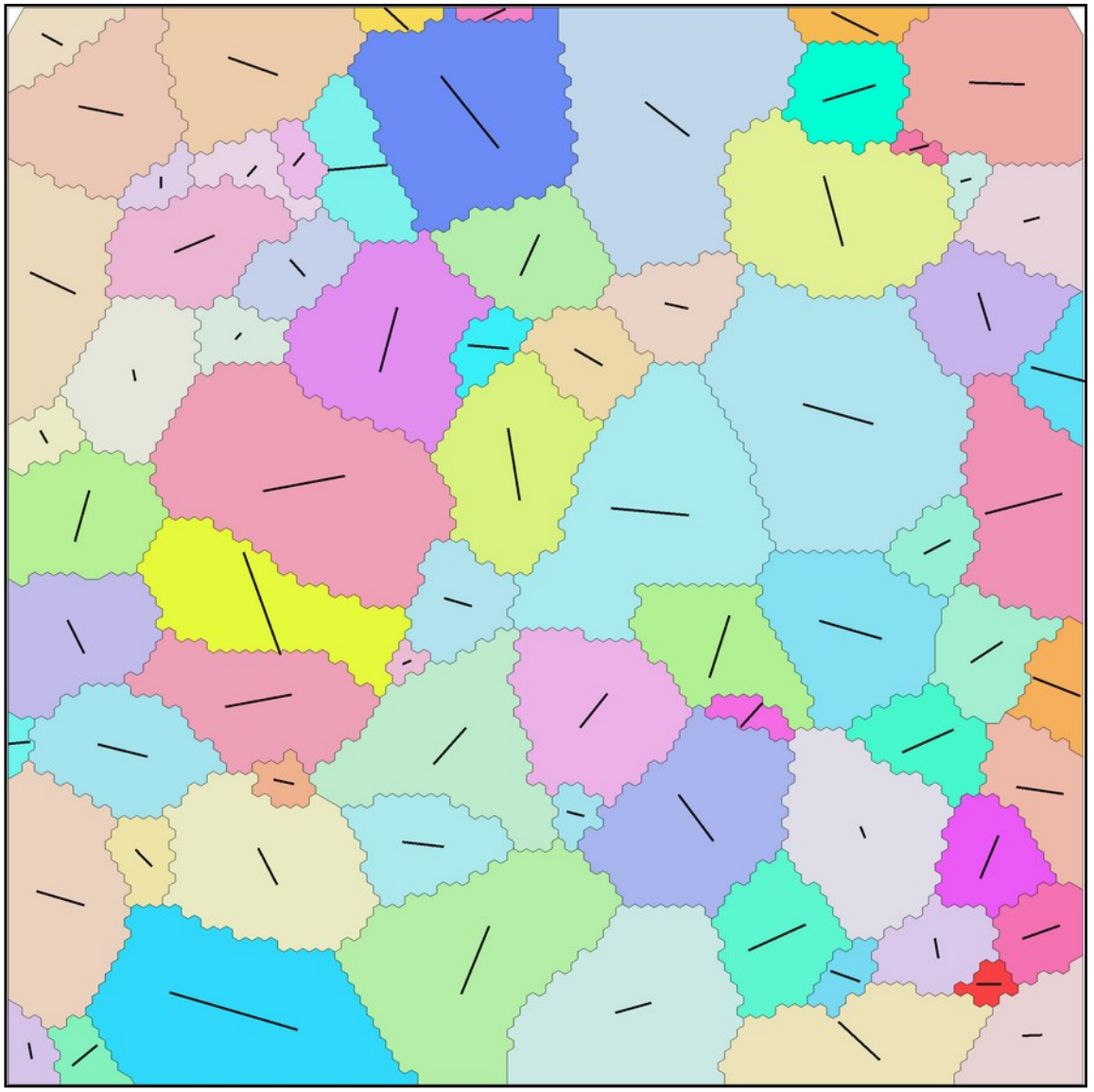
- representation of textures: pf color coding / coloring specimen directions
- only for unique directions

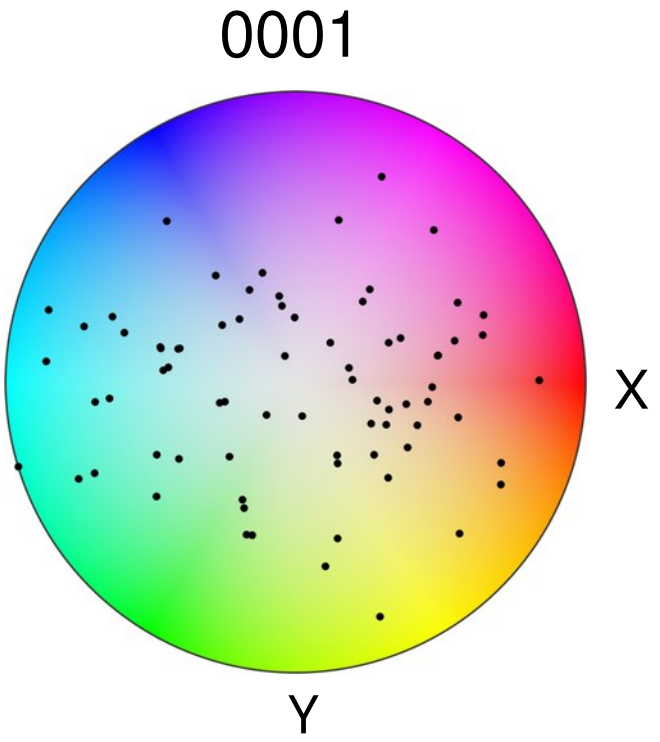




1. EBSD – plotting data

- representation of textures: pf color coding / coloring specimen directions
- only for unique directions

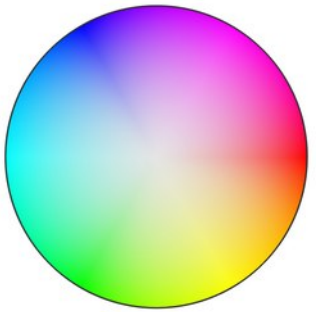


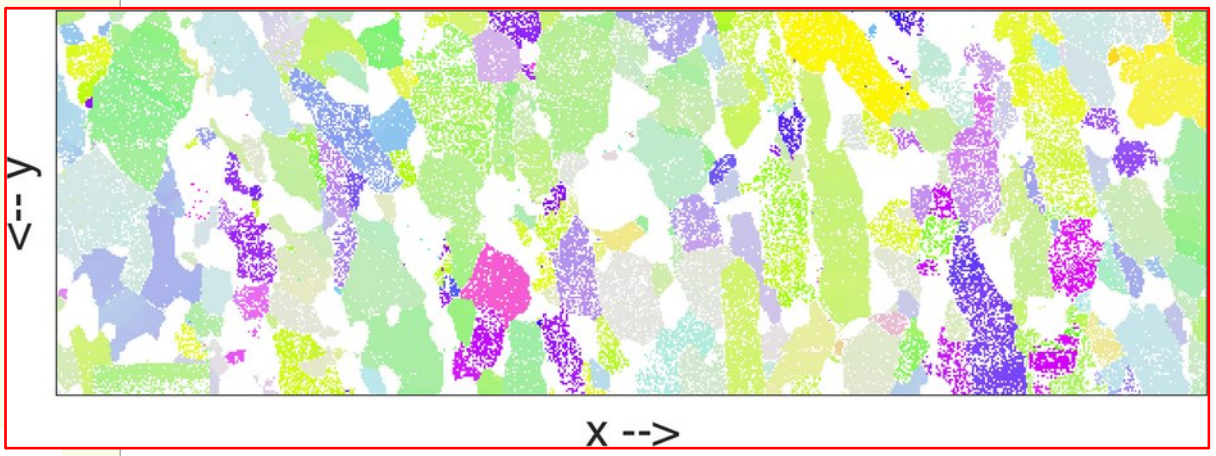


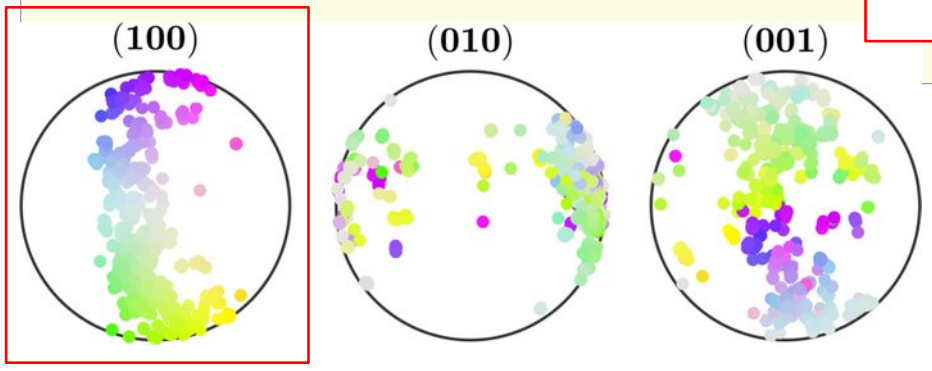
1. EBSD – plotting data

- direction key for color mappings of directions

```
% example for a direction color key
cK = HSVDirectionKey(specimenSymmetry('-1'))

Warning: Not a topological correct colormap!
Please use the point group "1"
> In HSVDirectionKey (line 35)

cK =  HSVDirectionKey with properties:

      colorStretching: 1
          whiteCenter: [1×1 vector3d]
            grayValue: [0.2000 0.5000]
...

% <a> mapping in specimen coordinates
h = Miller(1,0,0,ebsd('f').CS)
```

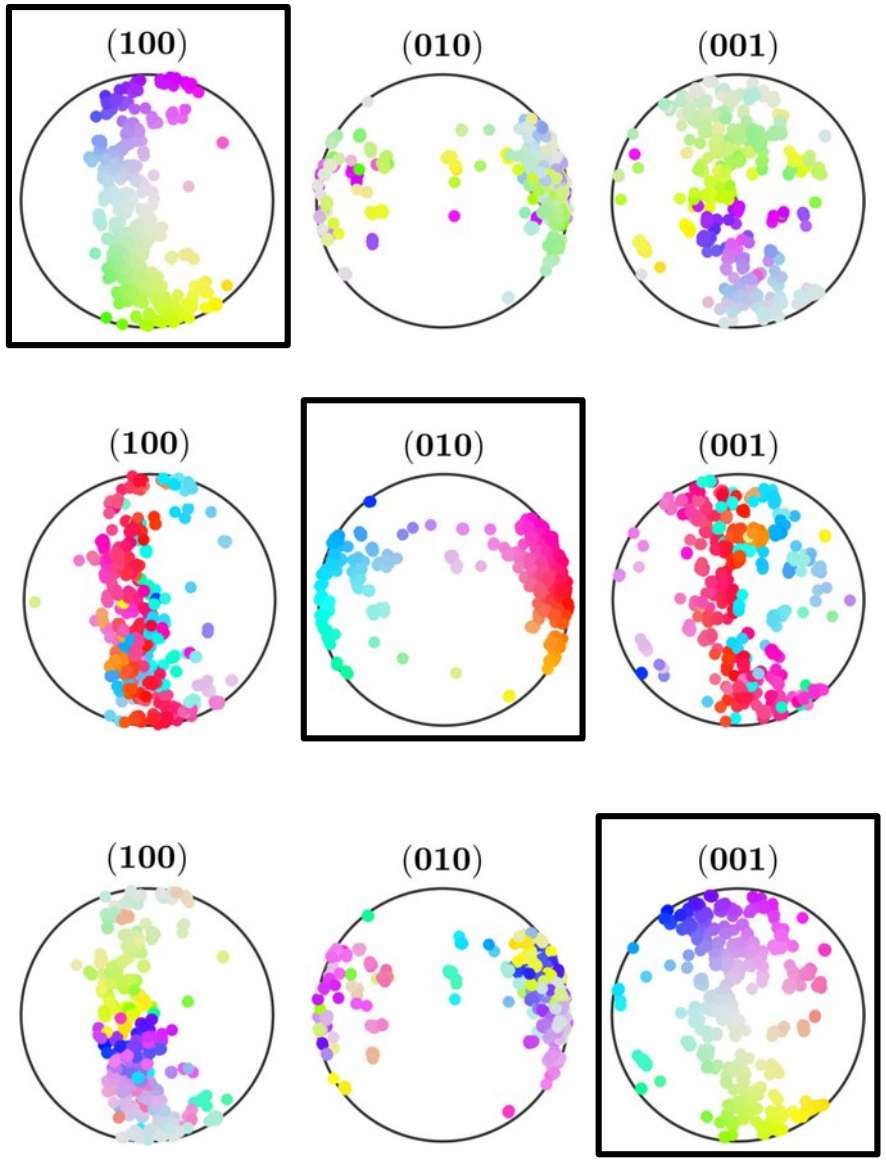


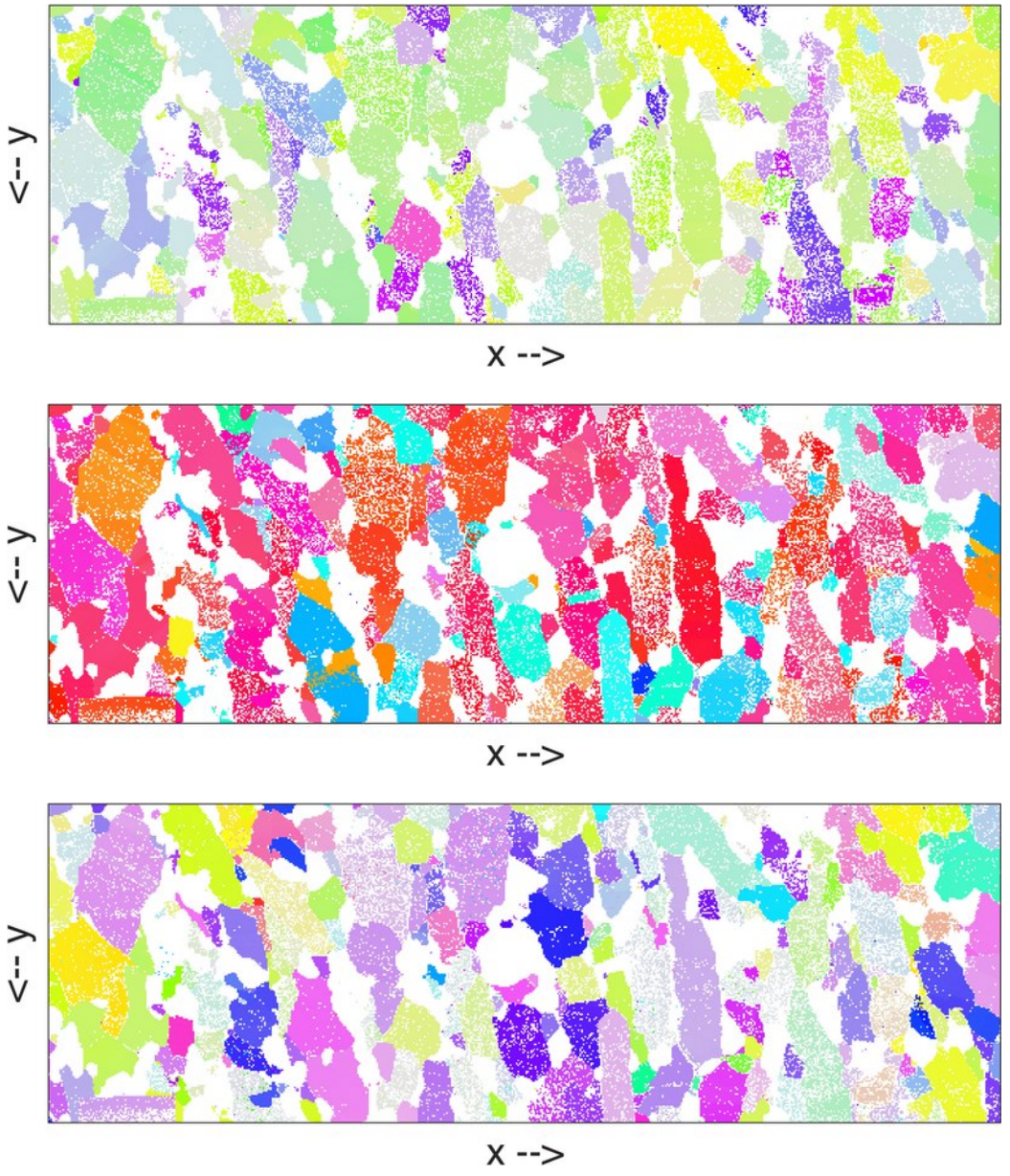




1. EBSD – plotting data

- direction key for color mappings of directions

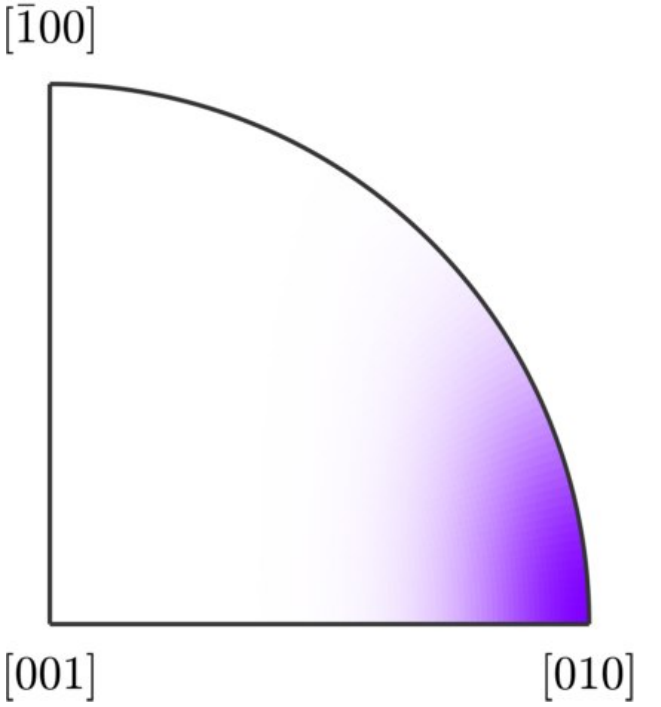


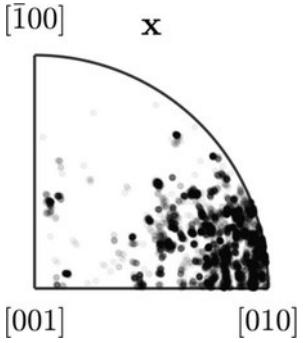


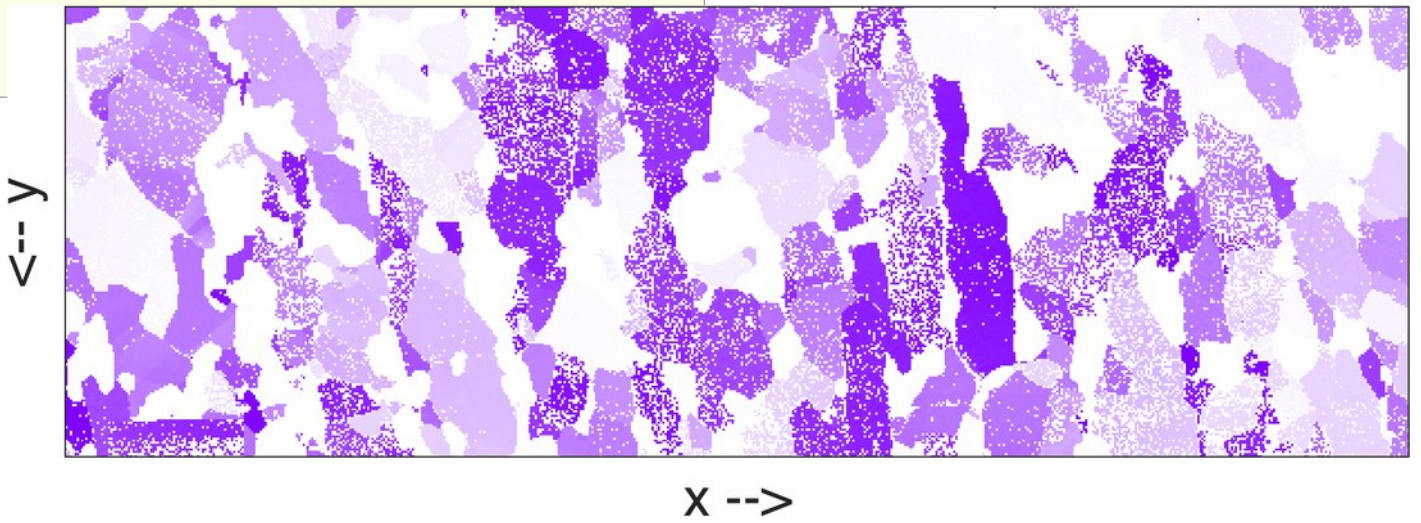
1. EBSD – plotting data

- spot color keys / coloring single directions

```
sk = ipfSpotKey(cs);
 ipfSpotKey with properties:
                       center: [1×1 Miller]
                        color: [1 0 0]
                          psi: [1×1 S2DeLaValleePoussinKernel]
  inversePoleFigureDirection: [1×1 vector3d]
                       dirMap: [1×1 directionColorKey]
                          CS1: [1×1 crystalSymmetry]
                          CS2: [1×1 specimenSymmetry]
                    antipodal: 0

sk.center=Miller(0,1,0,cs);
sk.inversePoleFigureDirection=xvector;
sk.psi = S2DeLaValleePoussinKernel('halfwidth',20*degree);
```



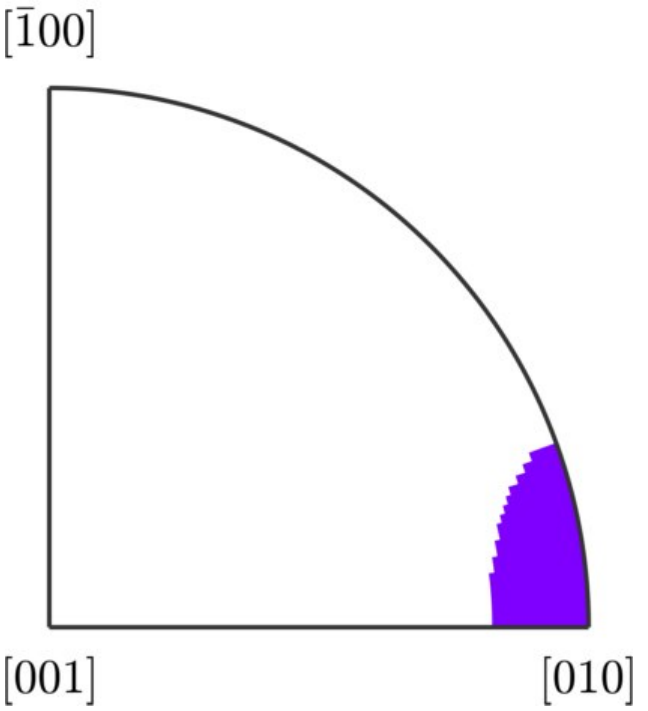


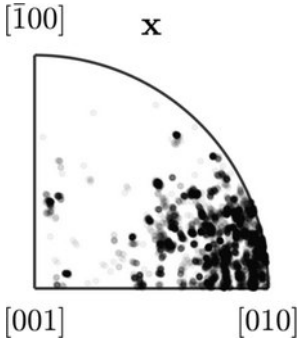


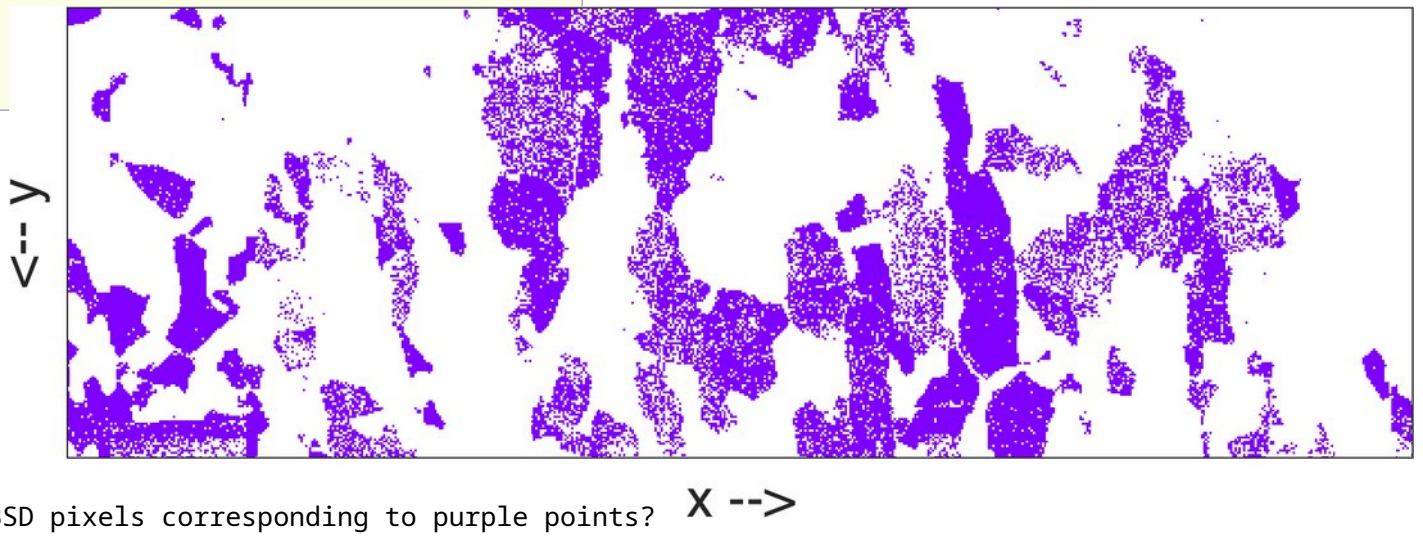
1. EBSD – plotting data

- spot color keys / coloring single directions

```
sk.center=Miller(0,1,0,cs);
sk.inversePoleFigureDirection=xvector;
sk.psi = S2BumpKernel('halfwidth',20*degree);
sk =
  ipfSpotKey with properties:

                        center: [1×1 Miller]
                         color: [0.5000 0 1]
                           psi: [1×1 S2BumpKernel]
    inversePoleFigureDirection: [1×1 vector3d]
                        dirMap: [1×1 directionColorKey]
                           CS1: [1×1 crystalSymmetry]
                           CS2: [1×1 specimenSymmetry]
                    antipodal: 0
```





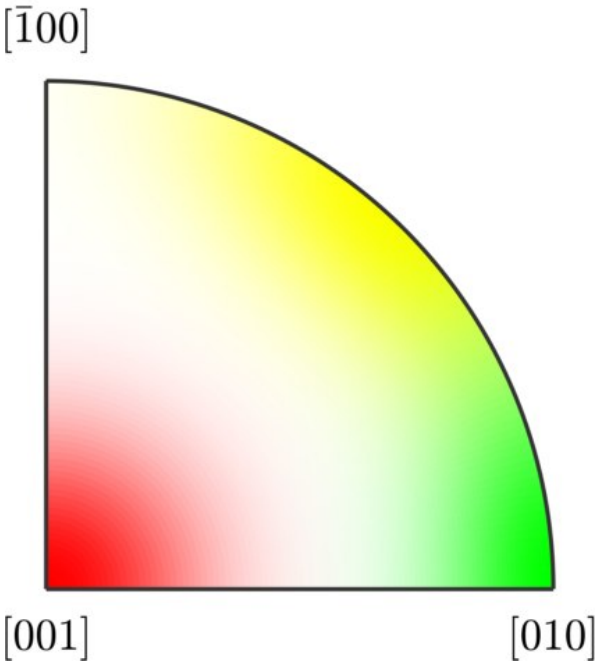


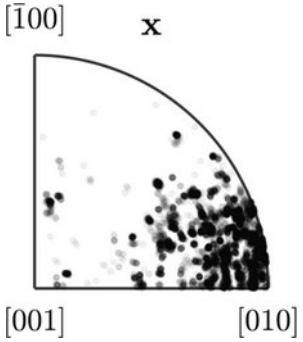
Q for the exercises: How do we find all EBSD pixels corresponding to purple points?

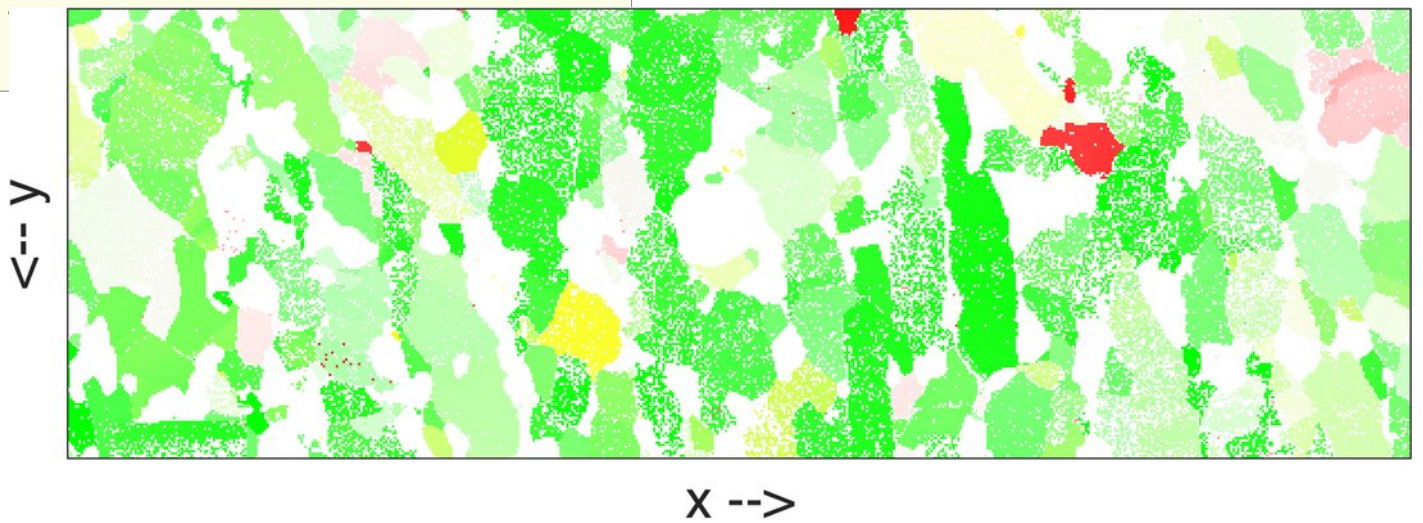
1. EBSD – plotting data

- spot color keys / coloring single directions

```
sk = ipfSpotKey(cs);

% one can have multiple spots
sk.center=Miller({0 0 1},{0 1 0},{1 2 0},cs)
sk.color=[1,0,0; 0 1 0; 1 1 0]

sk =

  ipfSpotKey with properties:

                      center: [1×3 Miller]
                       color: [3×3 double]
                         psi: [1×1
S2DeLaValleePoussinKernel]
                          ...
```



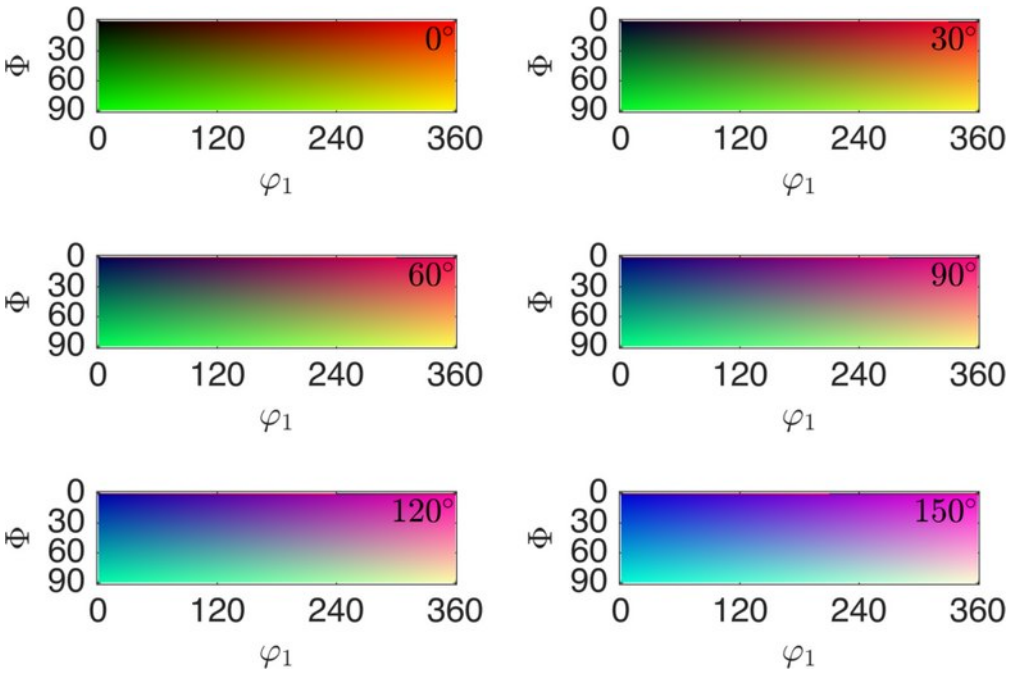


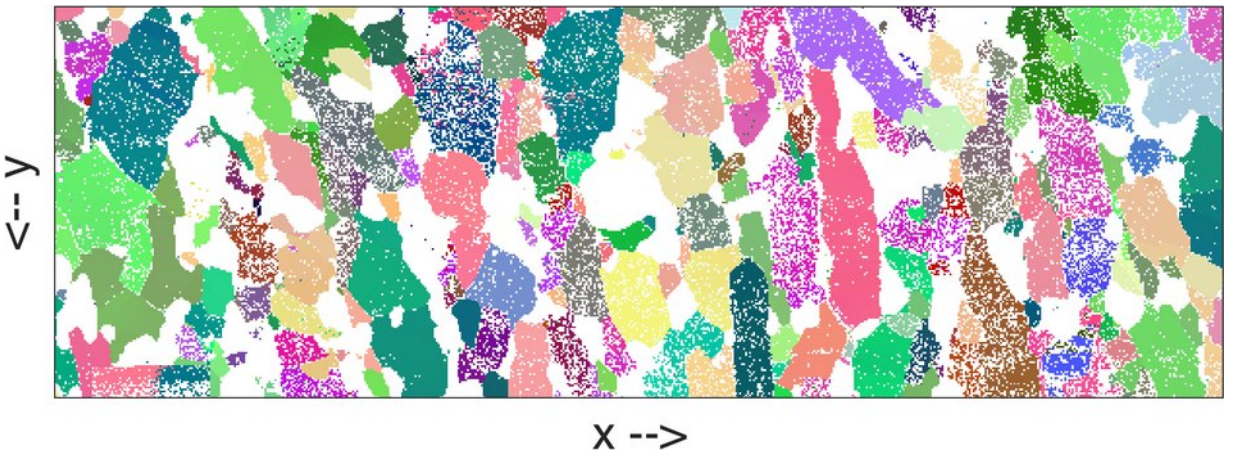


1. EBSD – plotting data

- coloring orientations: Euler angle color coding
- good: displays entire orientation
- bad: not human readable, distortion, eventually non-periodic

```
bk = BungeColorKey(cs)

bk =

  BungeColorKey with properties:

       center: [1×1 quaternion]
    phi1Range: [0 6.2832]
    phi2Range: [0 3.1416]
     PhiRange: [0 1.5708]
          CS1: [1×1 crystalSymmetry]
          CS2: [1×1 specimenSymmetry]
    antipodal: 0

color = bk.orientation2color(o);
```

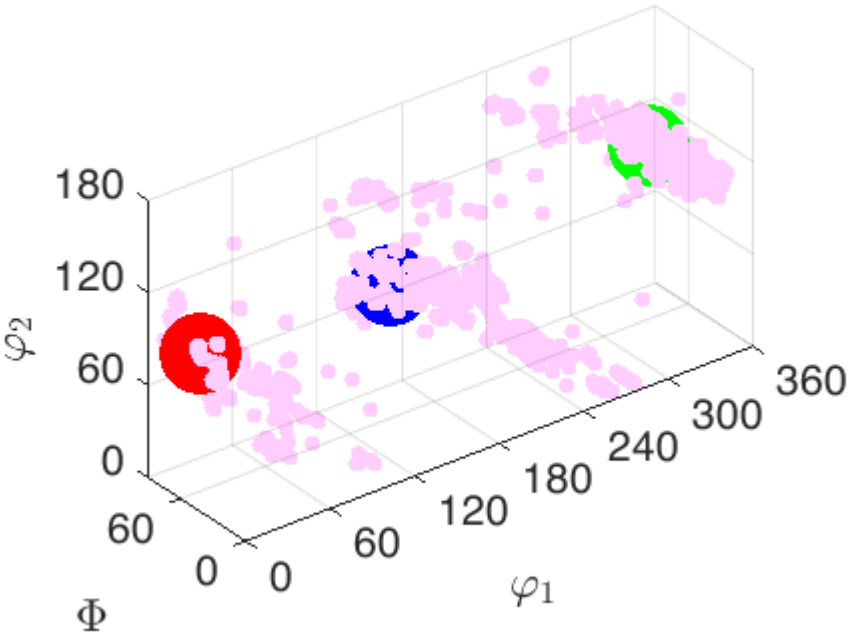




1. EBSD – plotting data

- coloring orientations: orientation spot color key
- highlight deviation reference orientation

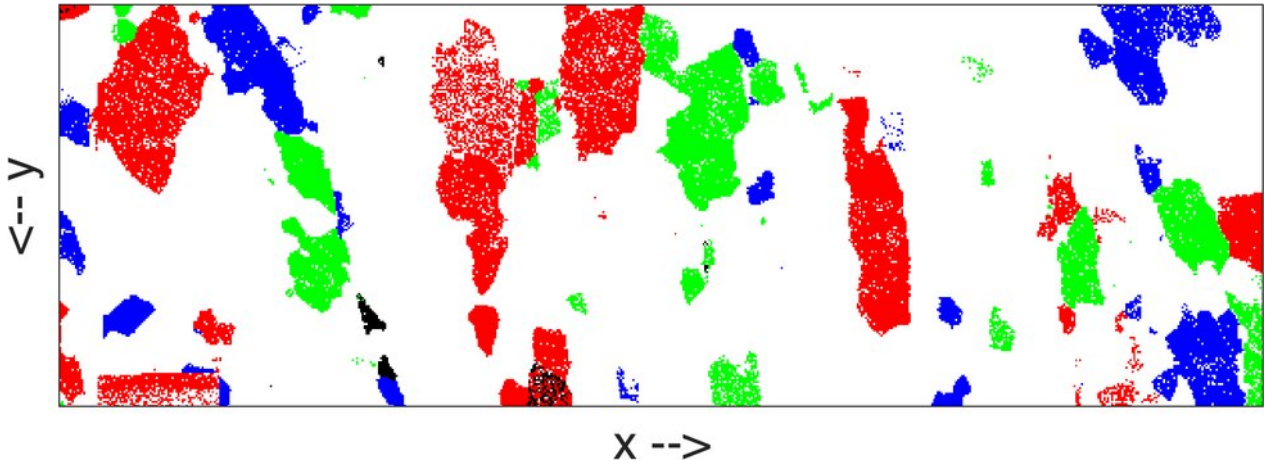
```
sk = spotColorKey(cs)

  spotColorKey with properties:

       center: [3×1 orientation]
        color: [3×3 double]
          psi: [1×1 SO3DeLaValleePoussinKernel]
          CS1: [1×1 crystalSymmetry]
          CS2: [1×1 specimenSymmetry]
    antipodal: 0

odf = calcDensity(o);
oref = max(odf,'numlocal',3)

>> oref
oref = orientation (show methods, plot)
  size: 3 x 1
  crystal symmetry : Forsterite (mmm)

  Bunge Euler angles in degree
     phi1     Phi    phi2    Inv.
  1.57038 43.9039 280.854       0
...
```

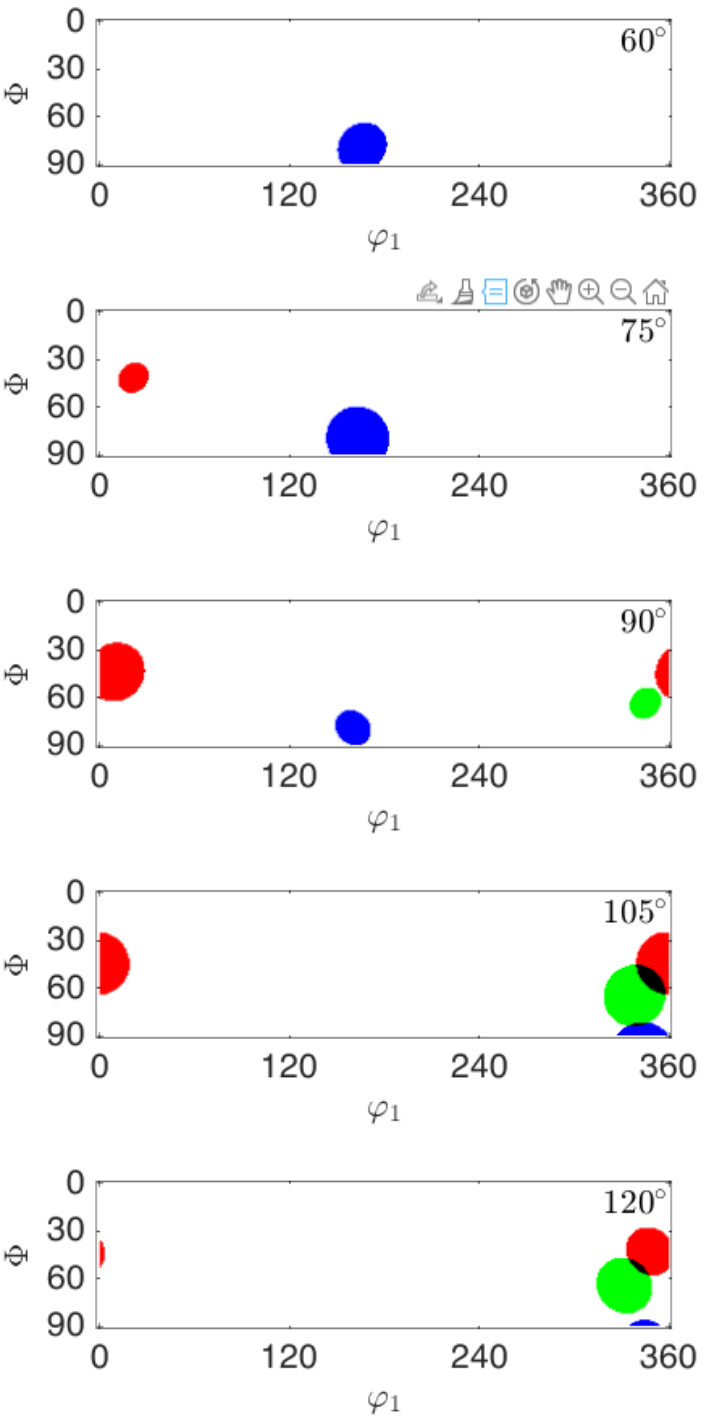


1. EBSD – plotting data

- coloring orientations: orientation spot color key
- highlight deviation reference orientation

```
sk = spotColorKey(cs)

odf = calcDensity(o);
oref = max(odf,'numlocal',3)

% set oref as local centers
sk.center = oref;
% set colors for each center
sk.color = [1 0 0; 0 1 0; 0 0 1]
% set a kernel for the spot
sk.psi = SO3BumpKernel('halfwidth',20*degree)
```

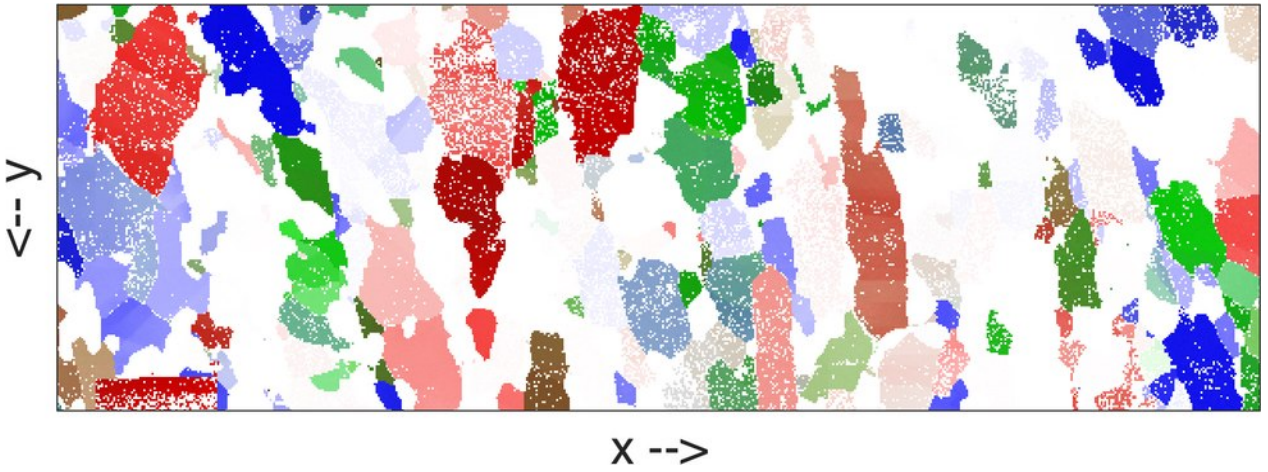


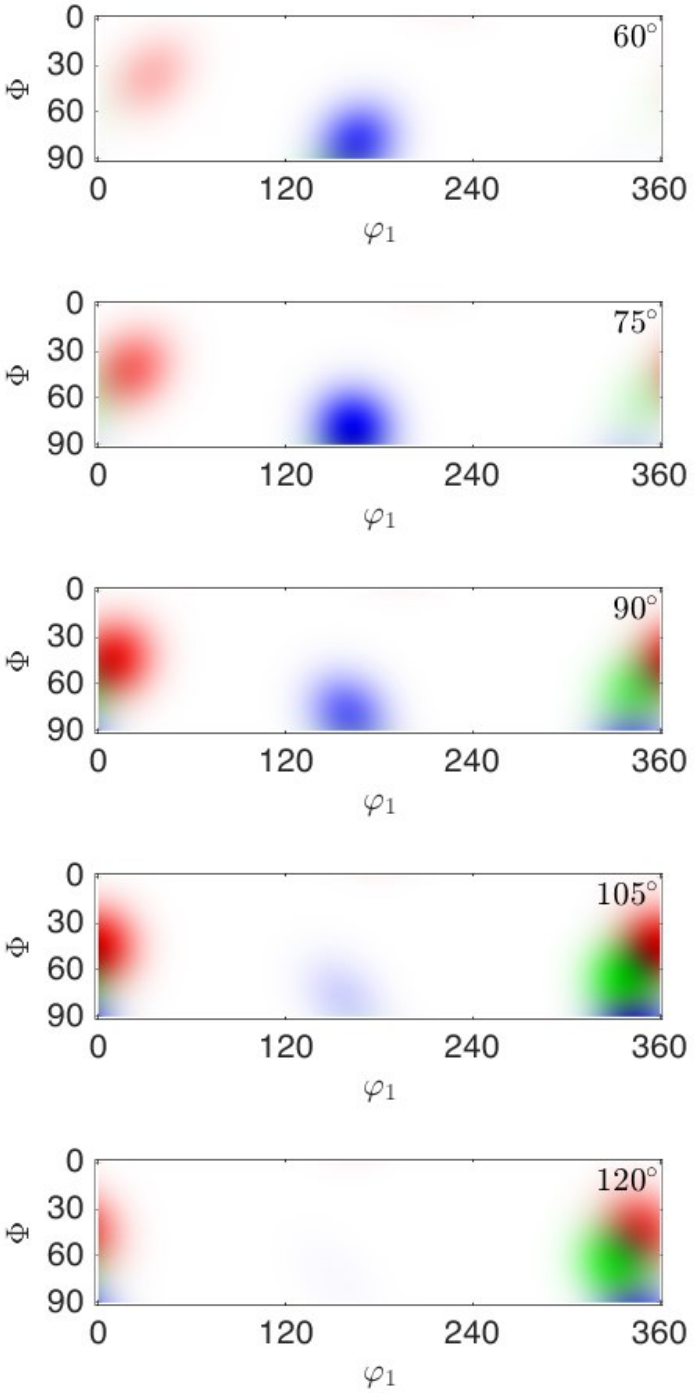


1. EBSD – plotting data

- coloring orientations: orientation spot color key
- highlight deviation reference orientation

```
sk = spotColorKey(cs)

odf = calcDensity(o);
oref = calcModes(odf,'numlocal',3)

% set oref as local centers
sk.center = oref;
% set colors for each center
sk.color = [1 0 0; 0 1 0; 0 0 1]
% set a kernel for the spot
sk.psi = ...
SO3DeLaValleePoussinKernel('halfwidth',20*degree)
```

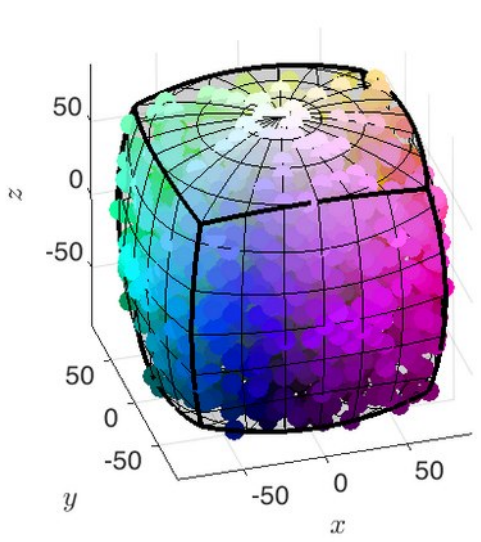


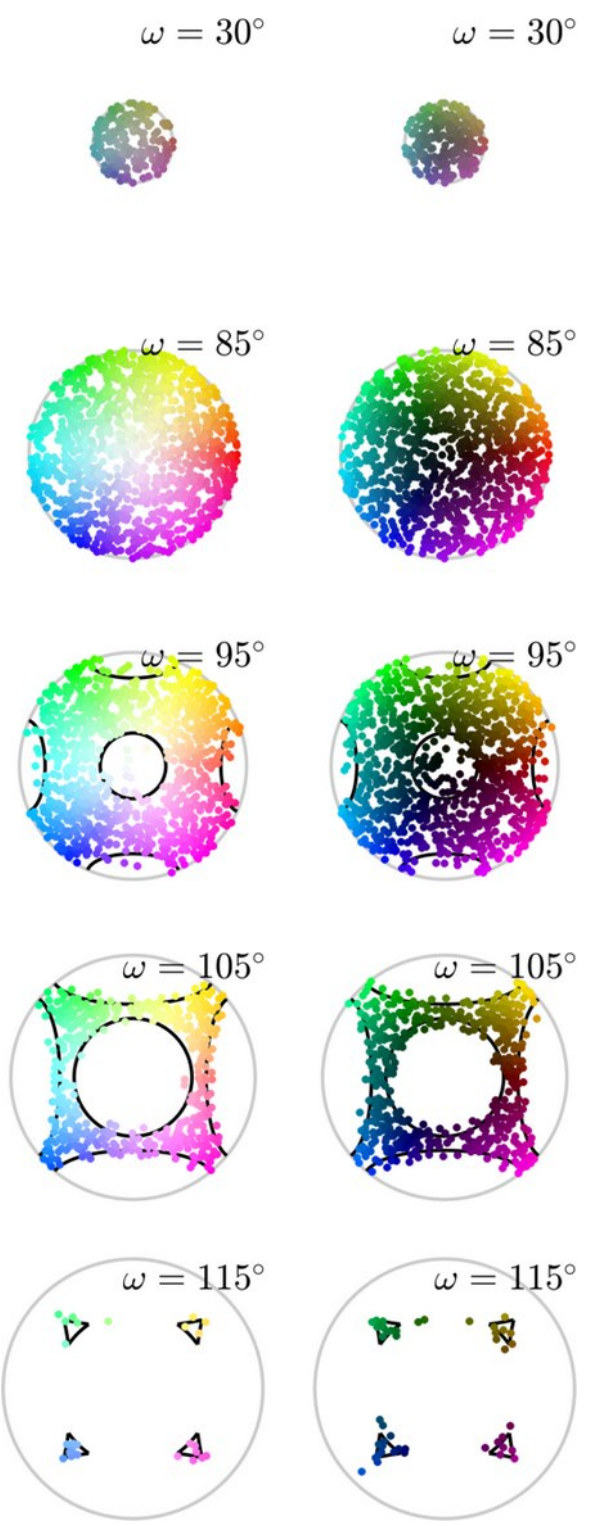


1. EBSD – plotting data

- axis angle color key
- colorize orientations in orientation space

```
ak = axisAngleColorKey(cs)
ak =

  axisAngleColorKey with properties:

        dirMapping: [1×1 HSVDirectionKey]
            oriRef: []
          maxAngle: Inf
     thresholdAngle: Inf
               CS1: [1×1 crystalSymmetry]
               CS2: [1×1 specimenSymmetry]
          antipodal: 0
```

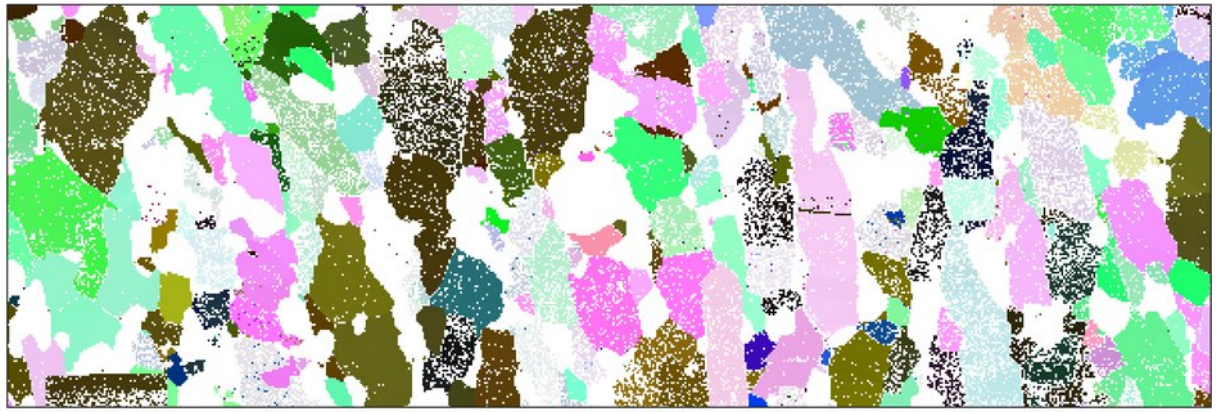


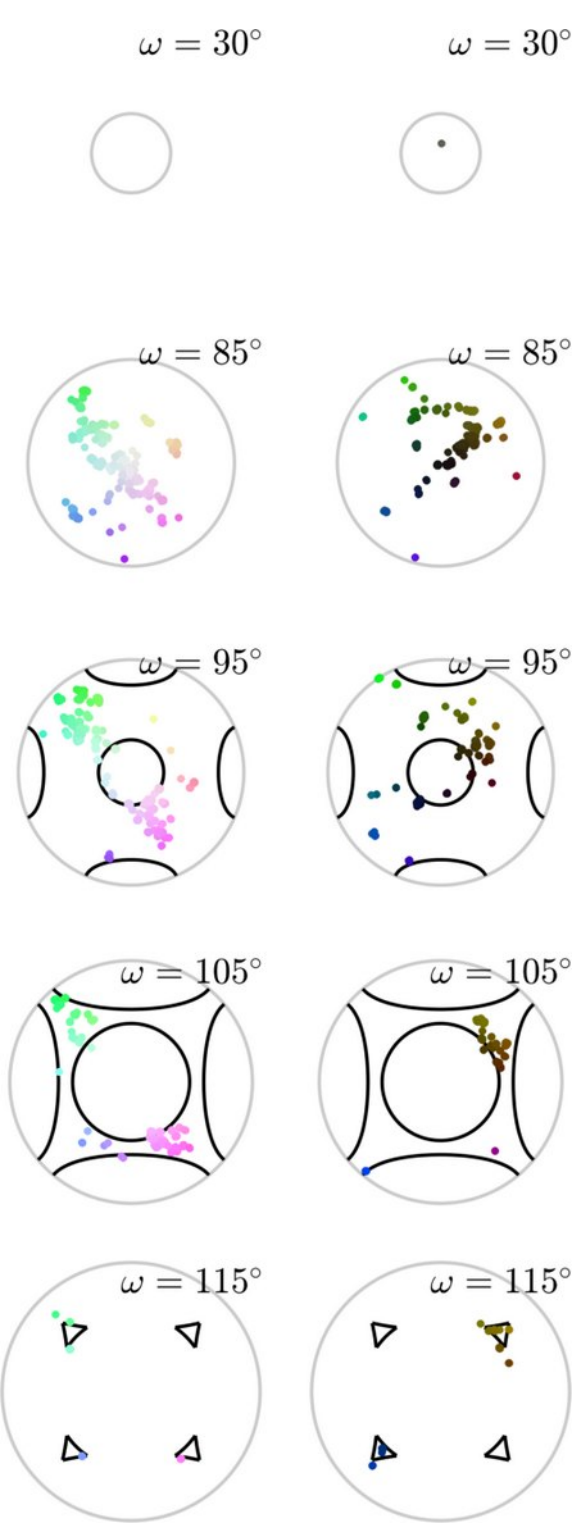


1. EBSD – plotting data

- axis angle color key
- colorize orientations in orientation space

```
ak = axisAngleColorKey(cs)
ak =

  axisAngleColorKey with properties:

        dirMapping: [1×1 HSVDirectionKey]
            oriRef: []
          maxAngle: Inf
     thresholdAngle: Inf
               CS1: [1×1 crystalSymmetry]
               CS2: [1×1 specimenSymmetry]
         antipodal: 0
```





1. EBSD – gridify -> EBSDsquare/hex

- EBSD data usually acquired on regular grid (square/hexagonal)
- data treated as a list by default
- gridify() converts 1D lists of measurements to 2D matrices
- faster for certain operations (e.g. gradient computation or plotting)

```
>> ebsd
 ebsd = EBSD
  Phase   Orientations     Mineral
     0  495637 (9.7%)  notIndexed
     1  1503511 (30%)  Quartz-new
     2  1700713 (33%)   Anorthite
     3  1390665 (27%)  Hornblende
 Properties: x, y, bc, bs, bands, ...
 Scan unit : um

>>ebsd.prop
ans =
  struct with fields:

          x: [5090526×1 double]
          y: [5090526×1 double]
         bc: [5090526×1 double]
      ...
```

```
>> ebsdg = ebsd.gridify
 ebsdg = EBSDsquare
  Phase   Orientations     Mineral
     0  495637 (9.7%)  notIndexed
     1  1503511 (30%)  Quartz-new
     2  1700713 (33%)   Anorthite
     3  1390665 (27%)  Hornblende
  Properties: x, y, bc, bs, bands, ...
 Scan unit : um
 Grid size (square): 1206 x 4221

>> ebsdg.prop
ans =
  struct with fields:

          x: [1206×4221 double]
          y: [1206×4221 double]
         bc: [1206×4221 double]
      ...
```

1. EBSD – gridify -> EBSDsquare/hex

- EBSD data usually acquired on regular grid (square/hexagonal)
- data treated as a list by default
- gridify() converts 1D lists of measurements to 2D matrices
- faster for certain operations (e.g. gradient computation or plotting)

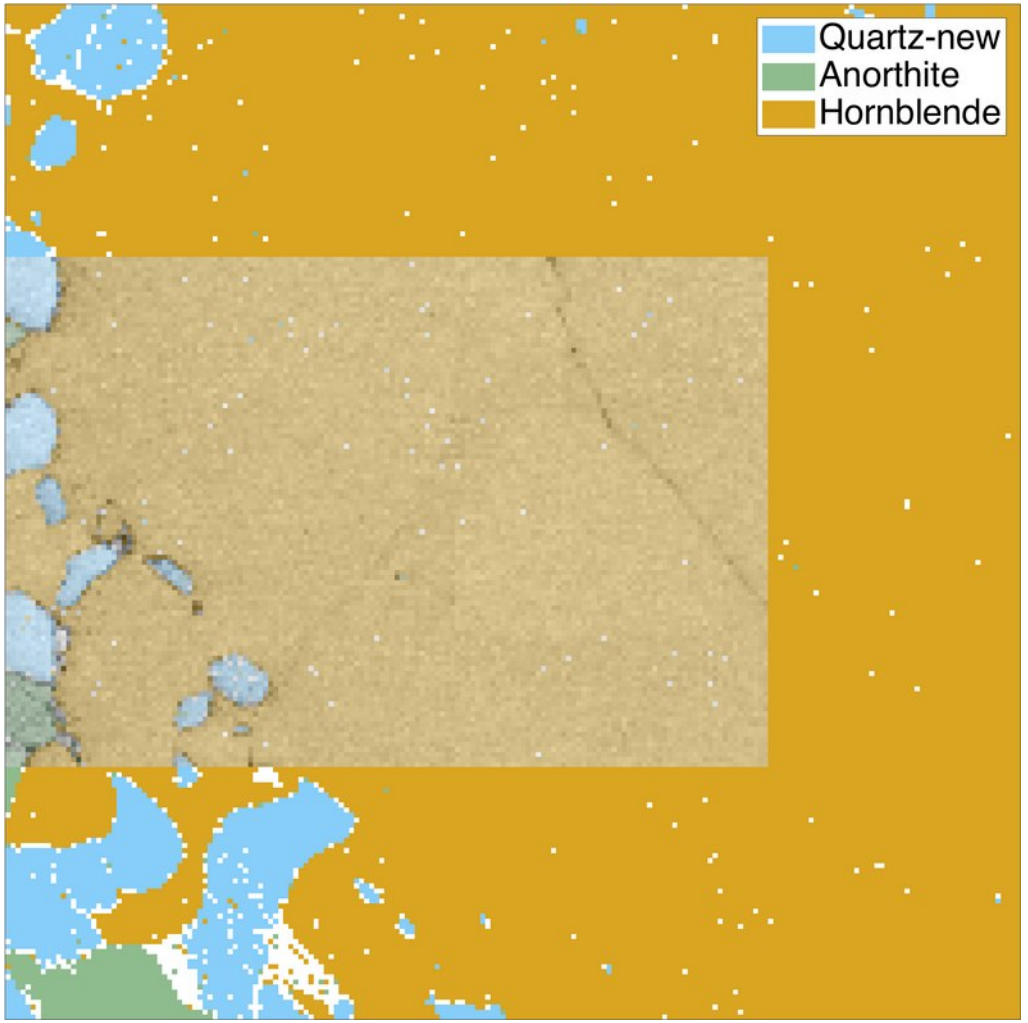
```
>> ebsdg = ebsd.gridify
 ebsdg = EBSDsquare
  Phase   Orientations     Mineral
     0  495637 (9.7%)  notIndexed
     1  1503511 (30%)  Quartz-new
     2  1700713 (33%)   Anorthite
     3  1390665 (27%)  Hornblende
  Properties: x, y, bc, bs, bands, ...
 Scan unit : um
 Grid size (square): 1206 x 4221

>> ebsdg.prop
ans =
  struct with fields:
           x: [1206×4221 double]
           y: [1206×4221 double]
          bc: [1206×4221 double]
          bs: [1206×4221 double]
       bands: [1206×4221 double]
         ...
```

```
>> size(ebsd.prop.Al_Wt_)

ans =
      5090526           1

>> size(ebsd.prop.Al_Wt_)

ans =
       1206        4221
```

1. EBSDsquare

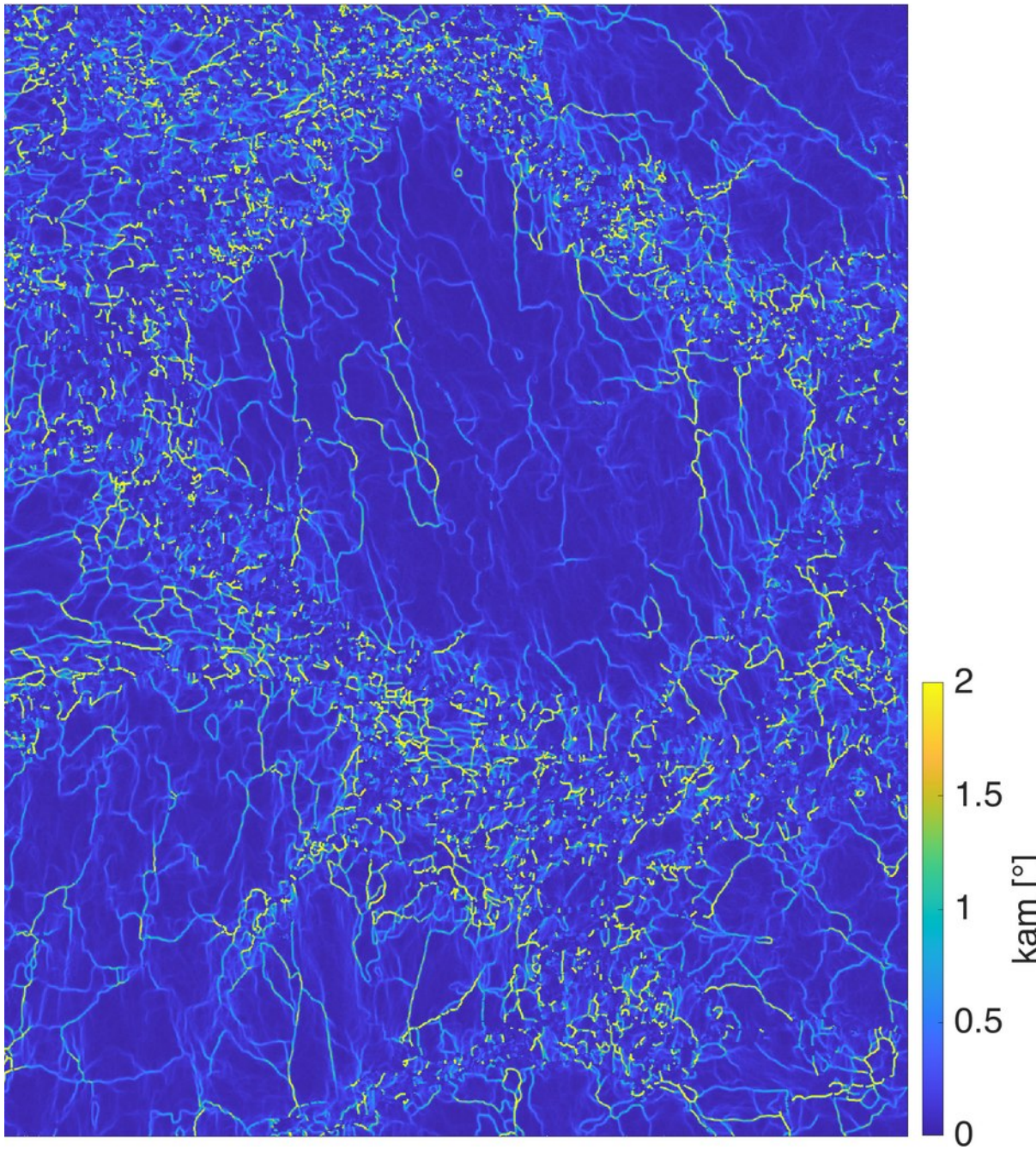
- gridify() converts 1D lists of measurements to 2D matrices
- faster for certain operations (e.g. gradient computation or plotting)
- gridded data can be selected by ebsd(rows,columns)
- rearrange hex data on a square grid

```
plot(ebsdg(1000:1200,1000:1200))
hold on
plot(ebsdg(1050:1150,1000:1150), ...
     ebsdg(1050:1150,1000:1150).bc, ...
     'FaceAlpha',0.6)
hold off
mtexColorMap black2white
```



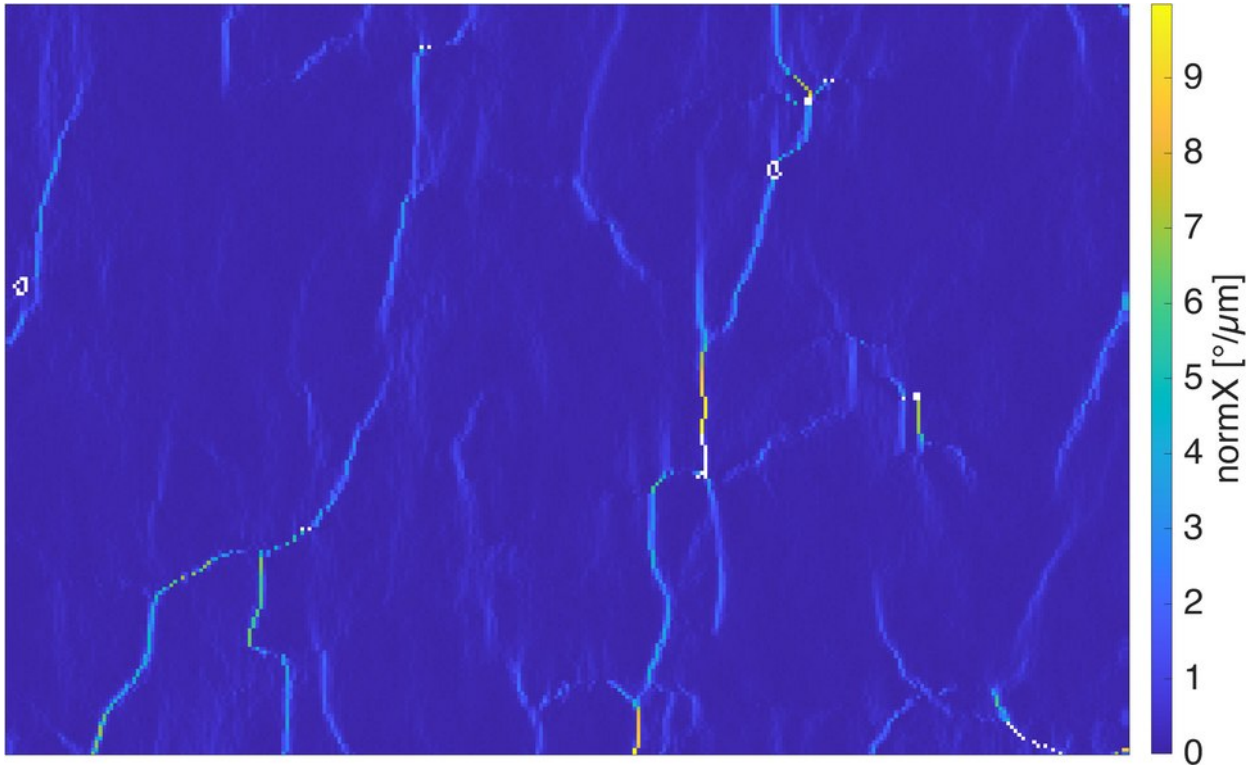
1. EBSDsquare – speed

```
ebsd = EBSD
Phase    Orientations
    1  3063577 (100%)


opts = {'order',2,'threshold',10*degree}
tic
kam = ebsd.KAM(opt{:});
toc

Elapsed time is 22.380967 seconds.

tic
ebsd = ebsd.gridify;
kam  = ebsd.KAM(opt{:});
toc

Elapsed time is 1.743969 seconds.



plot(ebsd,kam/degree)
setColorRange([0  2])
```

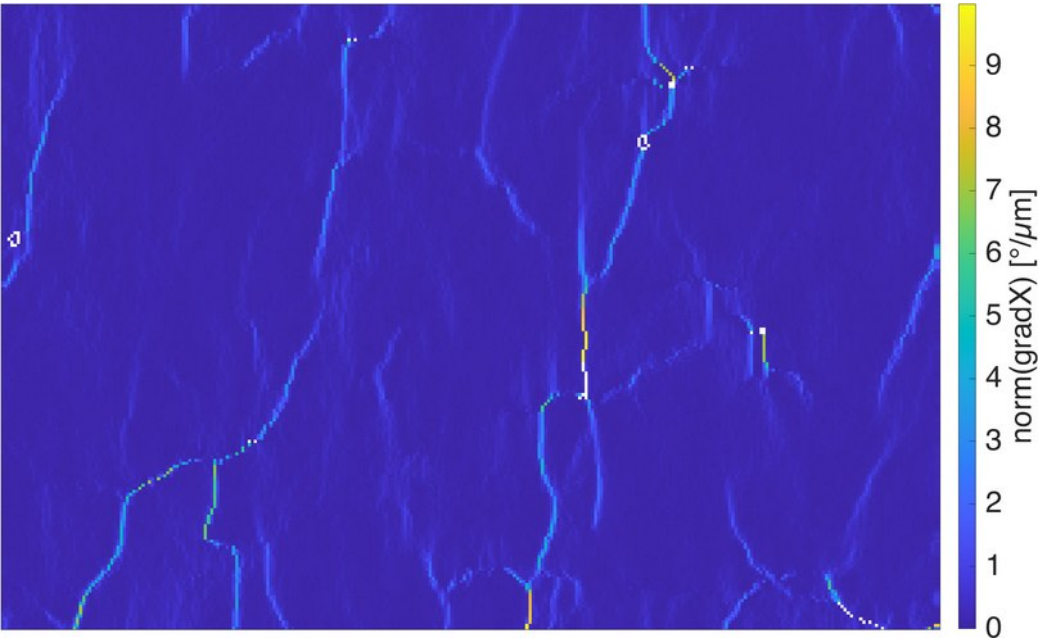


1. EBSDsquare – gradients

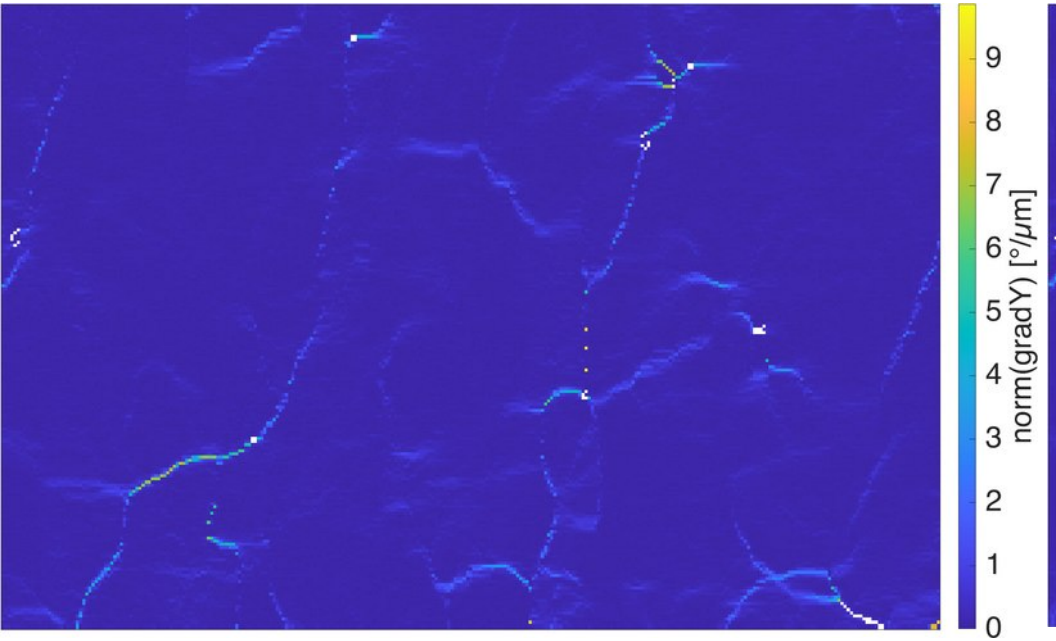
```
ebsd = ebsd.gridify

ebsd = EBSDsquare

 Phase  Orientations
     1  60501 (100%)
 ...
 Grid size (square): 201 x 301

gradX = ebsd.gradientX

gradX = vector3d
 size: 201 x 301

cond = norm(gradX) < 10*degree;
plot(ebsd(cond),norm(gradX(cond)))
```

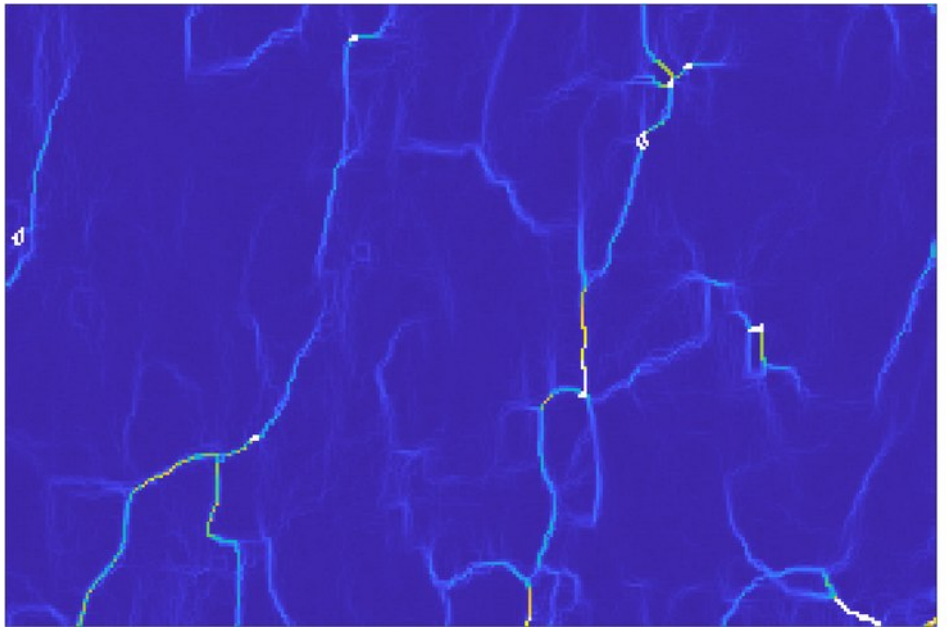


1. EBSDsquare – gradients



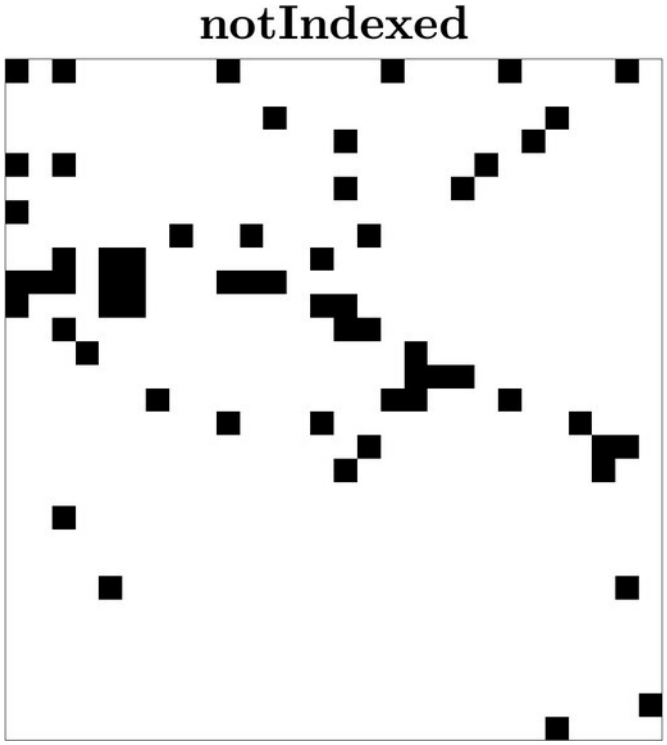

```
rGrad = sqrt(norm(gradX).^2 + norm(gradY).^2)
cond = rGrad < 10*degree;
plot(ebsd(cond), rGrad(cond)/degree)
```

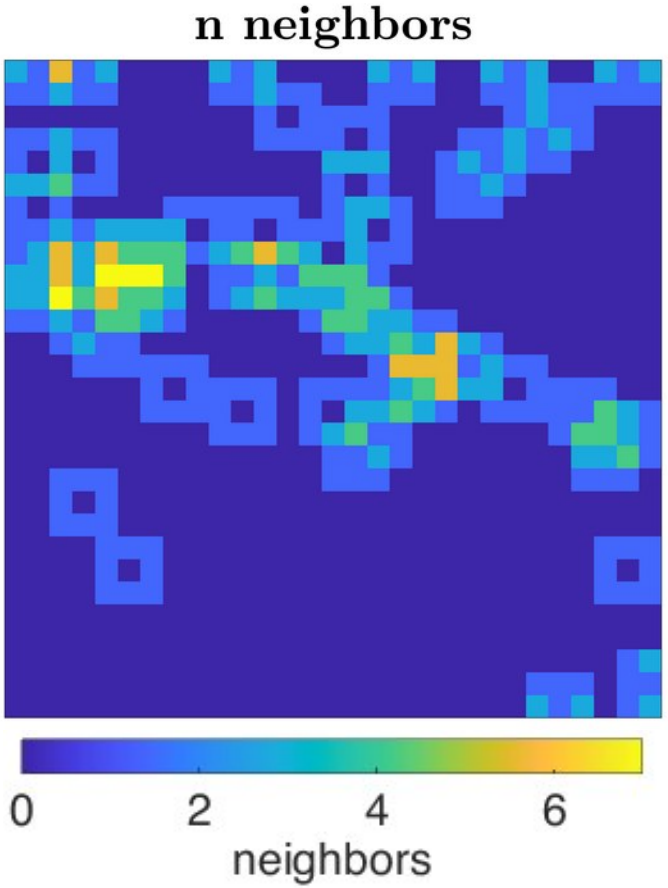


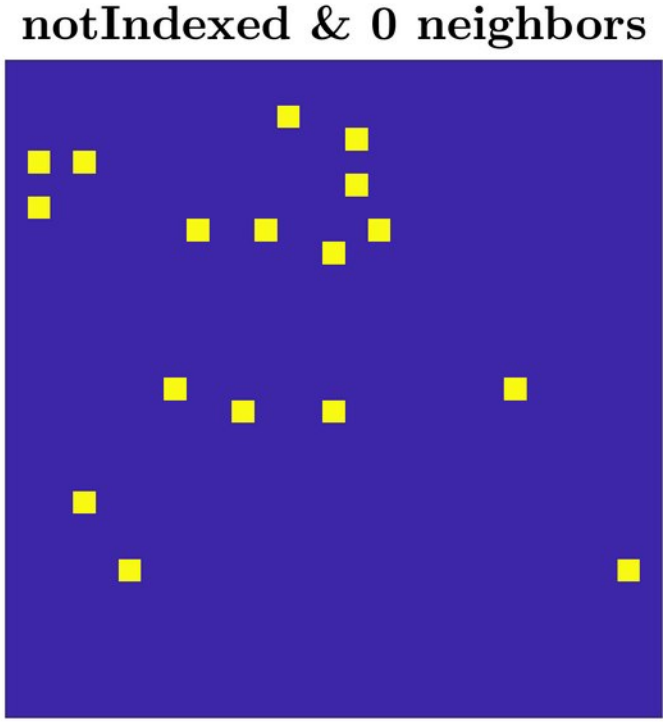


1. EBSDsquare: binary ranking filters

- allows to address neighboring pixels
- binary filtering, neighborhood operations e.g. erosion filters







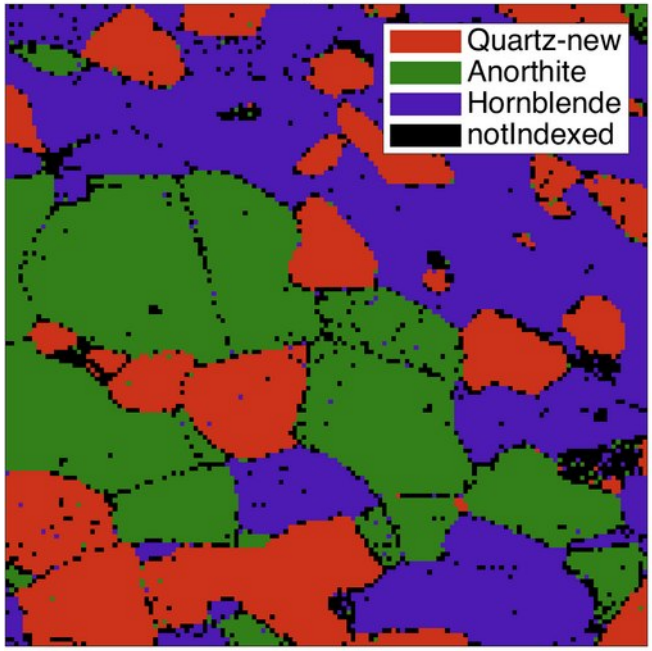

```
plot(ebsds('n'),'faceColor','k')
```

```
ind = squareNeighbors2(size(ebsd));
nNe = sum(~ebsd.isIndexed(ind),3);
plot(ebsds,nNe)
```

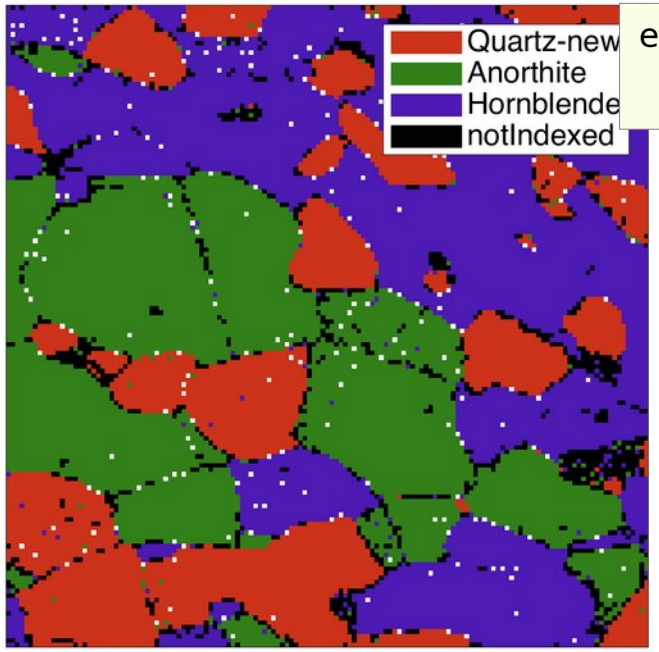
```
loner = ~ebsd.isIndexed & nNe == 0;
plot(ebsds,toerode)
```

1. EBSDsquare: binary filters

original



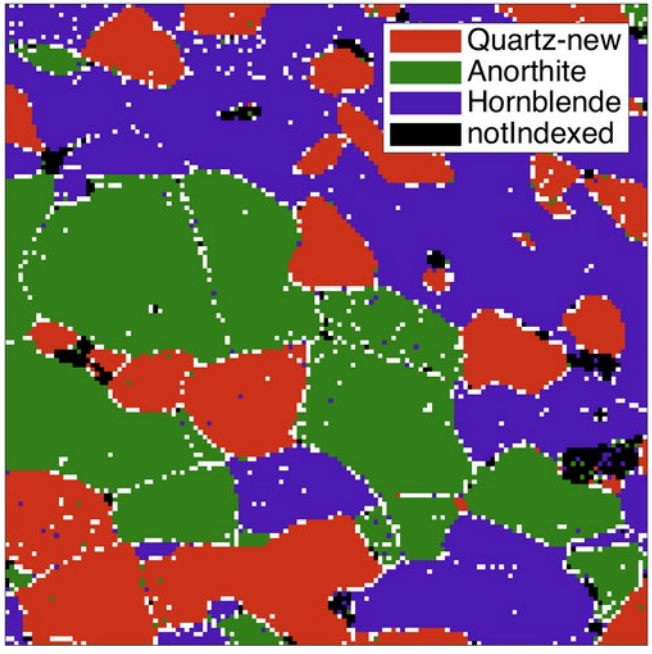
no notIndex with 0 neighbors



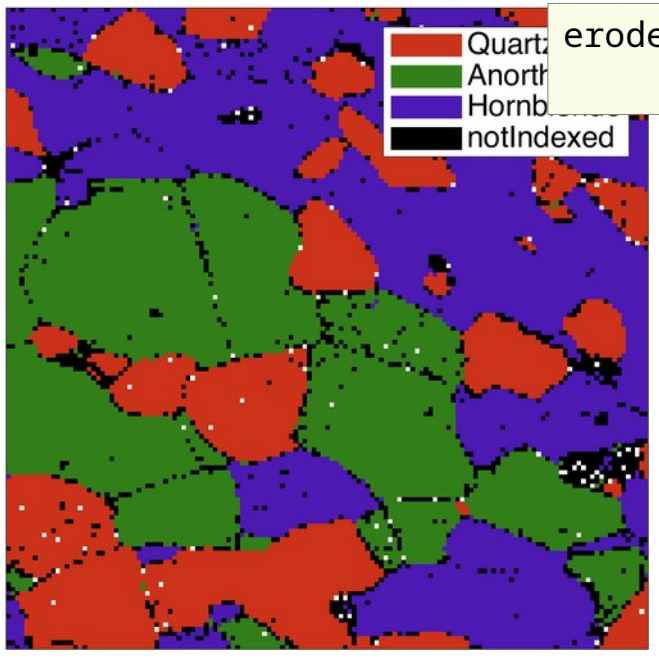

```
erode(ebsd,0)
```

no notIndex with 3 neighbors

```
erode(ebsd,3)
```

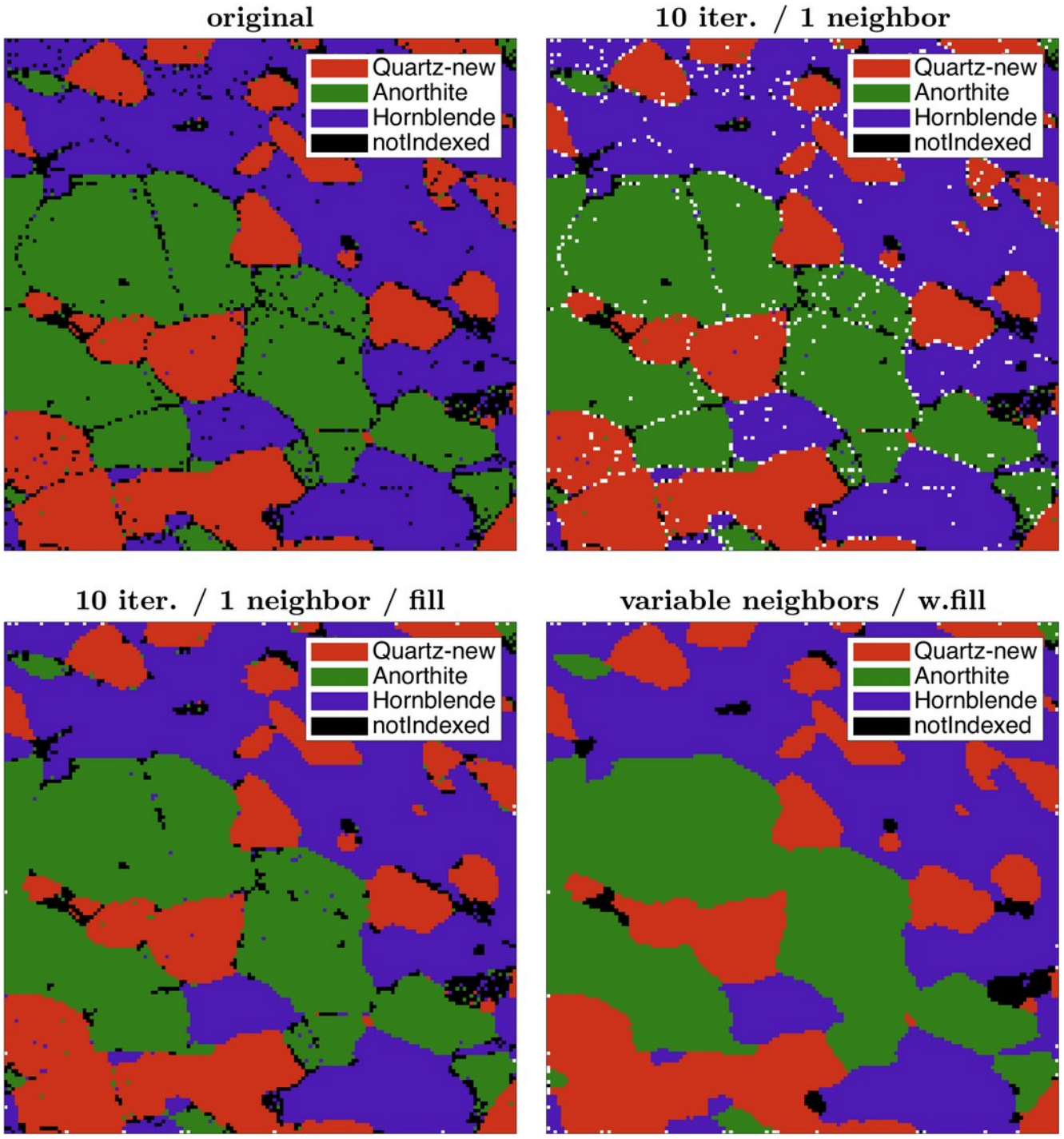


no Index with 0 neighbors



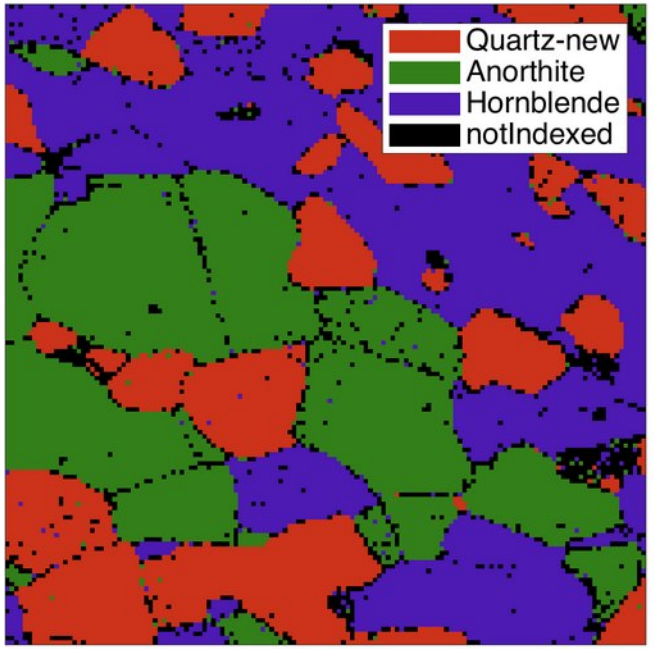

```
erode(ebsd,0,{'q' 'a' 'h'})
```

1. EBSDsquare: binary filters

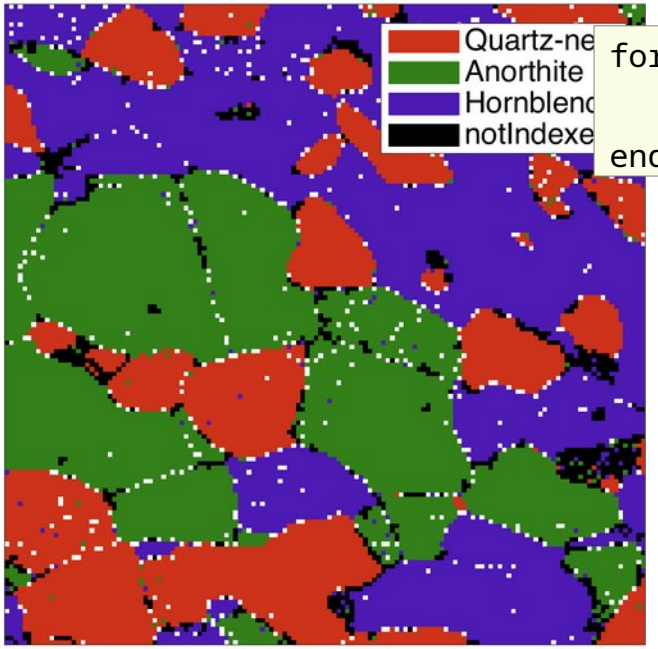


1. EBSDsquare: binary filters

original

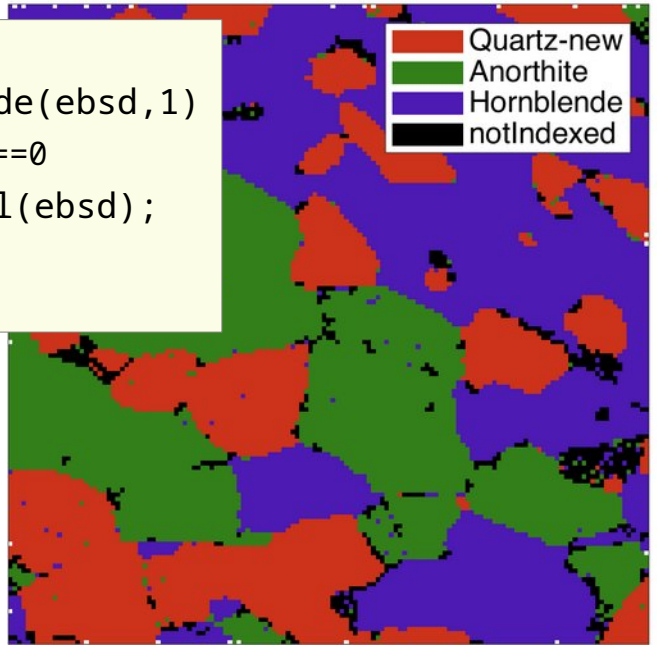


10 iter. / 1 neighbor



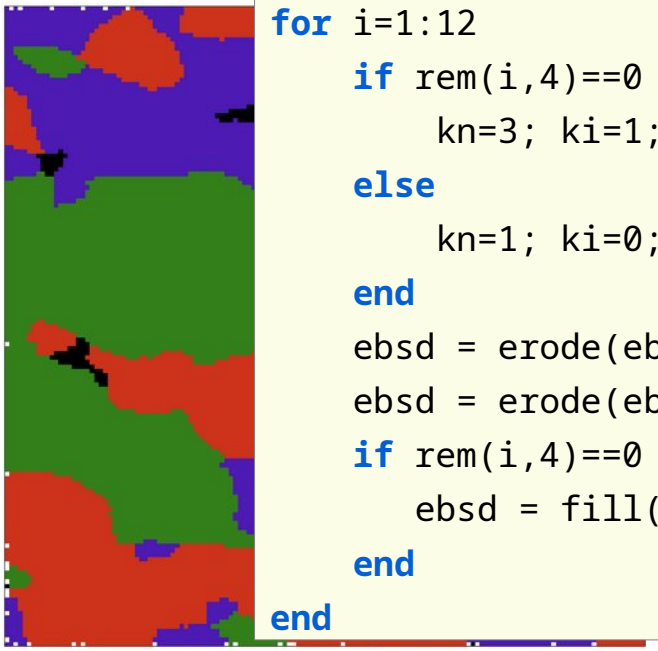

```
for i=1:10
    ebsd = erode(ebsd,1)
end
```

10 iter. / 1 neighbor / fill




```
for i=1:10
    ebsd = erode(ebsd,1)
    if rem(i,5)==0
    ebsd = fill(ebsd);
    end
end
```

variable neighbors / w.fill



```
for i=1:12
    if rem(i,4)==0
        kn=3; ki=1;
    else
        kn=1; ki=0;
    end
    ebsd = erode(ebsd,kn);
    ebsd = erode(ebsd,ki,{'q' 'a' 'h'});
    if rem(i,4)==0
       ebsd = fill(ebsd);
    end
end
```